

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО»

А. С. Момот, О. А. Повшенко

МІКРОКОНТРОЛЕРИ В СИСТЕМАХ НЕРУЙНІВНОГО КОНТРОЛЮ ЛАБОРАТОРНИЙ ПРАКТИКУМ

*Рекомендовано Методичною радою КПІ ім. Ігоря Сікорського
як навчальний посібник для студентів, які навчаються за спеціальністю
151 «Автоматизація та комп'ютерно-інтегровані технології»,
освітньою програмою «Роботизовані і автоматизовані системи неруйнівного
контролю та діагностики»*

Київ
КПІ ім. Ігоря Сікорського
2026

Рецензент *Прізвище, ініціали, науковий ступінь, вчене звання, посада, місце роботи*

Відповідальний редактор *Прізвище, ініціали, науковий ступінь, вчене звання*

*Гриф надано Методичною радою КПІ ім. Ігоря Сікорського (протокол № X від DD.MM.YYYY р.)
за поданням Вченої ради приладобудівного факультету (протокол № X від DD.MM.YYYY р.)*

Електронне мережне навчальне видання

*Момот Андрій Сергійович, доктор філософії, доцент
Повшенко Олександр Анатолійович, доктор філософії*

МІКРОКОНТРОЛЕРИ В СИСТЕМАХ НЕРУЙНІВНОГО КОНТРОЛЮ ЛАБОРАТОРНИЙ ПРАКТИКУМ

Мікроконтролери в системах неруйнівного контролю: лабораторний практикум. [Електронний ресурс]: навч. посіб. для студ. спеціальності G7 «Автоматизація, комп'ютерно-інтегровані технології та робототехніка», освітньої програми «Комп'ютерно-інтегровані технології та робототехніка» / А. С. Момот, В. Г. Баженов, О. А. Повшенко; КПІ ім. Ігоря Сікорського. – Електронні текстові дані (1 файл: 3,15 Мбайт). – Київ : КПІ ім. Ігоря Сікорського, 2026. – 77 с.

Навчальний посібник містить мету і завдання лабораторних практикумів, їх зміст та обсяг. Розглянуто основні положення, послідовність та порядок виконання роботи, наведено завдання для індивідуального опрацювання та самостійної роботи.

© А. С. Момот, О. А. Повшенко, 2026
© КПІ ім. Ігоря Сікорського, 2026

ЗМІСТ

1. Лабораторна робота №1. Робота з АЦП мікроконтролерів STM32	4
2. Лабораторна робота №2. Широтно-імпульсна модуляція	21
3. Лабораторна робота №3. Робота з інтерфейсом передачі даних UART.....	33
4. Лабораторна робота №4. Робота з інтерфейсом передачі даних SPI та технологією RFID.....	48
5. Лабораторна робота №5. Робота з інтерфейсом передачі даних I2C, OLED дисплеєм та модулем пульсоксиметру	70
6. Лабораторна робота №6. Робота зі стандартом бездротової передачі даних Bluetooth та годинником реального часу	94
7. Лабораторна робота №7. Робота з сенсорним TFT дисплеєм	114
8. Лабораторна робота №8. Розробка модуля інерційної навігації	143
9. Список рекомендованої літератури.....	171

Лабораторна робота №1

РОБОТА З АЦП МІКРОКОНТРОЛЕРІВ STM32

Мета: Ознайомитись з особливостями роботи АЦП мікроконтролерів STM32 та порядком підключення аналогових датчиків. Навчитись створювати програмний код для обробки оцифрованих сигналів.

Теоретичні відомості

Одним з найважливіших блоків для мікроконтролера є аналого-цифровий перетворювач (АЦП), (англ. Analog-to-Digital Converter, ADC). Він дозволяє перетворювати аналогові сигнали в цифрові коди, які в подальшому можуть оброблятися процесором. Суть перетворення зводиться до порівняння аналогового сигналу з опорною напругою і формування числового значення, яке відповідає коду вимірюваної напруги в діапазоні розрядності АЦП. Наприклад, 12-розрядний АЦП з опорною напругою 3,3 В перетворює аналогові сигнали від 0 до 3,3 В до діапазону цифрових значень від 0 до 4095.

За допомогою АЦП мікроконтролер здатний вирішувати набагато більше завдань для автоматизації процесів. Крім того, у разі наявності вбудованого в мікроконтролер АЦП істотно спрощується загальна схема пристрою і значно зменшується його вартість. Як правило, окрім самого АЦП, мікроконтролер має вбудований аналоговий мультиплексор, який дозволяє збільшити кількість аналогових входів шляхом почергового сканування декількох виводів мікроконтролера і подальшого перетворення сигналів від цих виводів.

Мікроконтролери серії STM32 також мають вбудований 12-розрядний АЦП послідовного наближення з тактовою частотою до 14 МГц. Деякі моделі мають кілька блоків АЦП. АЦП має 16 аналогових каналів і 2 внутрішніх. Зовнішні канали підключені до виводів мікроконтролера. Перед використанням цих виводів в якості аналогових необхідно налаштувати їх як аналогові входи.

Аналого-цифровий перетворювач **послідовного наближення** вимірює величину вхідного сигналу, здійснюючи ряд послідовних «зважувань», тобто порівнянь величини вхідної напруги з рядом величин, які генеруються за наступним принципом (рис. 1.1):

1. На першому етапі на виході вбудованого цифро-аналогового перетворювача встановлюється величина, яка дорівнює $\frac{1}{2} U_{\text{ref}}$ (вважається, що сигнал знаходиться в діапазоні $(0 \div U_{\text{ref}})$).
2. Якщо сигнал більше цієї величини, то він порівнюється з напругою, яка лежить посередині інтервалу, що залишився, тобто, в даному випадку, $\frac{3}{4} U_{\text{ref}}$. Якщо сигнал менше встановленого рівня, то таке порівняння буде проводитися з меншою половиною інтервалу, який залишився (тобто з рівнем $\frac{1}{4} U_{\text{ref}}$).
3. Крок 2 повторюється N раз. Таким чином, N порівнянь («зважувань») породжує N біт результату.

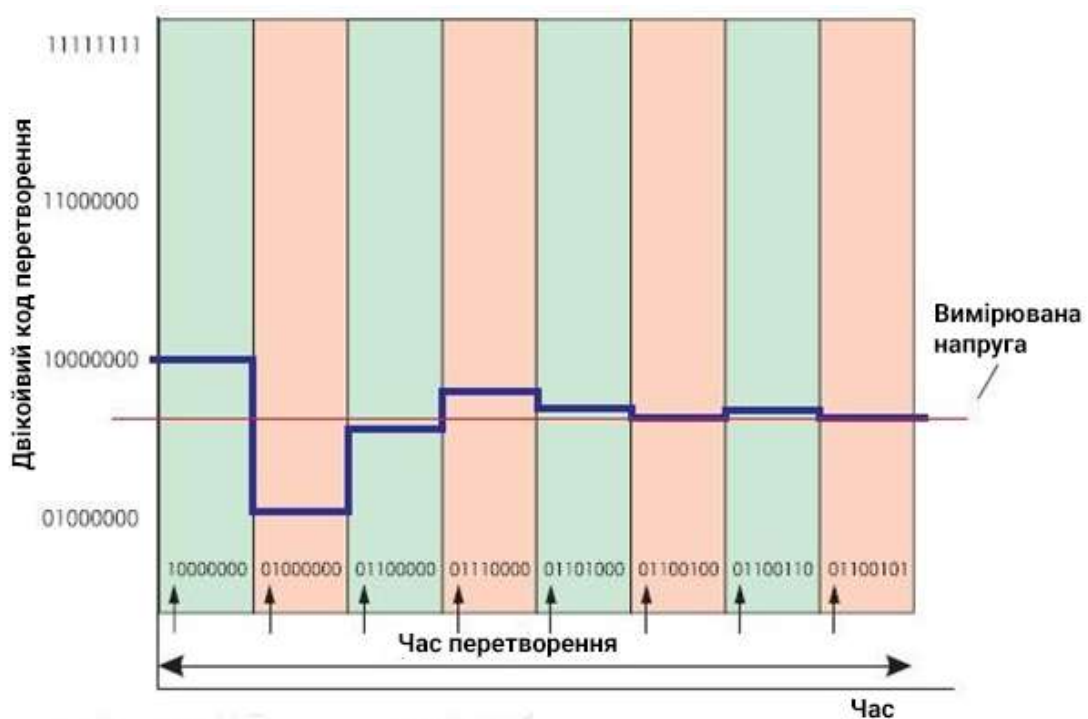


Рис. 1.1. Принцип роботи АЦП послідовного наближення

АЦП послідовного наближення складається з наступних вузлів (рис. 1.2):

1. Компаратор – порівнює вхідну величину і поточне значення «вагової» напруги (на рис. 1.2 позначений трикутником).

2. Цифро-аналоговий перетворювач (Digital to Analog Converter, DAC). Він генерує «вагове» значення напруги на основі цифрового коду, який надходить на вхід.
3. Регістр послідовного наближення (Successive Approximation Register, SAR). Він здійснює алгоритм послідовного наближення, генеруючи поточне значення коду, який подається на вхід ЦАП.
4. Схема вибірки/зберігання (Sample/Hold, S/H). Для роботи даного АЦП принципово важливо, щоб вхідна напруга зберігала незмінну величину протягом всього циклу перетворення. Дана схема її запам'ятовує.

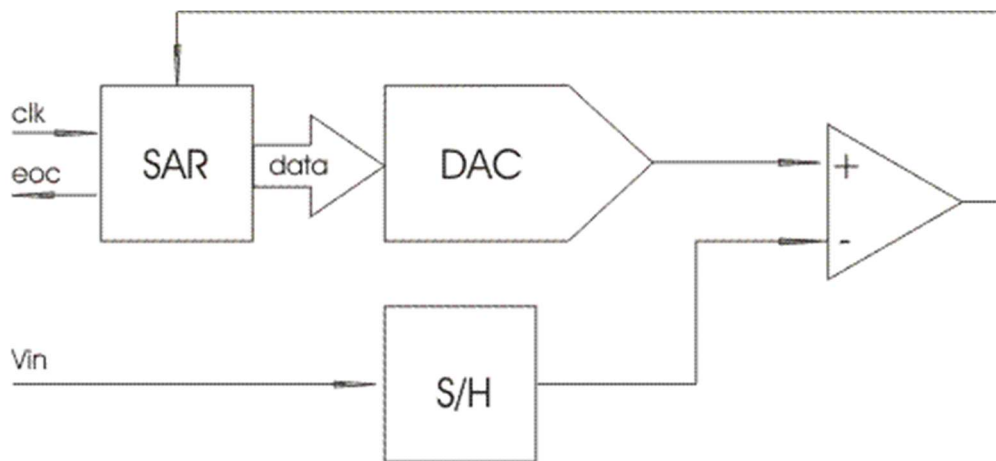


Рис. 1.2. Функціональна схема АЦП послідовного наближення

У початковий момент часу вимірювання до схеми вибірки та зберігання передається вимірювана напруга. Компаратор визначає, що величина напруги на виході схеми вибірки і зберігання більше напруги на виході ЦАП, і передає відповідний сигнал до регістру послідовного наближення. Регістр послідовного наближення виробляє імпульс, який подає на нижній вхід компаратора половину від найбільшої опорної напруги. Залежно від результату порівняння компаратором вхідних напруг, схема регістру послідовного наближення збільшує або зменшує напругу на виході ЦАП на чверть від найбільшої напруги. Така процедура повторюється N разів. Після кроку N напруга на виході ЦАП скидається, а регістр послідовного наближення формує запит на переривання ЕОС (end of conversion) та передає результат перетворення до головної програми.

Мікроконтролер STM32F103C8T6 містить 2 однотипних АЦП. Використовуючи режим здвоєних перетворень його можна перетворити в один АЦП з більш високою швидкістю перетворення. **Основні характеристики:**

- АЦП послідовного перетворення;
- Розрядність – 12 біт;
- Мінімальний час перетворення – 1 мкс;
- До 16 каналів зовнішніх аналогових сигналів, по одному каналу для вбудованого датчика температури і джерела опорного напруги. У STM32F103C8T6 10 каналів для зовнішніх сигналів.
- АЦП вимірює напругу в межах 0 ... 3,3 В;
- Є режим автокалібрування;
- Має багато режимів одноразових і циклічних перетворень з запуском від різних джерел, в тому числі і зовнішніх.

Зазвичай в мікроконтролерах модуль вимірювання аналогових сигналів являє собою АЦП та комутатор входних сигналів у вигляді мультиплексора. В такому випадку для вимірювання напруги на зовнішніх виводах необхідно виконати наступні дії:

- Встановити комутатор на потрібний входний канал;
- Запустити аналого-цифрове перетворення;
- Дочекатися його закінчення, часовою затримкою або перевіркою стану прапорця готовності;
- Зчитати результат напряму або в режимі обробки переривання.

В STM32 використовується складний автомат управління АЦП, який дозволяє проводити вимірювання аналогових сигналів в автоматичному режимі. Автомат управління модулем АЦП по заданій послідовності обробляє аналогові канали. За способом опитування канали діляться на 2 групи: регулярні і інжектовані:

- **Регулярні канали** (*regular channels*) призначені для обробки вхідних сигналів по черзі з певною періодичністю. Тому вони й називаються регулярними.
- У разі запуску опитування **інжекттованих каналів** (*injected channels*), обробка регулярних каналів призупиняється. Інжектувати (*inject*) означає вводити, впорскувати, вставляти. Інжекттовані канали опитуються між регулярними.

Частина функціональної схеми АЦП, на якій показані основні компоненти модуля, зображена на рис. 1.3.

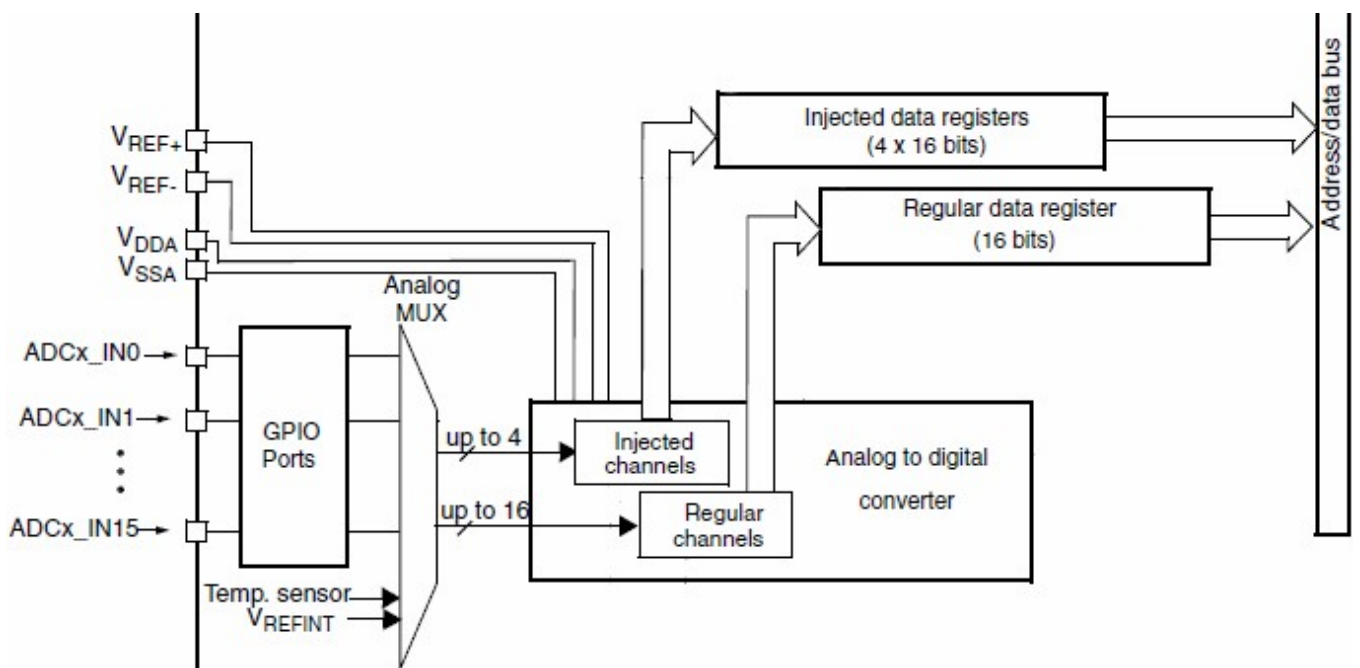


Рис. 1.3. Частина функціональної схеми АЦП STM32F103C8T6

Суть інжекттованого каналу полягає в тому, що у нього є свій окремий регістр для збереження результату. Тобто, якщо канали PA0, PA1, PA2, PA3 налаштувати як інжекттовані, то результати будуть збережені в чотири різні регістри. Інжекттованих каналів може бути не більше чотирьох. Будь-який аналоговий вхід можна налаштувати як інжекттований.

У регулярних каналів всього один регістр на всіх. Тобто, якщо канали PA0, PA1, PA2, PA3 налаштувати як регулярні, то результат роботи кожного каналу буде записуватися в один і той же регістр, затираючи попередні дані. Перетворення для каналів буде відбуватись послідовно, регулярно.

АЦП послідовного наближення – це синхронний пристрій, який виконує перетворення за певне число синхроімпульсів. Частота цих імпульсів обмежена. За документацією мікроконтролера STM32F103C8T6, максимальна допустима частота його тактування складає 14 МГц. АЦП знаходиться на шині ABR2. Між синхроімпульсами шини і входом модуля АЦП встановлений програмно керований подільник (ADC Prescaler) для управління частотою тактування.

Точність перетворення залежить, в тому числі, від часу заряду конденсатора схеми вибірки/зберігання. Разом з вихідним опором джерела аналогового сигналу конденсатор утворює RC-ланцюжок. Час, необхідний для повного заряду конденсатора, визначається постійною часу RC-ланцюжка. Для забезпечення точності вимірювання АЦП час вибірки має бути не менше певного значення, яке залежить від вихідного опору джерела сигналу. STM32 дозволяє встановлювати час вибірки для кожного каналу окремо. Час вибірки збільшує час перетворення. У мінімальному варіанті час вибірки дорівнює 1,5 циклу; перетворення вимагає 12,5 циклів; загальний час складає 14 циклів.

Модуль АЦП має вбудований механізм **калібрування**. Калібрування значно зменшує похибку перетворення аналогових сигналів. Під час калібрування обчислюються коригувальні коди, які компенсують похибки аналогових компонентів АЦП. Коди зберігаються до вимкнення живлення у разі скидання мікроконтролера. Рекомендується проводити калібрування один раз після кожної подачі живлення.

Існують наступні **режими перетворення АЦП**:

- *Один канал, одноразове перетворення.* Це режим, в якому запускається перетворення тільки по одному каналу. Після його завершення АЦП зупиняється і чекає наступного запуску.
- *Один канал, безперервне перетворення.* Після закінчення перетворення, воно буде запускатися знову. АЦП буде безперервно вимірювати значення напруги на обраному вході і поміщати його в регістр. Таким чином, в ході виконання програми зникає необхідність в управлінні АЦП.

- *Кілька каналів, одноразове перетворення.* У цьому режимі відбувається автоматичне опитування послідовності каналів. Іноді його називають *режимом сканування*. Єдиний спосіб опитування регулярних каналів в такому режимі – використання прямого доступу до пам'яті. Результат кожного перетворення АЦП буде переписуватися до зовнішньої пам'яті. Для зберігання результатів перетворень інжектованих каналів відведені 4 регістра, тому можна використовувати один запуск і отримати 4 значення, виміряні інжектованими каналами навіть без прямого доступу до пам'яті.
- *Кілька каналів, безперервне перетворення.* Аналогічно режимові для одного каналу, перетворення перезапускається автоматично.

Існує також так званий переривчастий режим (*Discontinuous Mode*). В переривчастому режимі послідовність сканування каналів можна розбити на більш короткі відрізки, підгрупи. Існує обмеження – переривчастий режим може використовуватися тільки для одного типу каналів, регулярних або інжектованих. На практиці застосовується не часто.

Для інжектованих каналів може бути задано математичне зміщення результату. Зміщення дозволяє реалізувати на однополярному АЦП вимірювання сигналів негативної полярності. Для цього необхідно апаратно додати позитивне зміщення вхідного сигналу. Після цього нульовому рівню сигналу на вході відповідатиме код напруги зміщення. У разі подачі негативного сигналу напруга на вході АЦП зменшиться, але залишиться позитивною.

Перетворення АЦП може бути запущено не лише програмно, а і від зовнішньої, по відношенню до модуля АЦП, події. Такою подією може бути зовнішній сигнал або апаратна подія таймера.

На практиці часто необхідно запускати АЦП циклічно, через однакові проміжки часу. Дана операція називається **дискретизацією**. Для циклічного запуску АЦП можна використовувати таймер. Наприклад, через кожні Δt мкс таймер повинен запускати перетворення каналу АЦП. Це і буде періодом дискретизації вхідних сигналів.

Про завершення перетворення заданих каналів повідомляють біти ЕОС і JEOS, розташовані в регістрі стану. За установкою даних бітів можуть формуватися переривання.

- Біт ЕОС встановлюється апаратно під час завершення перетворення. Скидається він програмно.
- Біт JEOS встановлюється апаратно під час завершення перетворення каналів тільки інжектованої групи. Скидається тільки програмно.

Отже, для **перетворення аналогового сигналу в STM32 необхідно:**

1. Задати загальні налаштування, джерело запуску, тактування.
2. Задати необхідний канал або канали для перетворення.
3. Вибрати режим перетворення.
4. Запустити перетворення.
5. Визначити, що перетворення закінчилося.
6. Зчитати результат перетворення напряму або з використанням переривань.

Зведений список налаштувань АЦП в STM32CubeIDE:

- *Independent mode* (незалежний режим) – використовується для налаштування роботи декількох АЦП в парному режимі.
- *Data Alignment* – вирівнювання даних по лівому або правому краю. Оскільки АЦП 12-розрядний, а регістри 16-розрядні, то передбачена можливість вирівнювання результату вимірювання по лівому або по правому краю цих регістрів.
- *Scan Conversion Mode* – цей режим стає активним коли опитуються декілька каналів на одному АЦП.
- *Continuous Conversion Mode* – опитування каналу / каналів проводиться безперервно (немає необхідності перезавпуску АЦП).
- *Discontinuous Conversion Mode* – цей режим дозволяє сканувати кілька каналів (на одному АЦП) так, щоб опитування відбувалось не по всіх каналах одразу, а по одному, або по заздалегідь заданим групам каналів.

- *Enable Regular Conversions* – канали налаштовані як регулярні.
- *Number Of Conversion* – кількість каналів для опитування.
- *External Trigger Conversion Source* – подія, яка буде запускати АЦП. Може бути програмний запуск або запуск за допомогою таймерів.
- *Rank* – черговість опитування.
- *Channel* – номер каналу. Канали можна поміняти місцями.
- *Sampling Time* – час (вказується в тактах), який витрачається на заряд конденсатора схеми вибірки/зберігання. Чим більше значення, тим точніше результат, але довше перетворення.

Хід виконання роботи

1. Виконати попередні налаштування плати «STM32 Blue pill» для роботи з модулем АЦП.
 - 1.1. Запустити середовище програмування STM32CubeIDE та створити новий проект.
 - 1.2. Встановити джерело тактування від зовнішнього кварцового резонатора та режим відлагодження «*Serial Wire*».
 - 1.3. На вкладці *Clock Configuration* налаштувати тактування мікроконтролеру. Встановити тактування від зовнішнього кварцового резонатора та основну частоту тактування 72 МГц.
 - 1.4. На вкладці *Pinout & Configuration* в розділі *Analog* активувати канал «IN0» АЦП1. На графічній моделі мікроконтролера автоматично активується вивід PA0, до якого підключено вхід каналу «IN0» АЦП1.
 - 1.5. В параметрах обраного каналу налаштувати канал для роботи в регулярному режимі (*Enable Regular Conversions*: «Enable») з однократним зчитуванням одного каналу (*Continuous Conversion Mode*: «Disabled»), а також встановити час перетворення на рівні 7,5 циклів (*Rank => Sampling Time*: «7.5 Cycles»). Інші налаштування на даному етапі залишити незмінними (рис. 1.4).

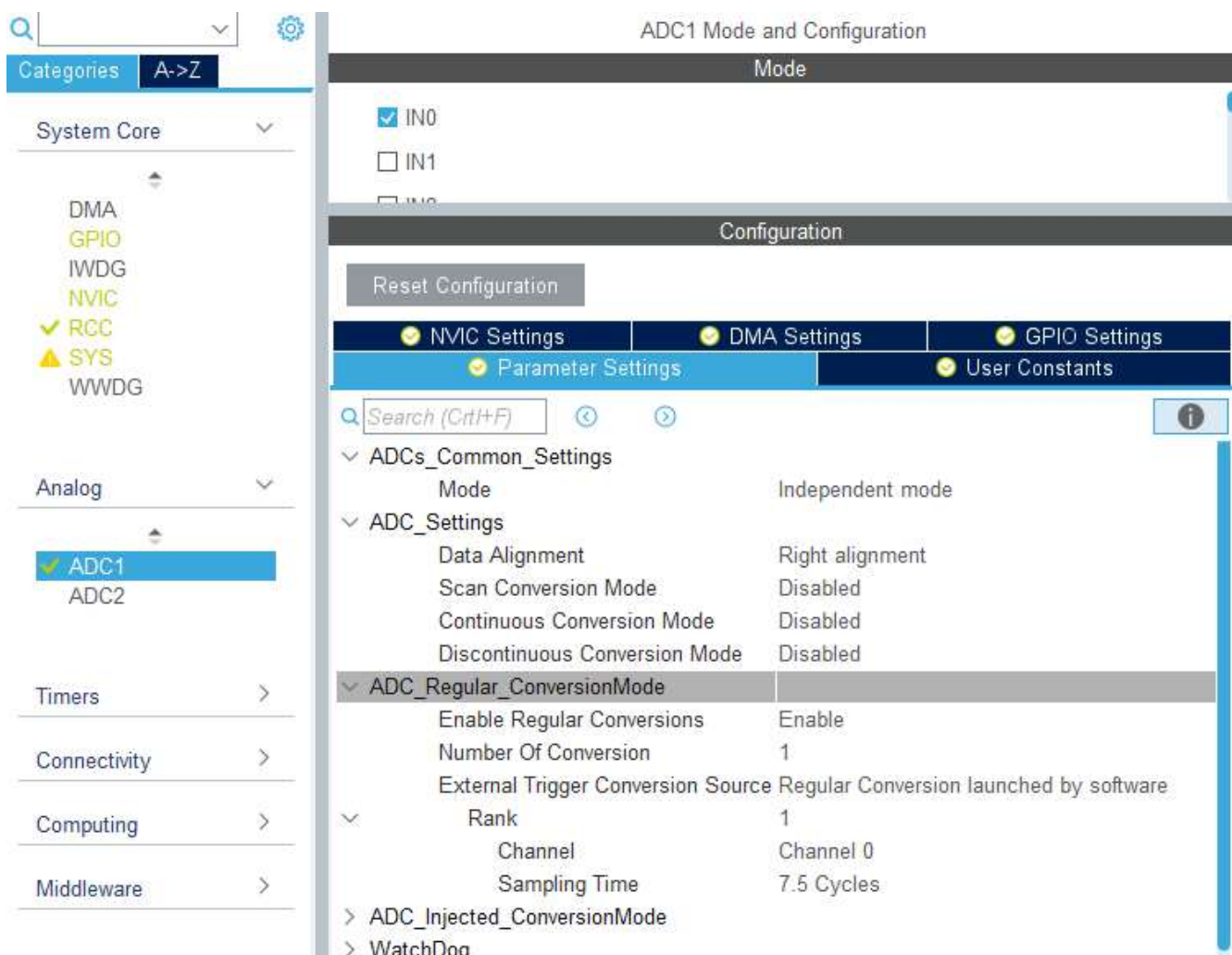


Рис. 1.4. Налаштування каналу АЦП

- 1.6. Активувати виводи PB3-PB8 в режимі активних виходів для LCD.
- 1.7. На вкладці *Clock Configuration* встановити частоту тактування АЦП1 на шині APB2 на рівні 12 МГц. Для цього обрати значення подільника частоти АЦП *ADC Prescaler*: «/6».
- 1.8. Зберегти проект та провести генерацію коду ініціалізації.
- 1.9. Перемістити файли створеної в попередніх роботах бібліотеки для роботи з LCD дисплеєм (*lcd.h*, *lcd.c*) до відповідних папок нового проекту.
- 1.10. В головному файлі програми *main.c* підключити заголовкові файли бібліотек *lcd.h* та *stdio.h*:

```
/* USER CODE BEGIN Includes */
/* Includes -----*/
#include "lcd.h"
#include "stdio.h"
/* USER CODE END Includes */
```

2. Написати програму для зчитування однократних вимірювань з каналу АЦП та виводу результату на LCD.

2.1. Створити змінну *i* та символьний рядок *str*, в яких буде зберігатись результат перетворення:

```
/* USER CODE BEGIN PV */
uint16_t i;
char str[50];
/* USER CODE END PV */
```

2.2. Провести ініціалізацію дисплею:

```
/* USER CODE BEGIN 2 */
LCD_init();
LCD_String("Voltage: ");
/* USER CODE END 2 */
```

2.3. Реалізувати алгоритм опитування АЦП та виводу результату на дисплей.

Для запуску АЦП використовується функція *HAL_ADC_Start()*, яка в якості аргументу приймає вказівник на структуру АЦП. Для зупинки АЦП після завершення перетворення використовується аналогічна функція *HAL_ADC_Stop()*. Для перевірки готовності результату використовується функція *HAL_ADC_PollForConversion()*, яка в якості аргументів приймає вказівник на структуру АЦП та значення таймауту (якщо АЦП не відповість за вказаний час, результат вважається неготовим). Для зчитування результату перетворення використовується функція *HAL_ADC_GetValue()*. Отже, код виглядає наступним чином:

```
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */

    // запускаємо АЦП
    HAL_ADC_Start(&hadc1);
    // перевіряємо, чи готовий результат, таймаут 100 мс
    HAL_ADC_PollForConversion(&hadc1, 100);
    // записуємо результат до змінної i
    i = HAL_ADC_GetValue(&hadc1);
```

```

// зупиняємо АЦП
HAL_ADC_Stop(&hadc1);
// перетворюємо змінну i в рядок str довжиною 4 символи
sprintf(str, "%04d", i);
// виводимо результат на дисплей
LCD_SetPos(9, 0);
LCD_String(str);
HAL_Delay(500);
}
/* USER CODE END 3 */

```

2.4. Підключити на вхід каналу 0 АЦП1 (вивід PA0) потенціометр та дисплей за схемою, показаною на рис. 1.5.

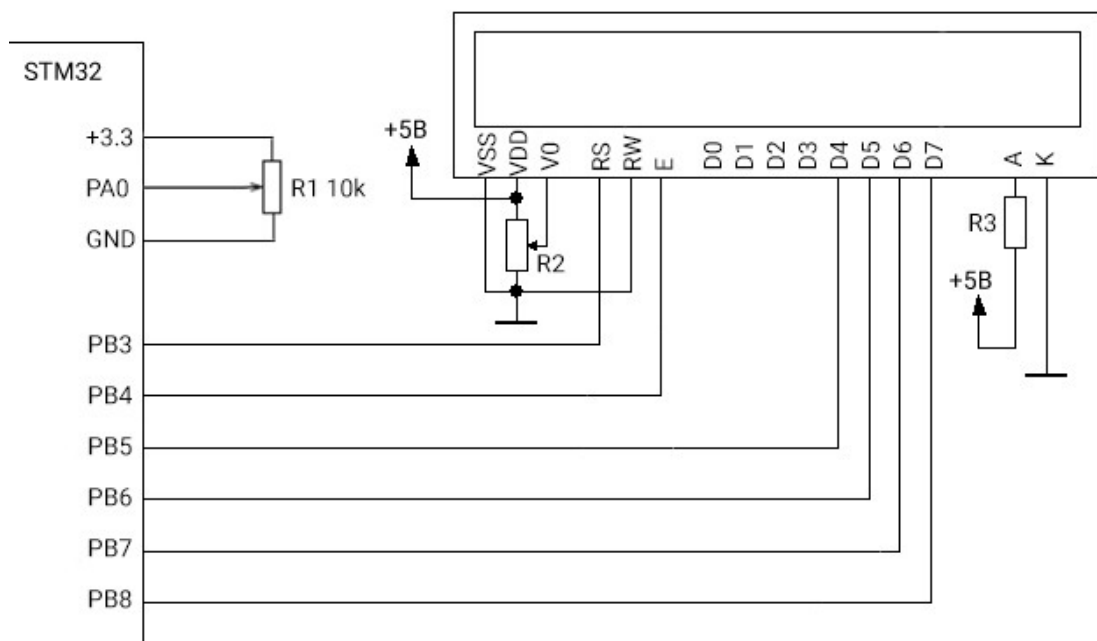


Рис. 1.5. Підключення потенціометра та LCD до входу АЦП

2.5. Завантажити програму до мікроконтролера та перевірити результат. Критерієм правильності роботи програми буде виведення на дисплей коду виміряної напруги в діапазоні від 0 до 4095, який змінюється під час прокрутки регулятора потенціометра.

2.6. Модифікувати програму так, щоб замість коду напруги на дисплей виводилось її реальне значення у Вольтах. Для цього створити змінну *u*:

```

/* USER CODE BEGIN PV */
float u;
char str[50];
/* USER CODE END PV */

```

2.7. Переписати код основної програми:

```
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */

    // запускаємо АЦП
    HAL_ADC_Start(&hadc1);
    // перевіряємо, чи готовий результат, таймаут 100 мс
    HAL_ADC_PollForConversion(&hadc1, 100);
    // виконуємо необхідні перетворення та записуємо результат
    u = ((float)HAL_ADC_GetValue(&hadc1)*3.3/4096);
    // зупиняємо АЦП
    HAL_ADC_Stop(&hadc1);

    // перетворюємо змінну u в рядок
    sprintf(str, "%.2fv", u);

    // виводимо результат на дисплей
    LCD_SetPos(9, 0);
    LCD_String(str);
    HAL_Delay(500);

}
/* USER CODE END 3 */
```

2.8. Завантажити програму до мікроконтролера та перевірити результат.

3. Модифікувати програму таким чином, щоб опитування АЦП здійснювалось в безперервному режимі за перериваннями, запит на які формується після завершення перетворення. Запуск АЦП має відбуватись апаратно за допомогою таймера з періодом 500 мс. Передбачити автоматичне калібрування АЦП під час увімкнення мікроконтролера.

- 3.1. Налаштувати АЦП на роботу в безперервному режимі. Для цього в конфігураторі проекту перейти на вкладку з налаштуваннями каналу IN0 АЦП1 та встановити режим роботи *Continuous Conversion Mode*: «Enabled» (рис. 1.6).

- 3.2. Встановити режим опитування АЦП за переповненням таймеру, обравши в опції *External Trigger Conversion Source*: «Timer 3 Trigger Out event».

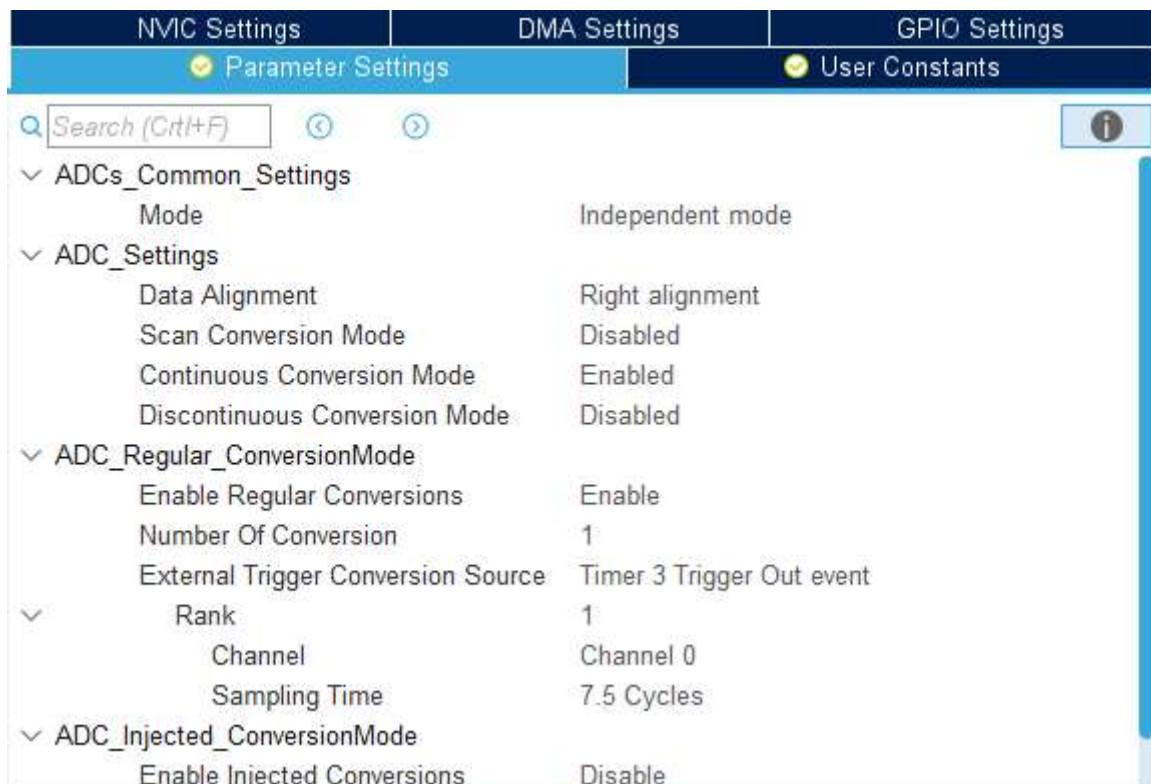


Рис. 1.6. Налаштування АЦП для роботи в безперервному режимі з запуском перетворення за таймером

- 3.3. На вкладці *NVIC Settings* дозволити глобальні переривання від АЦП, встановивши відповідний прапорець.
- 3.4. Налаштувати таймер TIM3 на генерацію переривань кожних 0,5 с. Для цього обрати у розділі *Timers* відповідний таймер та активувати його тактування від внутрішнього джерела (*Internal Clock*). Встановити значення переддільника частоти тактування таймеру *Prescaler*: «719» та значення регістру перезавантаження *Counter Period*: «50000».
- 3.5. У секції *Trigger Output (TRGO) Parameters* встановити *Trigger Event Selection*: «Update Event».
- 3.6. Зберегти проект та згенерувати код.
- 3.7. У файлі *main.c* створити змінну *ADC_Data*, яка буде зберігати результат перетворення АЦП:

```
/* USER CODE BEGIN PV */
uint16_t ADC_Data;
float u;
char str[50];
/* USER CODE END PV */
```

3.8. В блоці ініціалізації увімкнути автоматичне калібрування АЦП (функція *HAL_ADCEx_Calibration_Start()*), запустити АЦП в режимі генерації переривань (функція *HAL_ADC_Start_IT()*) та запустити таймер TIM3:

```
/* USER CODE BEGIN 2 */
LCD_init();
LCD_String("Voltage: ");
// Автоматичне калібрування АЦП
HAL_ADCEx_Calibration_Start(&hadc1);
// Запуск АЦП в режимі генерації переривань
HAL_ADC_Start_IT(&hadc1);
// Запуск таймера 3
HAL_TIM_Base_Start(&htim3);
/* USER CODE END 2 */
```

3.9. Модифікувати код основного циклу з урахуванням всіх змін:

```
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */

    u = ((float)ADC_Data*3.3/4096);
    sprintf(str, "%.2fv", u);
    LCD_SetPos(9, 0);
    LCD_String(str);
    HAL_Delay(100);

}
/* USER CODE END 3 */
```

3.10. Створити функцію-обробник переривання АЦП *HAL_ADC_ConvCpltCallback()*. В даній функції, яка буде автоматично викликатися після завершення перетворення АЦП1, організувати зчитування результату перетворення:

```
/* USER CODE BEGIN 4 */
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc1)
{
    ADC_Data = HAL_ADC_GetValue(hadc1);
}
/* USER CODE END 4 */
```

3.11. Запустити програму та перевірити результат.

4. Замість потенціометра підключити до входу АЦП фоторезистор за схемою, показаною на рис. 1.7.
- 4.1. Перезапустити мікроконтролер та відслідкувати результат. Критерієм правильності роботи системи буде зміна значення вихідної напруги під час зміни інтенсивності світла, яке потрапляє на фоторезистор.
- 4.2. Самостійно модифікувати програму таким чином, щоб у випадку недостатнього освітлення вмикався користувацький світлодіод (пори́г вмика́ння обрати довільно).



Рис. 1.7. Зовнішній вигляд та схема підключення фоторезистора до входу АЦП

Індивідуальні завдання

1. Отримати у викладача датчик температури LM35. Створити програму, яка виводитиме на дисплей значення поточної температури. Передбачити алгоритм ковзного усереднення останніх трьох значень.
2. Отримати у викладача датчик Холла A3144 та кубічний магніт. Створити програму, яка дозволить визначити полярність граней магніту. У випадку наявності спрацювання датчику має засвічуватись користувацький світлодіод. Визначити полярність всіх граней магніту.
3. Отримати у викладача датчик вогню YS-17. Розробити програму, яка буде засвічувати користувацький діод у випадку виявлення високої температури. Чутливість датчика має регулюватись за допомогою потенціометра.

Контрольні запитання

1. Поясніть принцип роботи АЦП послідовного наближення.
2. Назвіть типи каналів, які доступні під час роботи з АЦП STM32, то поясніть різницю між ними.
3. Опишіть режими роботи АЦП STM32.
4. Поясніть різні способи зчитати результат перетворення АЦП.
5. Що таке «час перетворення», як його визначити та на що впливає дана величина?
6. Яким чином можна задати частоту дискретизації АЦП? Як налаштувати мікроконтролер для роботи АЦП на частоті дискретизації 1 кГц?
7. Поясніть різницю між поняттями «частота дискретизації» та «частота тактування» АЦП. Якою є мінімальна частота дискретизації та максимальна частота тактування АЦП STM32F103C8T6?
8. Опишіть порядок налаштування та роботи з АЦП STM32F103C8T6. Яким чином налаштувати АЦП для роботи з одним регулярним каналом в безперервному режимі з опитуванням за перериваннями?

ШИРОТНО-ІМПУЛЬСНА МОДУЛЯЦІЯ

Мета: Ознайомитись з особливостями генерації ШІМ-сигналів за допомогою мікроконтролерів STM32. Навчитись створювати програмний код для управління яскравістю світлодіодів та швидкістю обертання валів двигунів постійного струму.

Теоретичні відомості

Дуже часто в робототехніці виникає необхідність плавно керувати якимось процесом, наприклад, яскравістю світлодіода або швидкістю обертання двигуна. Цілком очевидно, що таке управління безпосередньо пов'язано зі зміною напруги: якщо її знизити, світлодіод буде світити менш яскраво, а двигун обертатися з нижчою швидкістю. Але проблема в тому, що напругу управляти напругою може тільки ЦАП – цифро-аналоговий перетворювач. А мікроконтролер STM32F103C8T6 не має вбудованого ЦАПу, і може генерувати на виході лише цифровий сигнал, тобто логічні «0» або «1» (3,3 В або 0 В).

Широтно-імпульсна модуляція, або ШІМ (англ. pulse width modulation, PWM) – це операція отримання змінного аналогового значення за допомогою цифрових сигналів. Розберемо поняття ШІМ на прикладі управління швидкістю обертання двигуна постійного струму. Наприклад, необхідно запустити мотор на 50% від його максимальної швидкості. Щоб досягти заданої швидкості, нам необхідно в одиницю часу передавати на мотор в два рази менше потужності.

Уявімо двигун постійного струму з масивним маховиком, закріпленим на валу (таким маховиком може служити колесо). Увімкнемо живлення від акумулятора і мотор почне набирати обертів. Через якийсь час, мотор досягне номінальної потужності, а його ротор максимальної швидкості обертання. Відключимо живлення, і мотор поступово почне сповільнюватися до зупинки.

Далі знову увімкнемо мотор, і коли його швидкість досягне половини від максимальної – вимкнемо. Помітивши, що швидкість падає – знову увімкнемо. І так циклічно. Вмикаючи і вимикаючи живлення мотора, ми зможемо обернути ротор зі швидкістю, близькою до половини від максимальної.

Кількість переданої мотору енергії буде залежати від відношення часу, коли мотор включений – $t_{вкл}$ до часу, коли він вимкнений – $t_{вимк}$. Так, для передачі мотору 50% потужності, $t_{вкл}$ дорівнюватиме $t_{вимк}$ (рис. 2.1).

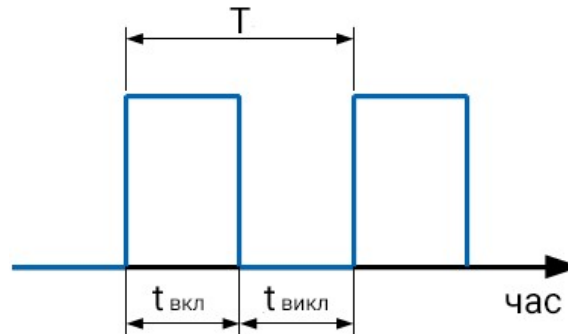


Рис. 2.1. Передача половини потужності

Щоб мотор обертася ще повільніше, наприклад, з потужністю 25% від номінальної, час включення мотора доведеться зменшити до цих 25% від загального періоду **управління** T (рис. 2.2). Таким чином, маючи можливість змінювати **ширину імпульсів**, ми можемо досить точно управляти швидкістю обертання двигуна. Розглянутий спосіб управління потужністю і є **широтно-імпульсною модуляцією** сигналу.

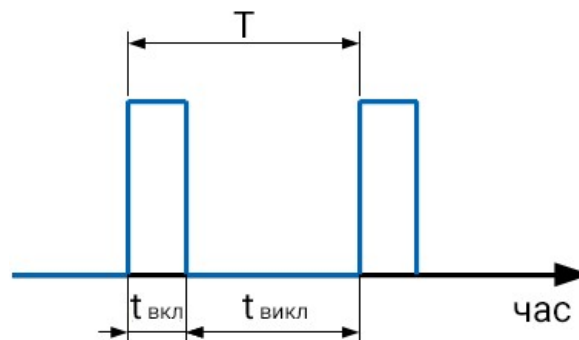


Рис. 2.2. Передача 25% потужності

Якщо потрібно управляти не швидкістю двигуна, а яскравістю світлодіода, також можна використати ШІМ. У випадку частоти включень 1 раз на секунду, ми будемо бачити як світлодіод запалюється і гасне. Але якщо частоту збільшити

до 50 разів на секунду, то перемикання світлодіода стане непомітним для людського ока. Це створить ефект світіння в половину потужності. Таким чином можна управляти яскравістю світлодіода, змінюючи час увімкнення-вимкнення за однакового періоду часу T .

Отже, ШІМ також може виконувати операції перемикання з дуже великою швидкістю, непомітною для людського ока, що широко застосовується в регулюванні яскравості моніторів, екранів мобільних телефонів, швидкості обертання двигунів, вихідної напруги імпульсних блоків живлення.

Найголовніший параметр ШІМ – **коефіцієнта заповнення D** (англ. duty cycle). Цей коефіцієнт дорівнює відношенню ширини імпульсу до періоду ШІМ сигналу (рис. 2.3):

$$D = t_{\text{вкл}} / T$$

Чим більше D , тим більше потужності ми передаємо пристрою, яким керуємо, наприклад, двигуну. Так, у випадку $D = 1$ двигун працює на 100% потужності, при $D = 0,5$ – на половину потужності, у випадку $D = 0$ – двигун повністю відключений.

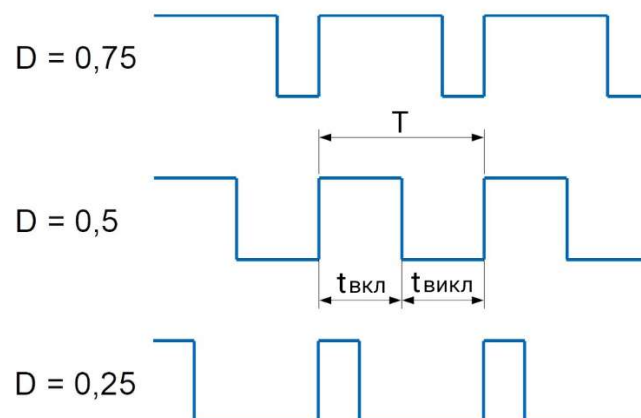


Рис. 2.3. Різні значення коефіцієнту заповнення

Фактично, можна співставити величину D з еквівалентною постійною напругою аналогового сигналу. Наприклад, якщо напруга живлення мікроконтролера становить 3,3 В, то у випадку $D = 0,5$ двигун буде обертатися так, ніби на нього подано напругу 1,65 В. Даний принцип проілюстровано на рис. 2.4.

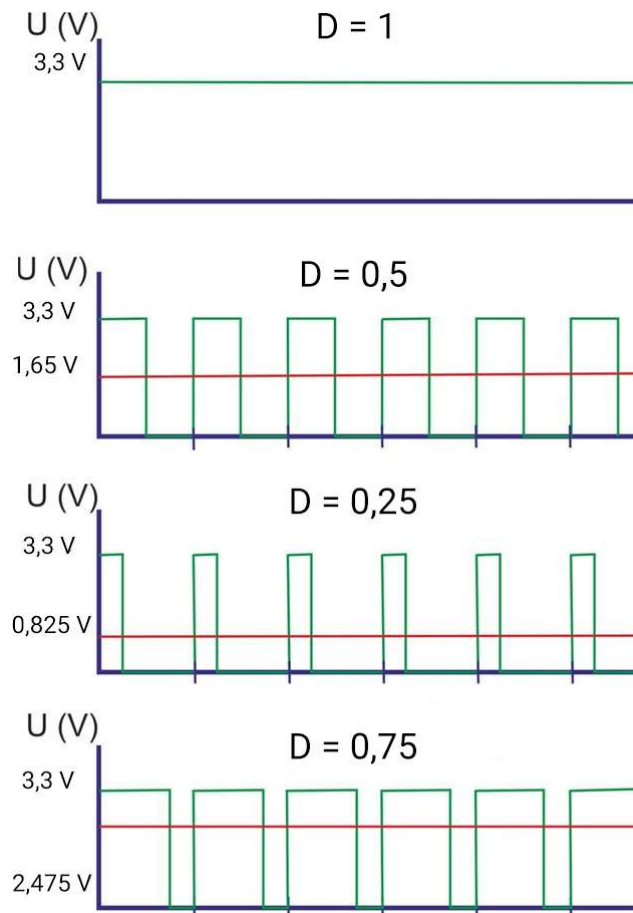


Рис. 2.4. Зміна еквівалентної напруги за різних значень коефіцієнту заповнення

Частота ШІМ визначає **період** імпульсів управління T . Вимоги до цієї частоти диктуються кількома факторами, в залежності від типу пристрою, яким управляють.

У разі управління світлодіодами одним з головних чинників стає видимість мерехтіння. Чим вище частота, тим менш помітне мерехтіння світлодіоду. Висока частота також допомагає знизити вплив температурних стрибків, які шкодять світлодіодам. На практиці для світлодіодів досить мати частоту ШІМ в межах 100-300 Гц.

З двигунами постійного струму ситуація трохи інша. З одного боку, чим більше частота, тим більш плавно і менш шумно працює мотор. З іншого – на високих частотах падає крутний момент. Тому, у випадку управління двигунами постійного струму для більшості завдань рекомендовано використовувати частоту ШІМ близько 2кГц.

Загальна проблема для всіх задач керування силовим навантаженням – втрати в ланцюгах силової комутації (наприклад, в транзисторах), які збільшуються з ростом частоти ШІМ. Чим більше частота, тим більше часу транзистори знаходяться в перехідних станах, активно виділяючи тепло і знижуючи ефективність системи.

Для генерації ШІМ сигналу в мікроконтролерах використовуються таймери. У режимі генерації ШІМ використовуються дві величини: одна відповідає за період перезавантаження виводу ШІМ (TIM_Period), а друга за заповнення (TIM_Pulse). Лічильник стартує з «0» і рахує до значення TIM_Period, потім перезавантажується і починає все спочатку. Під час підрахунку таймер пройде проміжне значення TIM_Pulse. Як тільки це відбудеться, він переключить вивід ШІМ на протилежний стан (рис. 2.5).

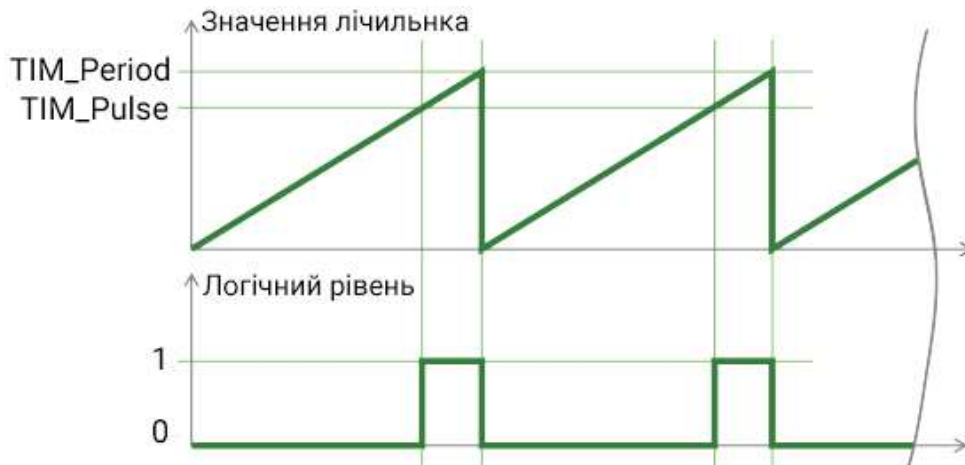


Рис. 2.5. Генерація ШІМ за допомогою таймера

В STM32 кожен таймер має 4 незалежних канали, кожен з яких може використовуватися для генерації імпульсів (compare mode) або для їх захоплення (capture mode). Відповідно, існує група регістрів захоплення-порівняння: TIMx_CCR1, TIMx_CCR2, TIMx_CCR3, TIMx_CCR4. Запис до них значень TIM_Pulse задає коефіцієнт заповнення ШІМ імпульсів. Для генерації ШІМ сигналу таймер повинен працювати в режимі генерації імпульсів. А значення TIM_Period можна задати за допомогою регістра перезавантаження таймера (AutoReload Register). Цей параметр також задає лічильнику таймера на скільки частин необхідно поділити період ШІМ сигналу – **роздільну здатність ШІМ**.

Хід виконання роботи

1. Виконати попередні налаштування плати «STM32 Blue pill» для генерації ШІМ сигналу.
 - 1.1. Запустити середовище програмування STM32CubeIDE та створити новий проект.
 - 1.2. Встановити джерело тактування від зовнішнього кварцового резонатора та режим відлагодження «*Serial Wire*».
 - 1.3. На вкладці *Clock Configuration* налаштувати тактування мікроконтролера. Встановити тактування від зовнішнього кварцового резонатора та основну частоту тактування 72 МГц.
 - 1.4. На вкладці *Pinout & Configuration* в розділі *Timers* активувати таймер TIM2, обравши внутрішнє джерело тактування *Clock Source*: «Internal Clock».
 - 1.5. Активувати канал 2 таймеру в режимі генерації ШІМ, обравши *Channel 2*: «PWM Generation CH2». Після цього на графічній моделі мікроконтролера автоматично активується вивід PA1, який і буде виходом ШІМ-генератора. Інші налаштування таймера залишити стандартними (рис. 2.6).
 - 1.6. В розділі *System Core => GPIO* на вкладці *TIM* обрати максимальну швидкість роботи виводу PA1 *Maximum output speed*: «High».
 - 1.7. Зберегти проект та провести генерацію коду ініціалізації.
 - 1.8. Оголосити змінні *i* та *d*, які будуть використовуватись для управління коефіцієнтом заповнення ШІМ сигналу та організації коротких затримок відповідно:

```
/* USER CODE BEGIN PV */  
  
uint32_t i, d;  
  
/* USER CODE END PV */
```

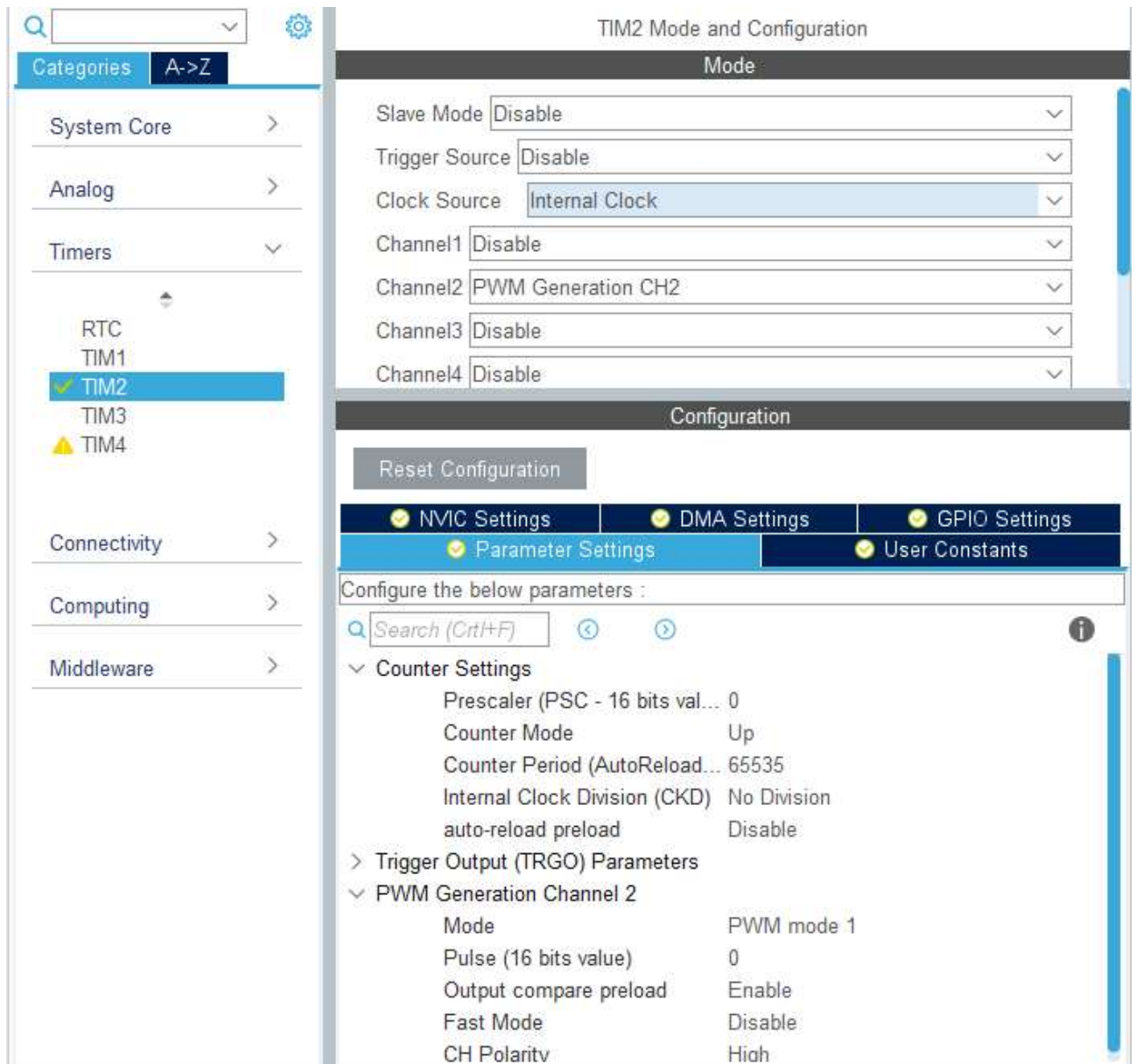


Рис. 2.6. Налаштування таймера TIM 2 для генерації ШІМ сигналу на каналі 2

1.9. Запустити таймер TIM 2 в режимі генерації ШІМ на каналі 2, використавши функцію *HAL_TIM_PWM_Start()*:

```
/* USER CODE BEGIN 2 */
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_2);
/* USER CODE END 2 */
```

2. Написати програму для ШІМ-управління зовнішнім світлодіодом. Світлодіод має плавно запалюватись і загасати з довільним періодом.

2.1. Самостійно підключити зовнішній світлодіод до виводу PA1. Обов'язково використати резистор номіналом від 500 Ом для обмеження максимального струму.

2.2. Написати відповідний код програми:

```
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */

    // повільно запалюємо світлодіод
    for(i=0;i<65536;i++)
    {
        TIM2 -> CCR2 = i;    //змінюємо коефіцієнт заповнення ШІМ
        // організуємо невелику затримку
        for(d=0;d<50;d++){ }
    }

    //повільно гасимо світлодіод
    for(i=65535;i>0;i--)
    {
        TIM2 -> CCR2 = i;
        for(d=0;d<50;d++){ }
    }
}
/* USER CODE END 3 */
```

2.3. Завантажити програму до мікроконтролера та перевірити результат.

Критерієм правильності буде повільне засвічення/згасання світлодіоду.

3. Модифікувати програму таким чином, щоб період ШІМ сигналу становив 4 мс (частота 250 Гц), а роздільна здатність – 500.

3.1. Для забезпечення таких параметрів, на вхід генератора ШІМ необхідно подати сигнал з частотою $250\text{Гц} \times 500 = 125\text{ кГц}$. Це потрібно для того, щоб протягом кожного періоду лічильник таймера зміг відраховувати імпульс потрібної довжини: 0,1,2, ..., 499 частин періоду.

3.2. Для забезпечення даної частоти підрахунку таймера, необхідно встановити значення регістра переддільника *Prescaler* на рівні «576-1», а значення регістра перезавантаження *Counter Period (AutoReload Register)*: «500-1». Після виконання відповідних налаштувань мікроконтролера провести повторну генерацію коду.

3.3. Модифікувати код основної програми наступним чином:

```

/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */

    // повільно запалюємо світлодіод
    for(i=0;i<500;i++)
    {
        TIM2 -> CCR2 = i;    //змінюємо коефіцієнт заповнення ШІМ
        HAL_Delay(1);
    }

    //повільно гасимо світлодіод
    for(i=499;i>0;i--)
    {
        TIM2 -> CCR2 = i;
        HAL_Delay(1);
    }
}

/* USER CODE END 3 */

```

3.4. Завантажити програму до мікроконтролера та перевірити результат.

Критерієм правильності буде повільне засвічення/згасання світлодіоду.

3.5. За допомогою осцилографа переконатися, що частота згенерованого ШІМ сигналу дійсно становить 250 Гц.

4. Реалізувати систему ШІМ-управління швидкістю обертання двигуна постійного струму.

4.1. Підключити двигун постійного струму до мікроконтролера за схемою, показаною на рис. 2.7. В якості джерела живлення двигуна (+5 В) використати зовнішній лабораторний блок живлення.

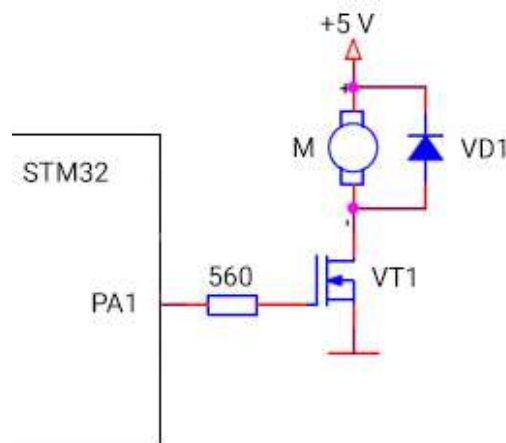


Рис. 2.7. Схема підключення двигуна постійного струму до мікроконтролера

Пояснення до схеми: Двигун ніколи не підключається напряму до виходів мікроконтролера, оскільки через нього буде протікати високий струм, а також напруга живлення двигуна зазвичай відрізняється від напруги логічних рівнів. ШІМ з виходу PA1 відкриває і закриває ключ, зібраний на MOSFET-транзисторі. Транзистор повинен мати низький поріг відкриття, тобто відкриватися за невеликої напруги (не більше 3.3 В). Інакше необхідно використовувати додатковий елемент – драйвер. Резистор необхідний для обмеження струму затвору на виводі порту мікроконтролера.

В схемі також необхідний діод. Двигун – це індуктивне навантаження, а в деяких випадках він стає електрогенератором. Тому після закриття транзистора на виводах двигуна можуть виникати імпульси високої напруги. Вони повинні замикатися через діод, щоб не спалити транзистор.

У разі управління за допомогою ШІМ діод повинен бути високочастотним. За закритого транзистора діод знаходиться у відкритому стані, через нього тече струм розмагнічування обмотки двигуна. Потім транзистор відкривається. А діод закривається тільки через певний час – час відновлення зворотного опору. Повільний діод призведе до потужних завад, зменшення ККД та нагрівання діода і транзистора.

4.2. Збільшити частоту ШІМ сигналу до ~2,5 кГц. Для цього необхідно в налаштуваннях таймера встановити значення регістра переддільника Prescaler на рівні «57». Значення регістру перезавантаження залишити без змін на рівні «500». Зберегти виконані налаштування і провести повторну генерацію коду.

4.3. В основній програмі передбачити поетапне збільшення швидкості обертання валу двигуна на довільну величину кожних 5 с, а після досягнення максимального значення – аналогічне поетапне зниження швидкості до мінімальної. Для цього модифікувати програмний код нескінченного циклу наступним чином:

```

/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */

    for(i=0;i<=500;i+=100)
    {
        TIM2 -> CCR2 = i;
        HAL_Delay(5000);
    }
    for(i=500;i>0;i-=100)
    {
        TIM2 -> CCR2 = i;
        HAL_Delay(5000);
    }
}
/* USER CODE END 3 */

```

4.4. Завантажити програму до мікроконтролера та перевірити результат.
Критерій правильності – зміна швидкості обертання двигуна кожних 5 с.

Індивідуальні завдання

1. Модифікувати програму таким чином, щоб керування двигуном відбувалося за допомогою матричної клавіатури. Клавіші «1», «2» та «3» мають задавати три різні стандартні швидкості обертання. Клавіша «А» повинна збільшувати швидкість обертання на 10%, клавіша «В» – зменшувати. Клавіша «#» має вимикати двигун, клавіша «*» – вмикати на довільній базовій швидкості.
2. Підключити замість двигуна комп'ютерний кулер. Використовуючи датчик LM-35, реалізувати автоматизований контроль швидкості обертання кулера. У разі підвищення температури швидкість обертання кулера має автоматично збільшуватись, у разі зменшення – знижуватись. У випадку досягнення температурою деякого критичного максимального значення, має запалюватись користувачський світлодіод, а кулер – вимикатись та автоматично вмикатись після охолодження датчика.

3. Отримати у викладача RGB-світлодіод. Використовуючи ШІМ, реалізувати мінімум 3 різні режими світіння світлодіода. Обов'язково передбачити плавну зміну кольорів в усіх режимах. Переключення режимів має відбуватися за натисканням кнопки. Натискання кнопки після останнього режиму має вимикати світлодіод.

Контрольні запитання

1. Поясніть принцип роботи та області застосування широтно-імпульсної модуляції.
2. Опишіть основні параметри ШІМ та наведіть приклади визначення оптимальних параметрів для різних задач.
3. Опишіть порядок генерації ШІМ сигналу за допомогою таймерів.
4. Яким чином налаштувати таймер для генерації сигналу ШІМ з частотою 1 кГц, роздільною здатністю 8000 та коефіцієнтом заповнення 0,25?
5. Поясніть порядок роботи з таймерами в режимі генерації ШІМ сигналу. Опишіть основні налаштування, регістри та функції для роботи з ШІМ в STM32.
6. Наведіть та поясніть принцип роботи схеми підключення двигуна постійного струму до мікроконтролера для управління за допомогою ШІМ.
7. Яким чином можна реалізувати простий ЦАП на основі ШІМ-регулятора? Які електронні компоненти можна для цього використати? Наведіть спрощену схему.
8. Яким чином можна згенерувати синусоїдальний сигнал за допомогою ШІМ? Наведіть графічне пояснення принципу роботи такого генератора та спрощену схему його апаратної реалізації.
9. Яким чином можна використати ШІМ для передачі інформації? Поясніть принцип роботи такого інтерфейсу, його переваги та недоліки.
10. Як можна застосувати ШІМ у складі найпростішого імпульсного блоку живлення? Поясніть принцип роботи такого пристрою.

РОБОТА З ІНТЕРФЕЙСОМ ПЕРЕДАЧІ ДАНИХ UART

Мета: Ознайомитись з особливостями роботи інтерфейсу передачі даних UART. Навчитись створювати програмний код для передачі та отримання даних за інтерфейсом UART в різних режимах.

Теоретичні відомості

UART (англ. Universal Asynchronous Receiver-Transmitter – універсальний асинхронний приймач-передавач) – найпоширеніший на сьогоднішній день фізичний протокол передачі даних. Найбільш відомий з сімейства UART протокол – RS-232 (звичайний COM-порт, який використовується в будь-якому комп'ютері). Історично це найперший комп'ютерний інтерфейс. Однак, він існує і по сьогоднішній день та не втратив своєї актуальності.

UART широко поширений у світі електроніки, його у різних видах використовують великі та малі комп'ютери, контролери, датчики, засоби комунікації та інші електронні пристрої. Слово "асинхронний" означає, що прийом та передача окремих бітів не вирівнюються синхроімпульсами. Передача бітів даних відбувається виключно за часовим інтервалом між перепадами рівнів сигналу, який заздалегідь задається швидкістю передачі.

Існує версія UART з синхронізацією сигналів – USART, де буква S означає Synchronous (синхронний), але для наших завдань вона не потрібна. До переваг асинхронної передачі даних відносять: простоту протоколу, мінімум дротів та задіяних у процесі апаратних засобів, можливість повного дуплексу (одночасної передачі у обох напрямках). Недоліками асинхронного режиму є: слабка завадостійкість і, як наслідок, менша максимальна швидкість та відстань передачі даних. Проте, для більшості завдань швидкості та стійкості асинхронного варіанту вистачить із великим запасом.

З апаратної точки зору UART використовує два виводи для передачі даних – RX і TX, де перші літери означають, відповідно, Receiver та Transmitter. Для зв'язку двох пристроїв використовуються два дроти, причому з'єднувати їх слід хрест-навхрест: RX першого в TX другого і навпаки (рис. 3.1). Іноді їх маркують RXD і TXD, наголошуючи, що це сигнали передачі. Також обов'язковим є підключення виводу землі GND. У такій мінімальній конфігурації підключення два пристрої вже можуть без проблем обмінюватись даними в обох напрямках.

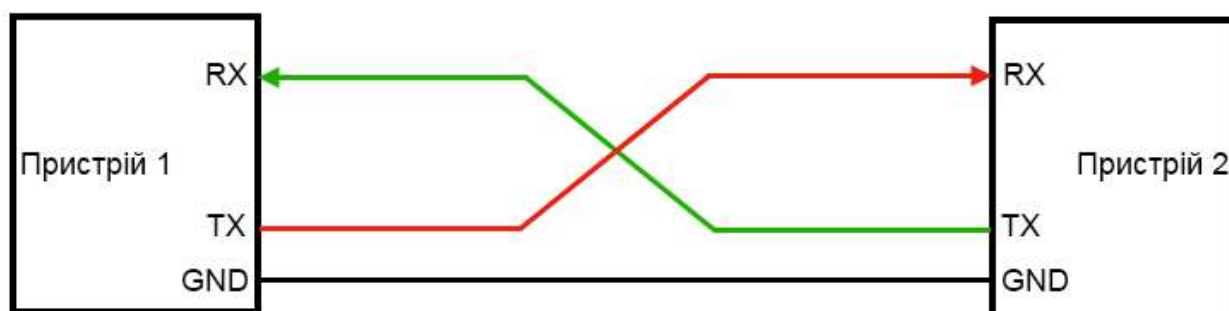


Рис. 3.1. Схема підключення двох пристроїв за інтерфейсом UART в мінімальній конфігурації

Якщо швидкодія двох пристроїв значно відрізняється (наприклад, другий пристрій не встигатиме записувати прийнятий пакет даних до буфера), то додатково можуть використовуватись виводи CTS та RTS. Вони також підключаються перехресно та призначені для позначення готовності приймача-передавача до прийому або передачі наступного пакету даних. Це називається апаратним контролем передачі. Однак, в даній лабораторній роботі такий режим не знадобиться.

Дані передаються асинхронним послідовним кодом. Це означає, що дані можуть бути передані в будь-який момент без використання між приймачем і передавачем синхронізуючих імпульсів. В одну сторону дані надсилаються за допомогою одного сигналу. Сам сигнал містить як власне дані, так і синхронізуючу інформацію. Щоб приймач зрозумів, що передача розпочинається, дані супроводжуються спеціальними бітами: стартовим і стоповим. Схема передачі одного пакету даних показана на рис. 3.2.

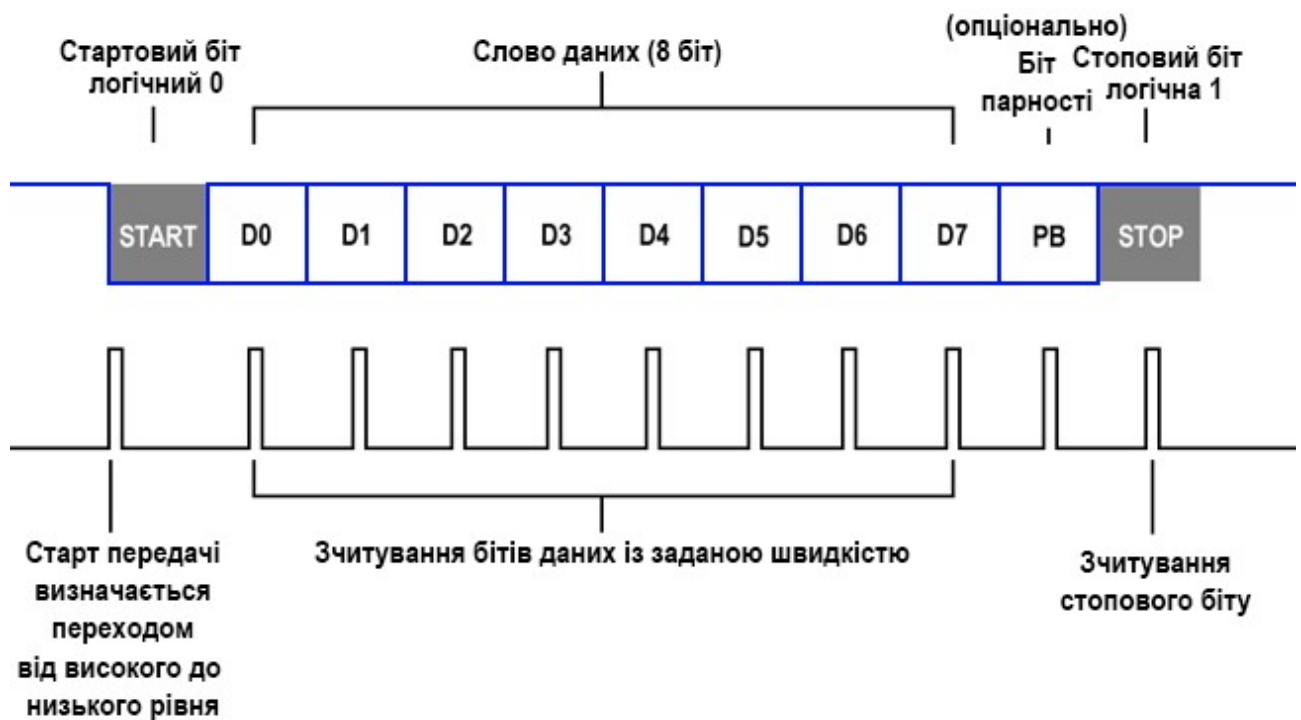


Рис. 3.2. Схема передачі даних за інтерфейсом UART

У режимі очікування сигнал на вході приймача знаходиться у високому рівні. Першим передається стартовий біт. Він має низький рівень. По першому спаду сигналу на низький рівень приймач починає відраховувати внутрішні синхроімпульси. Знаючи швидкість передачі, отже, і період надходження вхідних бітів, приймач зчитує стан наступних 8 бітів. Наприкінці передавач формує стоповий біт, завжди високого рівня.

Усі біти передаються за однакові проміжки часу. Час передачі одного біта визначає швидкість передачі інтерфейсу. Часто вона вказується у бодах (біт на секунду). Оскільки крім самих даних у потік додаються біти синхронізації, то для передачі байта потрібно не 8, а 10 бітів.

Ми розглянули найпоширеніший формат протоколу UART. Бувають варіанти з різними кількостями бітів даних, зазвичай від 5 до 9. Можуть використовуватися формати з двома стоповими бітами. Іноді додається біт контролю парності, який дозволяє перевіряти цілісність прийнятих даних. Але на практиці такі варіанти використовують рідко.

Важлива умова для роботи UART: обидва пристрої повинні бути налаштовані однаково, інакше вони не зрозуміють одне одного. Тобто швидкість передачі даних, розмір слова даних, необхідність використання біту парності та всі інші налаштування мають бути однаковими для передавача та приймача.

Таким чином, основні моменти, які потрібно пам'ятати:

- у неактивному режимі (стан очікування) вихід передавача UART знаходиться у високому рівні;
- передача байта починається зі стартового біта, який завжди дорівнює 0 (низький рівень сигналу);
- передача байта закінчується стоповим бітом, який завжди дорівнює 1 (високий рівень сигналу);
- дані передаються, починаючи з молодшого біта;
- для передачі байта потрібно 10 бітів (без використання біта парності);
- час передачі одного байта обчислюється виходячи зі швидкості обміну та кількості бітів у пакеті (10 біт при передачі байта).

Існують стандартні швидкості передачі інтерфейсу UART. Найбільш поширені із них наведені в таблиці 3.1.

Таблиця 3.1. Стандартні швидкості передачі інтерфейсу UART

Швидкість передачі, бод	Час передачі одного біта, мкс	Час передачі байта, мкс
4800	208	2083
9600	104	1042
19200	52	521
38400	26	260
57600	17	174
115200	8,7	87

У мікроконтролерів STM32F103C8T6 є 3 апаратних інтерфейси UART. Вони мають однакові структури та функціональні можливості. Не повторюючи інформацію про загальноприйняті функції UART, можна виділити такі можливості цього інтерфейсу в мікроконтролерах STM32:

- За частоти тактування 72 МГц швидкість передачі до 4,5 Мбіт/с.
- Розмірність даних 8 або 9 біт.

- Можуть бути задані формати з 1 або 2 стоповими бітами.
- Інтерфейс підтримує фізичний рівень інфрачервоного порту (IRDA).
- Також підтримується інтерфейс контактних смарт-карток ISO 7816.
- Передача та прийом даних можуть відбуватися за допомогою контролера прямого доступу до пам'яті (DMA).
- Інтерфейс може працювати у синхронному режимі (USART).

Частина функціональної схеми інтерфейсу, за якою відбувається передача і прийом даних, показана на рис. 3.3.

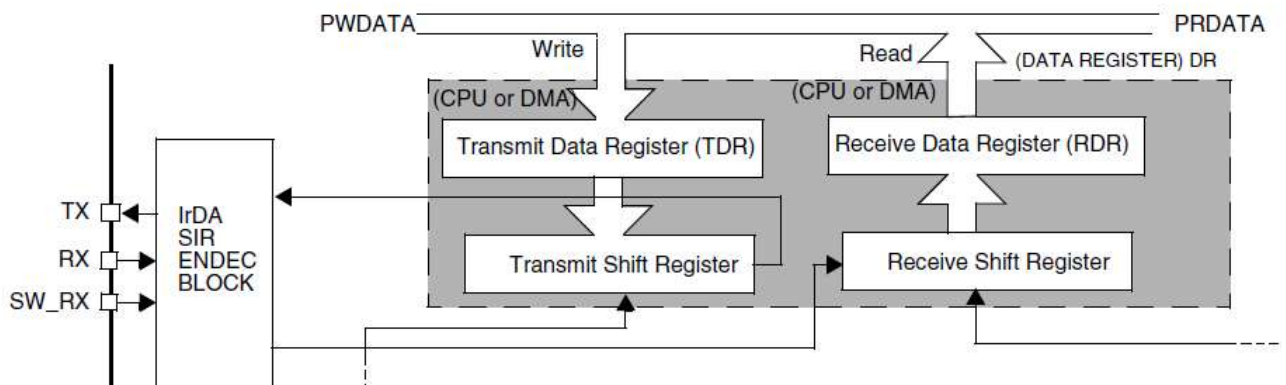


Рис. 3.3. Функціональна схема приймача-передавача UART в STM32

Передавальна та приймальна частина працюють абсолютно незалежно одна від одної. Зі спільного у них лише тактовий генератор. Тобто, може бути задіяний лише приймач, або лише передавач. Приймач та передавач можуть працювати з різними пристроями, різними протоколами верхнього рівня тощо. Спільними у них мають бути швидкість та формат даних.

Власне передавач складається з 2 регістрів: регістру зсуву (Transmit Shift Register) та буферного регістру (TDR). Пакет даних завантажується в буферний регістр передавача. Програмно доступний лише він. Якщо передача попереднього пакету даних завершена і регістр зсуву порожній, то пакет даних перевантажується в нього, зсувається і побітово надходить на вихід TX. Як тільки пакет даних перевантажено до регістру зсуву, буферний регістр звільняється і до нього може бути завантажено новий пакет даних. Він чекатиме на закінчення передачі і автоматично перевантажиться до регістру зсуву.

Таким чином, відбувається буферизація даних передавача. Це дозволяє реалізовувати передачу даних без пауз між пакетами. Після перевантаження буферного регістра є час на запис нового пакету даних, рівний часу передачі. Якщо встигати заповнювати буферний регістр, дані будуть передаватися суцільним потоком.

Про закінчення передачі даних регістром зсуву та звільнення буферного регістру повідомляють спеціальні прапорці. За ними можуть бути сформовані переривання.

Приймальна частина пристрою також складається з двох регістрів: регістру зсуву (Receive Shift Register) та буферного регістру (RDR). З входу RX дані надходять на регістр зсуву, зсуваються і, після формування повного слова, перевантажуються до буферного регістру. Буферний регістр доступний програмно. З нього зчитується отриманий пакет даних. Якщо дані надходять суцільним потоком, без пауз, то після прийому пакету є час на те, щоб зчитати його з буферного регістра. Цей час дорівнює часу передачі слова. Якщо не встигнути зчитати вміст буферного регістру, прийде новий пакет даних, а старий буде втрачено.

Завдяки використанню спеціальних адаптерів інтерфейс UART можна перетворити на більш зрозумілий для ПК порт USB. Одним із таких адаптерів є FT232RL (рис. 3.4.). Він дозволяє розробникам використовувати простий інтерфейс RS-232, не занурюючись у особливості USB-інтерфейсу. На вході модуля – виводи UART для обміну даними, на виході – інтерфейс з роз'ємом Mini USB 2.0. ПК побачить підключений модуль як віртуальний COM-порт.

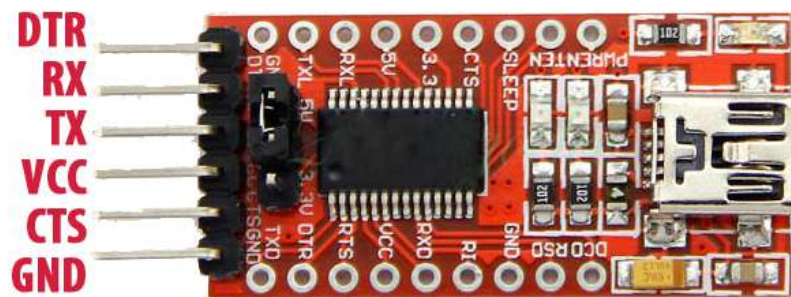


Рис. 3.4. Конвертер USB-UART FT232RL

Хід виконання роботи

1. Виконати попередні налаштування плати «STM32 Blue pill» для використання інтерфейсу USART та LCD дисплею.
 - 1.1. Запустити середовище програмування STM32CubeIDE та створити новий проект.
 - 1.2. Встановити джерело тактування від зовнішнього кварцового резонатора та режим відлагодження «*Serial Wire*».
 - 1.3. На вкладці *Clock Configuration* налаштувати тактування мікроконтролера. Встановити тактування від зовнішнього кварцового резонатора та основну частоту тактування 72 МГц.
 - 1.4. На вкладці *Pinout & Configuration* активувати виводи PB3-PB8 в режимі активних виходів для підключення LCD.
 - 1.5. В розділі *Connectivity* активувати інтерфейс USART2, обравши *Mode*: «Asynchronous». Після цього на графічній моделі мікроконтролера автоматично активуються виводи PA2-PA3, призначені для роботи з USART2.
 - 1.6. В параметрах USART2 встановити швидкість передачі даних на рівні 115200 біт/с, обравши *Baud Rate*: «115200». Встановити довжину слова *Word Length*: «8 Bits (including Parity)», вимкнути необхідність використання біту парності *Parity*: «None» та обрати кількість стопових бітів *Stop Bits*: «1». Тимчасово залишити всі інші налаштування незмінними (рис. 3.5).
 - 1.7. Зберегти проект та провести генерацію коду.
 - 1.8. Розмістити файли бібліотек для роботи з LCD дисплеєм (*lcd.h*, *lcd.c*) у відповідних папках створеного проекту.
 - 1.9. У файлі *main.c* підключити заголовковий файл бібліотеки для керування LCD:

```
/* USER CODE BEGIN Includes */
#include "lcd.h"
/* USER CODE END Includes */
```

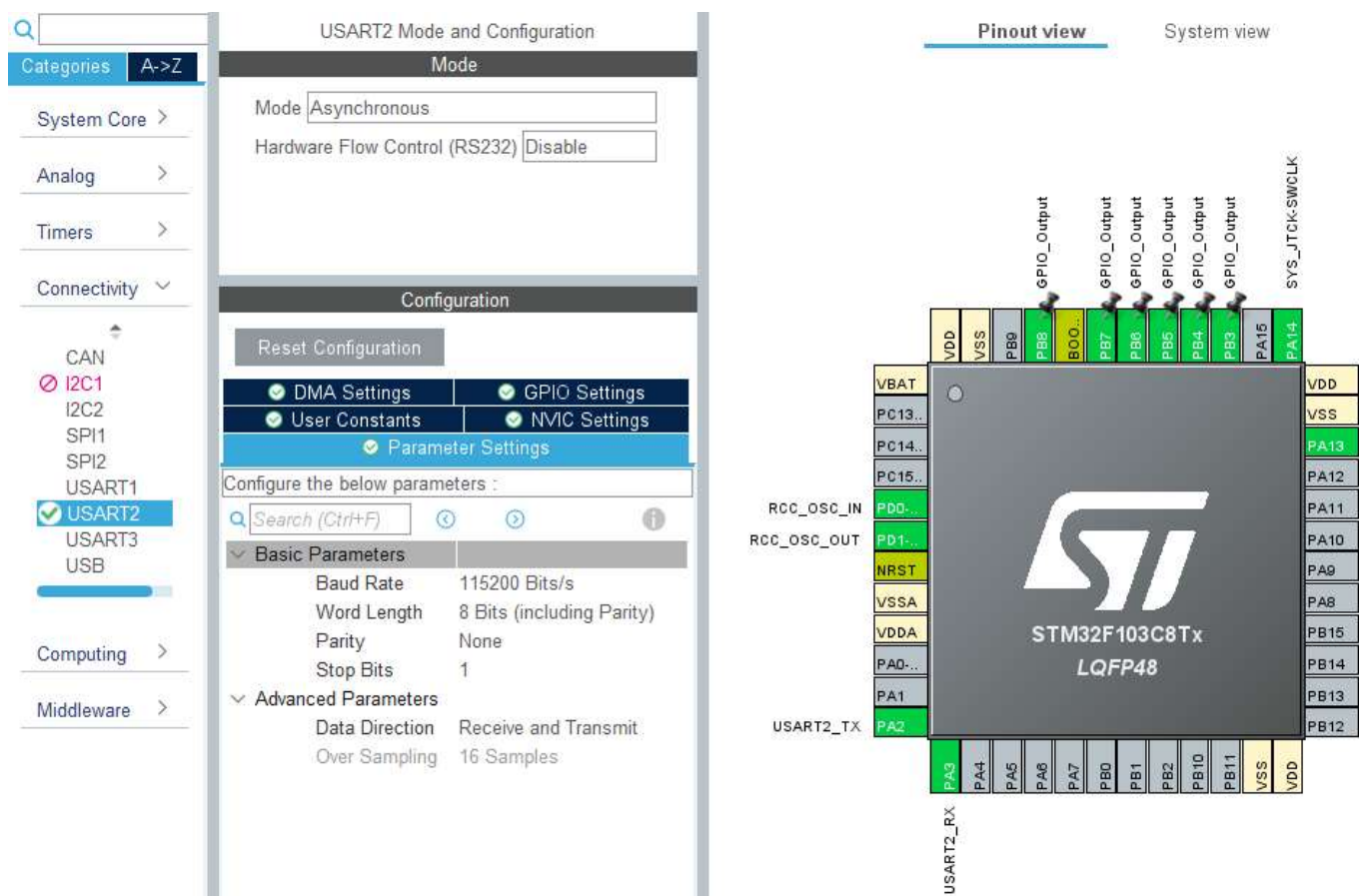



Рис. 3.5. Налаштування мікроконтролера для передачі даних по інтерфейсу USART2 та роботи з LCD

2. Здійснити передачу даних з мікроконтролера на ПК із використанням протоколу UART.

2.1. Створити змінну, в якій буде зберігатись рядок для передачі:

```
/* USER CODE BEGIN 1 */
uint8_t trans[] = "USART Transmit\r\n";
/* USER CODE END 1 */
```

2.2. Для передачі даних за протоколом UART в бібліотеці HAL існує функція **HAL_UART_Transmit()**. Інформація про функцію:

- **Формат:** `HAL_StatusTypeDef HAL_UART_Transmit (UART_HandleTypeDef * huart, uint8_t * pData, uint16_t Size, uint32_t Timeout)`.
- **Аргументи:**
 - *huart* – вказівник на структуру для роботи з UART.

- *pData* – вказівник на буфер даних для передачі.
- *Size* – кількість даних, які потрібно передати.
- *Timeout* – час таймауту операції (зазвичай 0xFFFF).

2.3. У головному циклі `while(1)` написати код для передачі даних, які містяться у раніше створеному масиві *trans*, на ПК. Використовується USART2, а розмір буферу даних має відповідати довжині масиву *trans* (в нашому випадку – 16 байт).

```
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */

    HAL_UART_Transmit(&huart2, trans, 16, 0xFFFF);
    HAL_Delay(100);

}
/* USER CODE END 3 */
```

2.4. Підключити плату STM32 Blue Pill до ПК, використовуючи в якості перехідника модуль FT232RL та кабель USB. Схема підключення модуля до мікроконтролера показана на рис. 3.6.

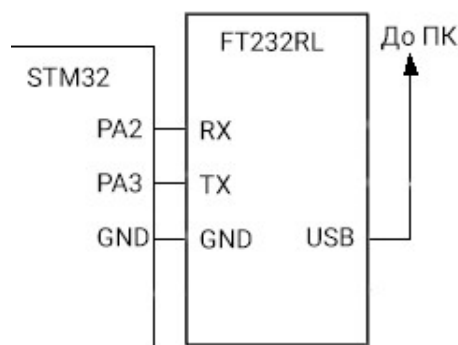


Рис. 3.6. Схема підключення модуля FT232RL до мікроконтролера

2.5. Після підключення до ПК, відкрити Диспетчер пристроїв Windows та дізнатись номер створеного віртуального COM-порту.

2.6. Відкрити програму Terminal для роботи з COM-портом, вказати відповідні налаштування (рис. 3.7) та натиснути кнопку *Connect*. У випадку успішного з'єднання, в нижньому лівому куті з'явиться статус *Connected*.

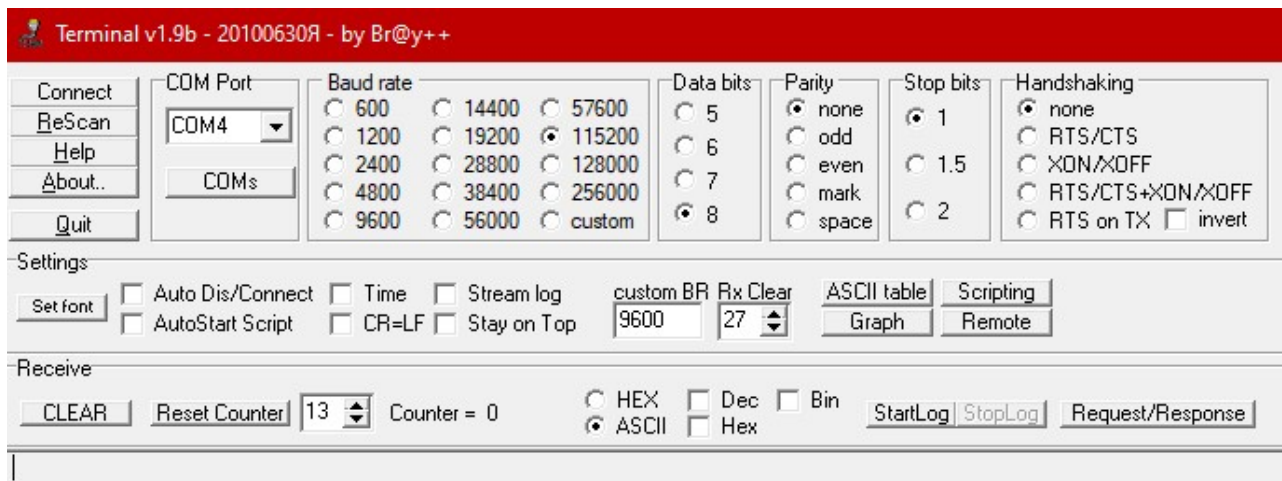


Рис. 3.7. Налаштування програми Terminal (COM-порт може відрізнятись!)

2.7. Тимчасово відключити кабель USB від ПК та натиснути Disconnect у програмі Terminal.

2.8. Завантажити програму до мікроконтролера.

2.9. Підключити кабель USB до ПК та підключитись до COM-порту в програмі Terminal. Критерієм правильності на даному етапі буде циклічна поява в консолі програми Terminal напису «USART Transmit». Це і є той рядок, який ми записували до змінної *trans* та у нескінченному циклі передаємо на ПК із затримкою 100 мс.

3. Передати дані з ПК на мікроконтролер із використанням протоколу UART та відобразити отриманий рядок на LCD.

3.1. У файлі .ioc у налаштуваннях USART2 на вкладці *NVIC Settings* дозволити переривання від USART2, поставивши прапорець «Enabled». Зберегти проект та згенерувати код.

3.2. Створити змінну *received*, до якої будуть записуватись прийняті дані:

```
/* USER CODE BEGIN 1 */
uint8_t trans[] = "USART Transmit\r\n";
char received[9] = {0};
/* USER CODE END 1 */
```

3.3. В основному циклі написати код, який дозволить прийняти дані з ПК по UART в режимі генерації переривань та відобразити їх на дисплеї. Для

прийому даних в режимі переривань використовується функція *HAL_UART_Receive_IT()*. Інформація про функцію:

- Формат: *HAL_StatusTypeDef HAL_UART_Receive_IT (UART_HandleTypeDef * huart, uint8_t * pData, uint16_t Size)*.
- Аргументи:
 - *huart* – вказівник на структуру для роботи з UART.
 - *pData* – вказівник на буфер для прийнятих даних.
 - *Size* – кількість даних, які потрібно прийняти.

Щоб код працював більш коректно, будемо відображати дані на дисплеї тільки тоді, коли до буфера буде прийнято усі дані. Для цього можна відслідковувати стан внутрішнього лічильника UART, доступ до регістру якого можна отримати через параметр *RxXferCount* структури *huart*. Цей лічильник відраховує кількість тактових імпульсів, необхідних для прийому обсягу даних, заданого в *Size*. Коли його значення досягає 0, дані прийнято повністю і їх можна відобразити на дисплеї.

Для початку проведемо ініціалізацію дисплею та у першому його рядку виведемо напис «Received:»:

```
/* USER CODE BEGIN 2 */
    LCD_init();
    LCD_String("Received:");
/* USER CODE END 2 */
```

Після цього в основному циклі реалізуємо описаний вище алгоритм.

```
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */

    if(huart2.RxXferCount==0)
    {
        HAL_UART_Receive_IT(&huart2, (uint8_t*)received, 8);
        LCD_SetPos(0, 1);
        LCD_String(received);
        HAL_Delay(100);
    }
}

/* USER CODE END 3 */
```

3.4. Підключити дисплей LCD 1602 до мікроконтролера за схемою, показаною на рис. 3.8.

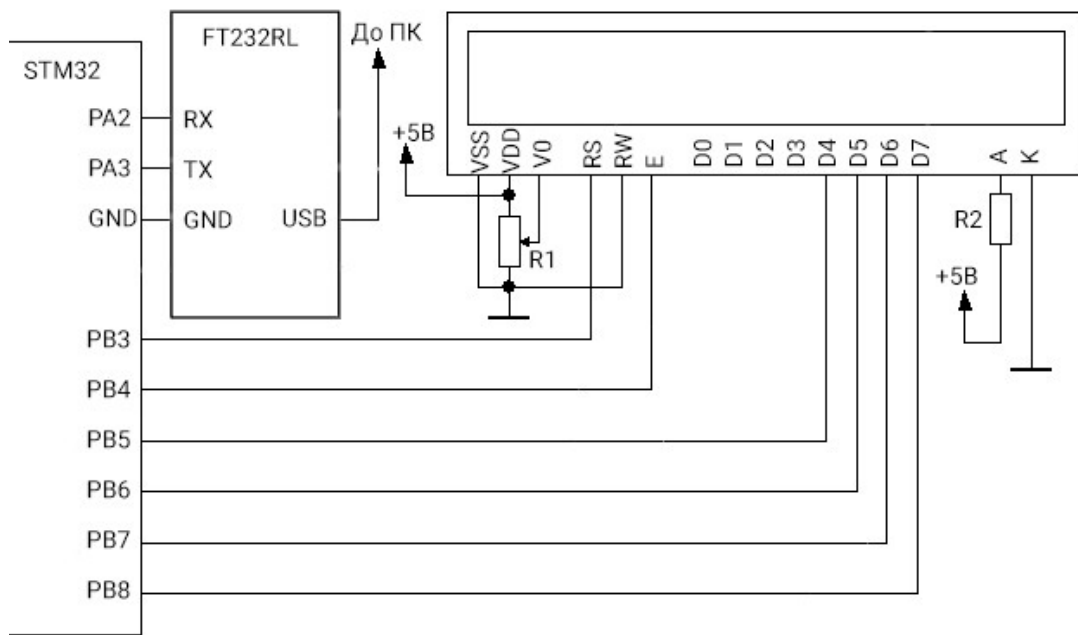


Рис. 3.9. Відправка символів до COM-порту у програмі Terminal

3.5. Тимчасово відключити кабель USB від ПК та натиснути Disconnect у програмі Terminal.

3.6. Завантажити програму до мікроконтролера.

3.7. Підключити кабель USB до ПК та підключитись до COM-порту в програмі Terminal. Після цього ввести будь-які 8 символів у полі для введення тексту у програмі Terminal та натиснути кнопку *Send* для їх відправки на мікроконтролер (рис. 3.9). Якщо все зроблено вірно, у другому рядку дисплею мають відобразитися відправлені символи. Символи не обов'язково відправляти одразу пакетом по 8, однак вони відобразяться на дисплеї лише тоді, коли заповниться масив *received*.

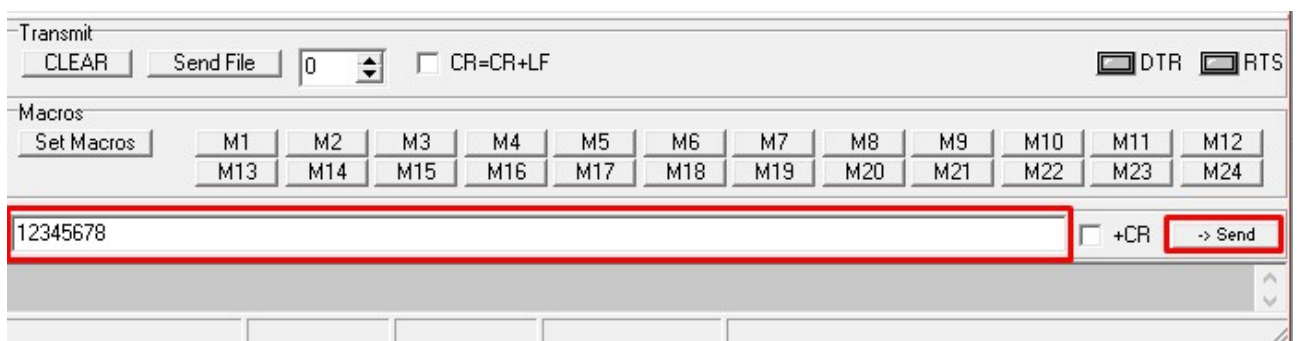


Рис. 3.9. Відправка символів до COM-порту у програмі Terminal

4. Передати дані на ПК із використанням DMA – прямого доступу до пам'яті. DMA дозволяє не завантажувати мікроконтролер операціями зчитування або запису до пам'яті, а одразу передавати дані із буфера UART напряду до зовнішніх пристроїв.

4.1. У файлі .іос в налаштуваннях USART2 на вкладці DMA Settings за допомогою кнопки *Add* додати запити USART2_TX та USART2_RX. Налаштування залишити стандартними. Зберегти проект та згенерувати код.

4.2. В основному циклі використати функцію *HAL_UART_Transmit_DMA()* для передачі даних з масиву *trans* на ПК. Аргументи цієї функції аналогічні аргументам функції для передачі даних через UART у звичайному режимі, окрім таймауту – тут він не потрібен.

```
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */

    HAL_UART_Transmit_DMA(&huart2, trans, sizeof(trans)-1);
    HAL_Delay(100);

}

/* USER CODE END 3 */
```

4.3. Тимчасово відключити кабель USB від ПК та натиснути Disconnect у програмі Terminal.

4.4. Завантажити програму до мікроконтролера.

4.5. Підключити кабель USB до ПК та підключитись до COM-порту в програмі Terminal. У консолі програми мають почати з'являтися передані дані.

5. Прийняти дані з ПК в режимі DMA.

5.1. Переписати код основної програми для приймання даних з ПК з урахуванням роботи в режимі DMA. Для прийому даних в цьому режимі використовується функція *HAL_UART_Receive_DMA()*:

```

while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */

    if(hdma_usart2_rx.State==HAL_DMA_STATE_READY)
    {
        HAL_UART_Receive_DMA(&huart2,
                             (uint8_t*)received, sizeof(received)-1);
        hdma_usart2_rx.State = HAL_DMA_STATE_BUSY;
        LCD_SetPos(0, 1);
        LCD_String(received);
        HAL_Delay(100);
    }
}
/* USER CODE END 3 */

```

- 5.2. Тимчасово відключити кабель USB від ПК та натиснути Disconnect у програмі Terminal.
- 5.3. Завантажити програму до мікроконтролера.
- 5.4. Підключити кабель USB до ПК та підключитись до COM-порту в програмі Terminal. Після цього ввести будь-які символи у полі для введення тексту у програмі Terminal та натиснути кнопку Send для їх відправки на мікроконтролер. Перевірити правильність отримання даних.

Індивідуальні завдання

1. Підключити до мікроконтролера датчик Холла. У випадку піднесення магніту до датчика, на ПК має передаватись рядок «Magnet detected!». Весь інший час на ПК має відправлятись рядок «Searching for magnet...».
2. Підключити до мікроконтролера температурний датчик LM-35. Реалізувати циклічну передачу наступного рядка даних на ПК: «Temperature: XX °C», де XX – виміряне значення температури.
3. Підключити до мікроконтролера комп'ютерний кулер. Використовуючи протокол UART, реалізувати управління кулером з ПК: за командою «Start», відправленою через COM-порт, кулер має почати обертатись. За командою «Stop» обертання має зупинитись.

Контрольні запитання

1. Опишіть принцип роботи та області застосування інтерфейсу передачі даних UART.
2. Проаналізуйте відмінності між інтерфейсами UART та USART.
3. Поясніть процедуру передачі даних за протоколом UART.
4. Поясніть призначення виводів CTS та RTS, обґрунтуйте необхідність їх використання.
5. Розкрийте призначення та необхідність використання біту парності.
6. Наведіть та поясніть схему підключення пристроїв за інтерфейсом UART.
7. Поясніть, що таке «Baud rate» та з яких міркувань обирається швидкість передачі даних за інтерфейсом UART.
8. Обґрунтуйте необхідність використання технології DMA під час передачі даних. Наведіть приклади, коли використання DMA може бути недоцільним.
9. Поясніть, як і для чого виконується перевірка заповненості буфера прийнятих даних під час прийому інформації за протоколом UART із використанням бібліотеки HAL.
10. Наведіть блок-схему структури апаратної частини інтерфейсу UART в мікроконтролерах STM32 та розкрийте принципи її роботи.

РОБОТА З ІНТЕРФЕЙСОМ ПЕРЕДАЧІ ДАНИХ SPI ТА ТЕХНОЛОГІЄЮ RFID

Мета: Ознайомитись з особливостями роботи інтерфейсу передачі даних SPI. Навчитись створювати програмний код для зчитування даних з RFID-пристроїв.

Теоретичні відомості

SPI (англ. Serial Peripheral Interface, SPI bus, 4-wire – послідовний периферійний інтерфейс) – популярний інтерфейс для послідовного обміну даними між мікросхемами. Інтерфейс SPI відноситься до найбільш поширених інтерфейсів для з'єднання мікросхем. Він був розроблений компанією Motorola, а в даний час використовується в продукції багатьох виробників. Як видно з назви, його призначення – шина для підключення зовнішніх (периферійних) пристроїв. Шина SPI організована за принципом «ведучий-підлеглий» (Master - Slave). В якості ведучого шини зазвичай виступає мікроконтролер. Зовнішні пристрої, які підключені до ведучого шини, утворюють підлеглих шини. В їх ролі виступають різного роду мікросхеми, запам'ятовуючі пристрої (Flash-пам'ять, SRAM), АЦП / ЦАП, цифрові потенціометри, спеціалізовані контролери тощо.

Основою інтерфейсу SPI є регістр зсуву, сигнали синхронізації та введення/виведення потоку даних якого і утворюють сигнали інтерфейсу. Таким чином, протокол SPI правильніше назвати не протоколом передачі даних, а протоколом обміну даними між двома регістрами зсуву, кожен з яких одночасно виконує і функцію приймача, і функцію передавача. Обов'язковою умовою передачі даних по SPI є генерація тактового сигналу синхронізації шини. Цей сигнал має право генерувати тільки ведучий і від цього сигналу повністю залежить робота підлеглого.

Згідно з протоколом SPI, один з взаємодіючих пристроїв завжди є ведучим і контролює підлеглі периферійні пристрої. Як правило, всі пристрої об'єднані чотирма загальними лініями:

- **MISO** (Master In Slave Out) – лінія для передачі даних від підлеглого пристрою (Slave) до ведучого (Master);
- **MOSI** (Master Out Slave In) – лінія для передачі даних від ведучого пристрою (Master) до підлеглого (Slave);
- **SCLK** (Serial Clock) – тактовий сигнал, який генерується ведучим пристроєм (Master) для синхронізації процесу передачі даних;
- **SS** (Slave Select) – вивід, призначений для активації ведучим того чи іншого периферійного пристрою.

Периферійний підлеглий пристрій взаємодіє з ведучим тоді, коли на виводі SS присутній сигнал низького рівня. В іншому випадку дані від ведучого пристрою будуть ігноруватися. Така архітектура дозволяє взаємодіяти з декількома SPI-пристроями, підключеними до однієї і тієї ж шини.

Взагалі, існують різні типи підключення до шини SPI. Найпростіше підключення, в якому беруть участь тільки дві мікросхеми, показане на рис. 4.1. В цьому випадку ведучий передає дані по лінії MOSI синхронно зі згенерованим ним же сигналом SCLK, а підлеглий захоплює передані біти даних за певними фронтами прийнятого сигналу синхронізації. Одночасно з цим підлеглий відправляє свою порцію даних.

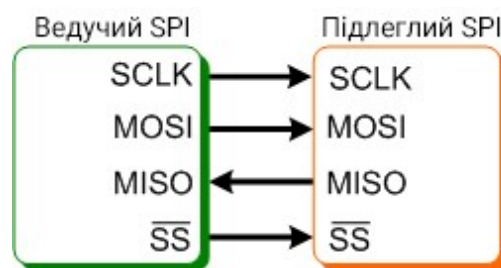


Рис. 4.1. Найпростіше підключення до шини SPI

Представлену схему можна спростити виключенням лінії MISO, якщо використовувана підлегла мікросхема не передбачає зворотну передачу даних або в ній немає потреби.

Частота передачі бітів по лініях передачі даних визначається тактовим сигналом SCLK, який генерує ведучий пристрій. Підлеглі пристрої використовують тактовий сигнал для визначення моментів зміни бітів на лінії даних, водночас підлеглі пристрої ніяк не можуть впливати на частоту слідування бітів. Як в ведучому пристрої, так і в підлеглому пристрої є лічильник імпульсів синхронізації (бітів). Лічильник в підлеглому пристрої дозволяє йому визначити момент закінчення передачі пакета. Лічильник у ведучому пристрої скидається у разі вимкнення підсистеми SPI. У підлеглому пристрої лічильник зазвичай скидається деактивацією сигналу SS.

Передача даних за протоколом SPI здійснюється пакетами. Довжина пакета становить 1 байт (8 біт) або 2 байти (16 біт). Ведучий пристрій ініціює цикл зв'язку установкою низького рівня на виводі вибору підлеглого пристрою (SS), з яким необхідно встановити з'єднання. За низького рівня сигналу SS:

- мікросхема підлеглого пристрою знаходиться в активному стані;
- вивід MISO переводиться в режим «вихід»;
- тактовий сигнал SCLK від ведучого пристрою приймається підлеглим та викликає зчитування на вході MOSI переданих від ведучого пристрою бітів і зсув регістра підлеглого пристрою.

Дані, які підлягають передачі, ведучий та підлеглий пристрій поміщають до регістрів зсуву (рис. 4.2). Після цього, ведучий пристрій починає генерувати імпульси синхронізації на лінії SCLK, що призводить до взаємного обміну даними. Передача даних здійснюється біт за бітом від ведучого по лінії MOSI і від підлеглого по лінії MISO. Передача здійснюється, як правило, починаючи зі старших бітів, але деякі виробники допускають зміну порядку передачі бітів програмними методами. Після передачі кожного пакета даних ведучий пристрій, з метою синхронізації підлеглого пристрою, може перевести лінію SS до високого стану.

Назви виводів інтерфейсу SPI можуть відрізнятися в залежності від виробника мікросхеми. Найбільш розповсюджені аббревіатури показані у таблиці 4.1.

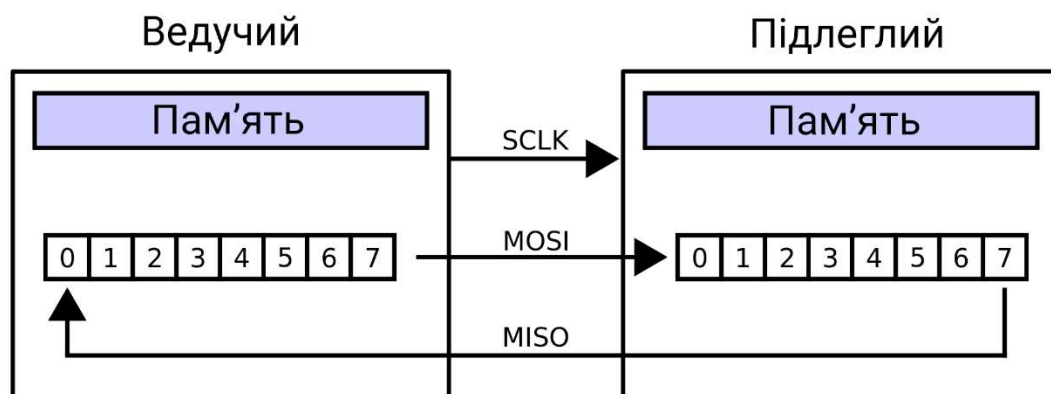


Рис. 4.2. Організація передачі даних по шині SPI

Протокол передачі даних по інтерфейсу SPI є ідентичним роботі регістру зсуву, яка полягає у виконанні операції побітового зсуву та введення / виведення даних за певними фронтами тактового сигналу. Установка даних під час передачі та зчитування під час прийому завжди виконуються за протилежними фронтами тактового сигналу. Це необхідно для гарантування зчитування даних після їх надійного встановлення. Якщо врахувати, що в якості першого фронту в циклі передачі може виступати передній або задній фронт, то всього існує чотири варіанти логіки роботи інтерфейсу SPI (рис. 4.3). Ці варіанти отримали назву **режимів SPI** і описуються двома параметрами:

- **CPOL** – **полярність** (вихідний рівень) сигналу синхронізації (якщо $CPOL = 0$, то лінія синхронізації до початку циклу передачі і після його закінчення має низький рівень (тобто перший фронт передній, а останній – задній). Інакше, якщо $CPOL = 1$, до і після передачі лінія має високий рівень (тобто перший фронт задній, а останній – передній);
- **CPHA** – **фаза синхронізації**; від цього параметра залежить, в якій послідовності виконується установка і зчитування даних. Якщо $CPHA = 0$, то по передньому фронті в циклі синхронізації буде виконуватися зчитування даних, а потім, по задньому фронті – установка даних; якщо ж $CPHA = 1$, то установка даних в циклі синхронізації буде виконуватися по передньому фронті, а зчитування – по задньому.

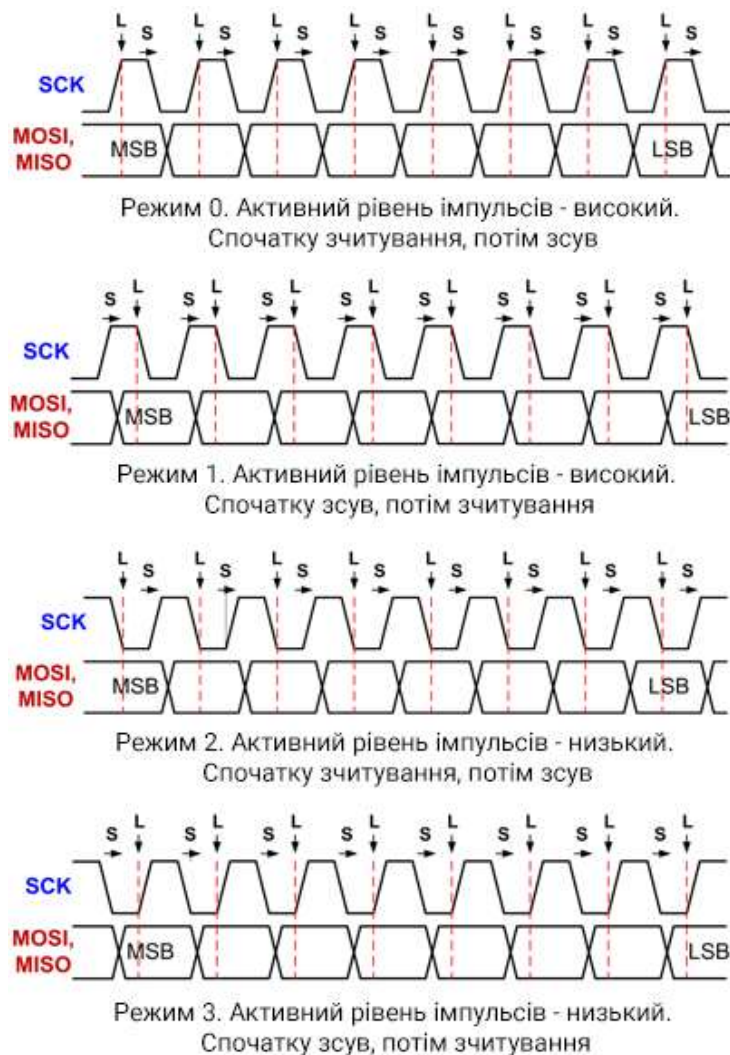


Рис. 4.3. Режими роботи SPI

Таблиця 4.1. Альтернативні позначення виводів SPI

Ведучий шини			Підлеглий шини		
Основне позначення	Альтернативне позначення	Опис	Основне позначення	Альтернативне позначення	Опис
MOSI	DO, SDO, DOUT	Вихід послідовної передачі даних	MOSI	DI, SDI, DIN	Вхід послідовного прийому даних
MISO	DI, SDI, DIN	Вхід послідовного прийому даних	MISO	DO, SDO, DOUT	Вихід послідовної передачі даних
SCLK	DCLOCK, CLK, SCK	Вихід синхронізації передачі даних	SCLK	DCLOCK, CLK, SCK	Вихід синхронізації прийому даних
SS	CS	Вихід вибору підлеглого (вибір схеми)	SS	CS	Вхід вибору підлеглого (вибір схеми)

Для безконтактної передачі даних, наприклад, за інтерфейсом NFC, використовуються технології RFID. **RFID (радіочастотна ідентифікація)** – це метод забезпечення передачі, запису та зберігання даних за допомогою радіосигналів. Дана технологія широко використовується в багатьох галузях для контролю доступу, управління ланцюжками доставки, відстеження бібліотечних книг, системи нарахування бонусів та інших завдань. Система RFID складається з двох основних компонентів: прикріпленого на об'єкті транспондера (мітки) та пристрою зчитування карт (прийомо-передавача), як це показано на рис. 4.4.

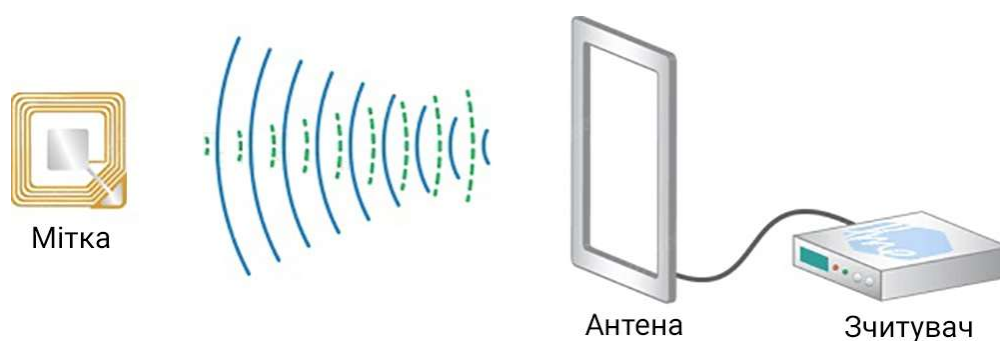


Рис. 4.4. Схема системи RFID

Пристрій зчитування RFID-міток включає в себе радіочастотний модуль, блок управління та антенну котушку, яка генерує високочастотне електромагнітне поле. З іншого боку, мітка зазвичай являє собою пасивний елемент, який складається тільки з антени та мікросхеми. Коли мітка (картка або брелок) знаходиться в зоні електромагнітного поля пристрою зчитування карт (приймача), в її антені утворюється ЕРС індукції, яка використовується в якості джерела живлення мікросхеми (рис. 4.5). Інтегральна мікросхема мітки дозволяє зберігати і обробляти дані, антена – приймати і передавати інформацію.

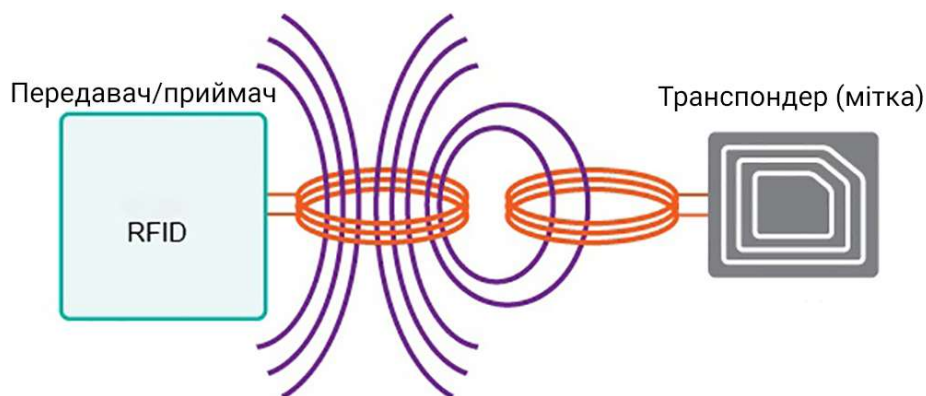


Рис. 4.5. Принцип роботи RFID системи

Всі RFID-системи можна розділити за дальністю дії:

- Близької ідентифікації – відстань не більше 20 см;
- Середньої ідентифікації – відстань від 20 см до 5 м;
- Дальньої ідентифікації – максимум 300 м.

З точки зору частот можна виділити системи, які працюють:

- В низькочастотному діапазоні (125 кГц - 134 кГц);
- В середньочастотному діапазоні (13,56 МГц);
- У високочастотному діапазоні (800 МГц - 2,4 ГГц).

Найбільш популярним діапазоном є середньочастотний – він широко використовується в транспортних додатках та інших проектах, де необхідний перезапис карток. Одним із популярних приймачів є модуль RFID RC522 (рис. 4.6), який працює саме в середньочастотному діапазоні.

Модуль RFID RC522 генерує електромагнітне поле з частотою 13,56 МГц, яке використовується для зв'язку з RFID мітками (стандартні мітки ISO 14443A). Для взаємодії з мікроконтролерами модуль використовує інтерфейс SPI. Також модуль підтримує протоколи зв'язку I2C та UART. Додатково виведений контакт переривання IRQ, який дозволяє опитувати модуль тільки тоді, коли до нього прикладено картку. Максимальна швидкість передачі даних складає 10 Мбіт/с, дальність виявлення мітки – до 6 см (на практиці – до 2-3 см). Для роботи з модулем існує стороння програмна бібліотека MFRC522.

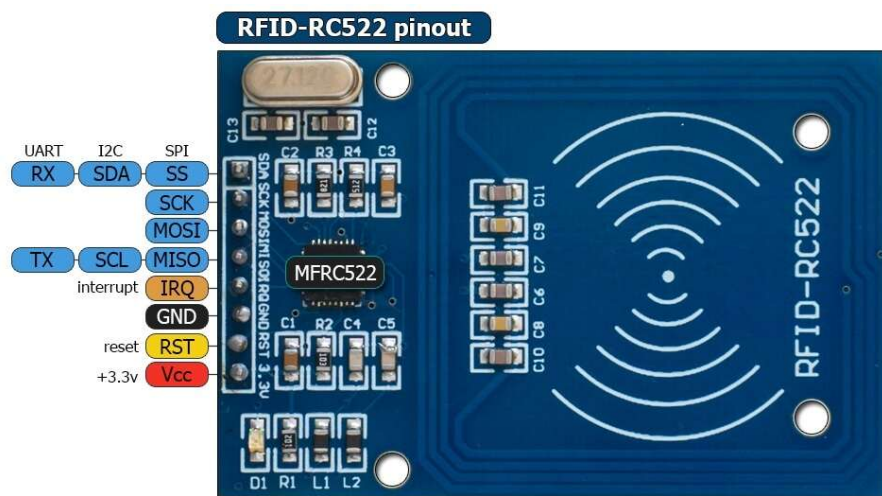


Рис. 4.6. Зовнішній вигляд модуля RC522

Призначення виводів модулю:

- SDA – вибір підлеглого SPI;
- SCK – сигнал синхронізації SPI;
- MOSI – передача від ведучого до підлеглого;
- MISO – передача від підлеглого до ведучого;
- RST – вивід для скидання;
- IRQ – вивід переривання;
- GND – земля;
- Vcc – живлення 3.3 В.

Сигнал скидання RST – це сигнал, який надходить від цифрового виходу контролера. У разі надходженні сигналу низького рівня відбувається перезавантаження зчитувача. Також рідер установкою низького рівня на RST повідомляє мікроконтролеру, що знаходиться в режимі сну. Для виведення модуля з режиму сну необхідно подати на даний вивід сигнал високого рівня.

Існує велика різноманітність RFID-міток. Мітки бувають активні і пасивні (без вбудованого джерела енергії). Мітки працюють у різних частотних діапазонах. Часто у комплекті з модулем RFID-RC522 йдуть дві мітки, одна у вигляді картки, інша у вигляді брелока (рис. 4.7).



Рис. 4.7. Зовнішній вигляд RFID міток

Білою карткою з даного набору є MIFARE Classic EV1 1K – безконтактна картка з 1 КБ вбудованої пам'яті. Кожна картка має власний 4-байтний ідентифікатор (UID). Доступ до блоків пам'яті захищений паролем. Схема організації пам'яті картки показана на рис. 4.8.

Sector	Block	Byte Number within a Block																Description
		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
15	3	Key A					Access Bits					Key B					Sector Trailer 15	
	2																	Data
	1																	Data
	0																	Data
14	3	Key A					Access Bits					Key B					Sector Trailer 14	
	2																	Data
	1																	Data
	0																	Data
⋮	⋮																	
	⋮																	
	⋮																	
	⋮																	
1	3	Key A					Access Bits					Key B					Sector Trailer 1	
	2																	Data
	1																	Data
	0																	Data
0	3	Key A					Access Bits					Key B					Sector Trailer 0	
	2																	Data
	1																	Data
	0	Manufacturer Data																Manufacturer Block

Рис. 4.8. Карта пам'яті картки MIFARE Classic EV1 1K

Пам'ять на карті організована у 16 секторів, кожен з яких складається з чотирьох блоків. Кожен блок, в свою чергу, містить 16 байтів пам'яті. В блоках №3 кожного сектору зберігається заголовок – інформація про ключі доступу до блоку (ключ А і ключ В) та службові біти доступу (Access Bits), які визначають режим роботи з ключами. Для отримання доступу до сектора можна використовувати лише ключ А, лише ключ В або обидва ключі одразу. Ключ А доступний тільки для читання, ключ В може використовуватись для зберігання даних користувача, якщо він не використовується в якості паролю.

Блоки №0-№2 доступні для запису користувацьких даних. Запис або зчитування даних з картки відбувається лише цілими блоками – можливості зчитати/записати окремий байт до блоку відсутня. Блок №0 сектора №0 містить службову інформацію від виробника, тому його неможливо перезаписати. Кожен блок також має власний наскрізний ідентифікатор (0-63), за яким до нього можна звертатися. Приклад вмісту останніх п'яти секторів пам'яті чистої картки (за замовчуванням) показано на рис. 4.9.

Sector	Block	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	AccessBits
15	63	00	00	00	00	00	00	FF	07	80	69	FF	FF	FF	FF	FF	FF	[0 0 1]
	62	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	[0 0 0]
	61	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	[0 0 0]
	60	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	[0 0 0]
14	59	00	00	00	00	00	00	FF	07	80	69	FF	FF	FF	FF	FF	FF	[0 0 1]
	58	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	[0 0 0]
	57	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	[0 0 0]
	56	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	[0 0 0]
13	55	00	00	00	00	00	00	FF	07	80	69	FF	FF	FF	FF	FF	FF	[0 0 1]
	54	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	[0 0 0]
	53	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	[0 0 0]
	52	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	[0 0 0]
12	51	00	00	00	00	00	00	FF	07	80	69	FF	FF	FF	FF	FF	FF	[0 0 1]
	50	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	[0 0 0]
	49	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	[0 0 0]
	48	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	[0 0 0]
11	47	00	00	00	00	00	00	FF	07	80	69	FF	FF	FF	FF	FF	FF	[0 0 1]
	46	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	[0 0 0]
	45	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	[0 0 0]
	44	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	[0 0 0]

Рис. 4.9. Приклад вмісту деяких секторів пам'яті картки MIFARE Classic EV1

Для роботи з картою необхідно виконати наступні операції:

1. Провести ініціалізацію рідера;
2. Відправити стандартний запит для пошуку картки;
3. Увімкнути процедуру антиколізії – для роботи лише з однією картою навіть у випадку, коли до рідера їх прикладено декілька;
4. Обрати потрібну картку за UID;
5. Пройти авторизацію у відповідному секторі;
6. Виконати операцію зчитування/запису;
7. Перевести рідер до режиму очікування.

Хід виконання роботи

1. Виконати попередні налаштування плати «STM32 Blue pill» для підключення RFID-модуля RC522 та LCD дисплею.
 - 1.1. Запустити середовище програмування STM32CubeIDE та створити новий проект.
 - 1.2. Встановити джерело тактування від зовнішнього кварцового резонатора та режим відлагодження «*Serial Wire*».
 - 1.3. На вкладці *Clock Configuration* налаштувати тактування мікроконтролера. Встановити тактування від зовнішнього кварцового резонатора та основну частоту тактування 72 МГц.
 - 1.4. На вкладці *Pinout & Configuration* активувати виводи PB3-PB8 в режимі активних виходів для підключення LCD.
 - 1.5. В розділі *Connectivity* активувати інтерфейс SPI1, обравши *Mode*: «Full-Duplex Master». Після цього на графічній моделі мікроконтролера автоматично активуються виводи PA5-PA7, призначені для роботи з SPI1.
 - 1.6. Додатково вручну активувати вивід PA4, який буде виконувати роль лінії SS інтерфейсу SPI.
 - 1.7. В параметрах SPI1 встановити подільник частоти для шини SPI на рівні 32 для забезпечення швидкості передачі даних 2,25 Мбіт/с, обравши *Prescaler (for Baud Rate)*: «32». Всі інші налаштування залишити незмінними (перевірити біти *CPOL*: «Low» та *CPHA*: «1 Edge» для визначення режиму роботи SPI, рис. 4.10).
 - 1.8. Додатково активувати вивід користувацького світлодіода PC13 для роботи в режимі активного виходу з високим рівнем за замовчуванням.
 - 1.9. Зберегти проект та провести генерацію коду.
 - 1.10. Розмістити файли бібліотек для роботи з LCD дисплеєм (*lcd.h*, *lcd.c*) та з рідером RC522 (*rc522.h*, *rc522.c* – отримати у викладача) у відповідних папках створеного проекту.

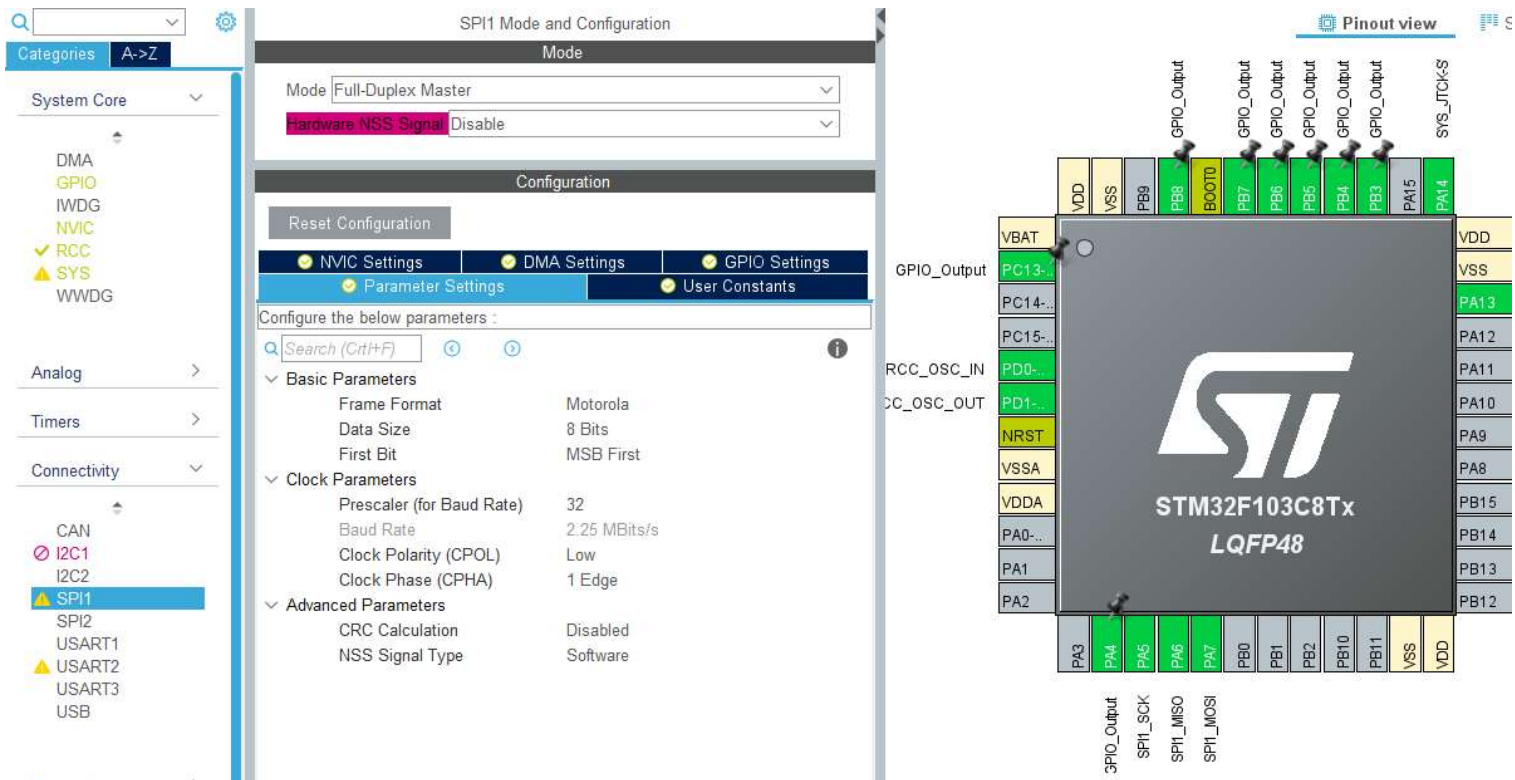


Рис. 4.10. Налаштування мікроконтролера для роботи з SPI та LCD

1.11. Підключити заголовкові файли зазначених бібліотек у відповідному блоці коду, додатково підключивши бібліотеку `STDIO`:

```
/* USER CODE BEGIN Includes */
#include "rc522.h"
#include "lcd.h"
#include "stdio.h"
/* USER CODE END Includes */
```

1.12. Створити необхідні змінні:

```
/* USER CODE BEGIN PV */

uint8_t status;           // службова змінна
uint8_t str[MFR522_MAX_LEN]; // змінна для обміну даними
uint8_t sn[4];           // серійний номер картки
char CardID[8];           // серійний номер картки (символьний)
char DataChar[2][24];     // дані для виводу на дисплей

/* USER CODE END PV */
```

1.13. В блоці ініціалізації провести ініціалізацію дисплею:

```
/* USER CODE BEGIN 2 */
LCD_init();
/* USER CODE END 2 */
```

1.14. Залишити пустим цикл **while(1)** та підключити рідер і LCD до мікроконтролера за схемою, показаною на рис. 4.11.

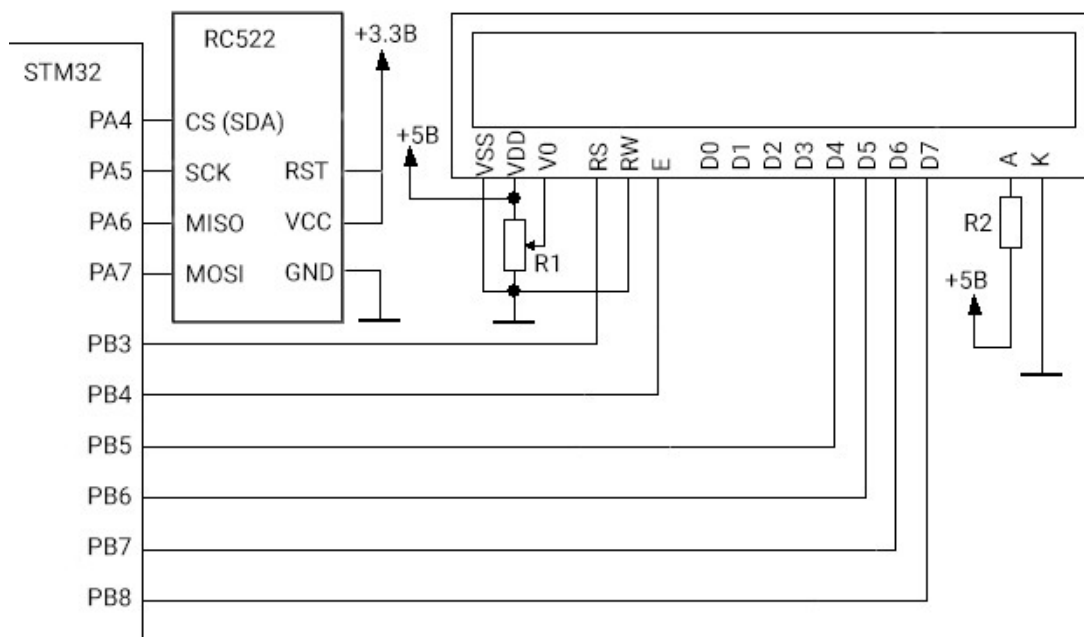


Рис. 4.11. Підключення RC522 та LCS до мікроконтролера

1.15. Завантажити програму до мікроконтролера. Критерієм правильності на даному етапі буде вимкнений користувацький світлодіод та увімкнений світлодіод PWR на рідері RC522.

2. Написати програму для зчитування ідентифікатора (UID) картки та виводу його на дисплей. У випадку, коли картка піднесена до рідера та виявлена (є зв'язок з картою), має запалюватись користувацький світлодіод.

2.1. Модифікувати код циклу **while(1)** наступним чином:

```
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */
    /* USER CODE BEGIN 3 */
    // Старт пошуку картки
    MFRC522_Init();
    status = MI_ERR;

    // Перевірка присутності картки
    status = MFRC522_Request(PICC_REQIDL, str);
    if (status == MI_OK)
    {
        HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_RESET);
    }
}
```

```

else
{
    HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_SET);
    LCD_Clear();
    LCD_String("No card");
    MFRC522_StopCrypto1();
    HAL_Delay(200);
    continue;
}

// Антиколізія, зчитування номера картки
status = MFRC522_Anticoll(sn);
if (status == MI_OK)      // якщо є зв'язок з карткою
{
    snprintf(CardID, sizeof(CardID), "%02X%02X%02X%02X", sn[0],
sn[1], sn[2], sn[3]);
    LCD_Clear();
    LCD_String(CardID);
}
else
{
    LCD_Clear();
    LCD_String("UID error");
    HAL_Delay(500);
    MFRC522_StopCrypto1();
    continue;
}

// Завершення циклу опитування картки
// Переведення рідера до режиму сну
MFRC522_Halt();
// Вимкнення антени
MFRC522_AntennaOff();
MFRC522_StopCrypto1();
HAL_Delay(500);
}
/* USER CODE END 3 */

```

- 2.2. Завантажити програму до мікроконтролера та перевірити правильність її роботи. Критерієм правильності буде запалювання користувацького світлодіода у випадку піднесення карти до рідера на відстань до 2 см та відображення на дисплеї зчитаного номера картки (4 байти).
3. Модифікувати програму для зчитування та виведення на дисплей даних з блоку №3 (сектор 0).
 - 3.1. Ввести додаткові змінні, в тому числі ключі доступу A:

```

/* USER CODE BEGIN PV */
uint8_t status;      // службова змінна
uint8_t str[MFRC522_MAX_LEN]; // змінна для обміну даними
uint8_t sn[5] = {0}; // серійний номер картки
char CardID[9];      // серійний номер картки (символьний)
char DataChar[2][24]; // дані для виводу на дисплей
uint8_t blockAddr;   // адреса блоку даних
// блок ключів A
uint8_t sectorKeyA[16][16] = {{0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF},
                                {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF},
                                {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF},
                                {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF}};

uint8_t sel;         // для перевірки чи правильно обрана картка

/* USER CODE END PV */

```

3.2. Доповнити код основного циклу наступним чином:

```

/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */
    /* USER CODE BEGIN 3 */
        // Старт пошуку картки
        MFRC522_Init();
        status = MI_ERR;

        // Перевірка присутності картки
        status = MFRC522_Request(PICC_REQIDL, str);
        if (status == MI_OK)
        {
            HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_RESET);
        }
        else
        {
            HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_SET);
            LCD_Clear();
            LCD_String("No card");
            MFRC522_StopCrypto1();
            HAL_Delay(200);
            continue;
        }
        // Антиколізія, зчитування номера картки
        status = MFRC522_Anticoll(sn);
        if (status == MI_OK) // якщо є зв'язок з картою
        {
            snprintf(CardID, sizeof(CardID), "%02X%02X%02X%02X", sn[0],
sn[1], sn[2], sn[3]);
            LCD_Clear();
            LCD_String(CardID);
        }
    }
}

```

```

else
{
    LCD_Clear();
    LCD_String("UID error");
    HAL_Delay(500);
    MFRC522_StopCrypto1();
    continue;
}

// Вибір картки
sel = MFRC522_SelectTag(sn);
if (sel == 0)
{
    LCD_Clear();
    LCD_String("SEL error");
    MFRC522_StopCrypto1();
    HAL_Delay(500);
    continue;
}

// Авторизація для читання
blockAddr = 3; // звернення до блоку 3 для авторизації в секторі 0
status = MFRC522_Auth(PICC_AUTHENT1A, blockAddr, sectorKeyA[0],
sn);
if (status == MI_OK)
{
    // Зчитування даних з блоку 3
    blockAddr = 3;
    status = MFRC522_Read(blockAddr, str);
    if (status == MI_OK)
    {
        sprintf(DataChar[0],
"%02X%02X%02X%02X%02X%02X%02X%02X",
str[0],str[1],str[2],str[3],str[4],str[5],str[6],str[7]);
        sprintf(DataChar[1],
"%02X%02X%02X%02X%02X%02X%02X%02X",
str[8],str[9],str[10],str[11],str[12],str[13],str[14],str[15]);
        LCD_Clear();
        LCD_String(DataChar[0]);
        LCD_SetPos(0, 1);
        LCD_String(DataChar[1]);
    }
}
else
{
    LCD_Clear();
    LCD_String("AUTH error");
    MFRC522_StopCrypto1();
    HAL_Delay(500);
    continue;
}
// Завершення циклу опитування картки

```



```

    // Переведення рідера до режиму сну
    MFRC522_Halt();
    // Вимкнення антени
    MFRC522_AntennaOff();
    MFRC522_StopCrypto1();
    HAL_Delay(500);
}
/* USER CODE END 3 */

```

3.3. Завантажити програму до мікроконтролера та перевірити правильність її роботи. У випадку правильної роботи після встановлення зв'язку з картою на дисплеї має виводитись інформація, яка міститься в блоці №3 сектора 0. Оскільки даний блок є заголовковим, то, скоріш за все, інформація буде наступного вмісту:

«00 00 00 00 00 00 FF 07 80 69 FF FF FF FF FF»

Перші шість байтів відповідають ключу А, тому карта з метою безпеки замінює значення ключів на нулі. «FF 07 80 69» – це біти доступу (Access bits) за замовчуванням. «FF FF FF FF FF FF» – ключ В (повертається картою, оскільки не використовується).

4. Модифікувати програму для запису довільних даних до блоку №1 (сектор 0). Записані дані мають зчитуватися та виводитися на дисплей. Передбачити лише однократний запис після запуску мікроконтролера (оскільки карта має обмежене число перезаписів). Виводити повідомлення про успішний запис.

4.1. Додати необхідні змінні:

```

/* USER CODE BEGIN PV */

uint8_t status;           // службова змінна
uint8_t str[MFRC522_MAX_LEN]; // змінна для обміну даними
uint8_t sn[5] = {0};      // серійний номер картки
char CardID[9];           // серійний номер картки (символьний)
char DataChar[2][24];     // дані для виводу на дисплей
uint8_t blockAddr;       // адреса блоку даних
// блок ключів А
uint8_t sectorKeyA[16][16] = {{0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF},
                               {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF},
                               {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF},
                               {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF}},};

uint8_t sel;              // для перевірки чи правильно обрана картка
uint8_t write_on = 1;    // ознака дозволу на запис даних
// довільні дані користувача для запису на картку

```



```
uint8_t Data[MFRC522_MAX_LEN] = {0x00, 0x01, 0x02, 0x03, 0x04, 0x05,
0x06, 0x07, 0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F};
```

```
/* USER CODE END PV */
```

4.2. Модифікувати код головної програми:

```
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */
    /* USER CODE BEGIN 3 */
    // Старт пошуку картки
    MFRC522_Init();
    status = MI_ERR;

    // Перевірка присутності картки
    status = MFRC522_Request(PICC_REQIDL, str);
    if (status == MI_OK)
    {
        HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_RESET);
    }
    else
    {
        HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_SET);
        LCD_Clear();
        LCD_String("No card");
        MFRC522_StopCrypto1();
        HAL_Delay(200);
        continue;
    }

    // Антиколізія, зчитування номера картки
    status = MFRC522_Anticoll(sn);
    if (status == MI_OK) // якщо є зв'язок з карткою
    {
        snprintf(CardID, sizeof(CardID), "%02X%02X%02X%02X", sn[0],
sn[1], sn[2], sn[3]);
        LCD_Clear();
        LCD_String(CardID);
    }
    else
    {
        LCD_Clear();
        LCD_String("UID error");
        HAL_Delay(500);
        MFRC522_StopCrypto1();
        continue;
    }

    // Вибір картки
```

```

sel = MFRC522_SelectTag(sn);
if (sel == 0)
{
    LCD_Clear();
    LCD_String("SEL error");
    MFRC522_StopCrypto1();
    HAL_Delay(500);
    continue;
}

//Якщо запис дозволено
if(write_on)
{
    // Авторизація для запису
    blockAddr = 3; // звернення до блоку 3 для авторизації в
секторі 0
    status = MFRC522_Auth(PICC_AUTHENT1A, blockAddr,
sectorKeyA[0], sn);
    if (status == MI_OK)
    {
        // Записування даних до блоку 1
        blockAddr = 1;
        Data[0] = 0xFF;
        status = MFRC522_Write(blockAddr, Data);
        if (status == MI_OK)
        {
            sprintf(DataChar[0], "Success!");
            write_on = 0; // заборонити наступний запис
            LCD_Clear();
            LCD_String(DataChar[0]);
            HAL_Delay(1000);
        }
        else
        {
            sprintf(DataChar[0], "Fail");
            LCD_Clear();
            LCD_String(DataChar[0]);
            MFRC522_StopCrypto1();
            HAL_Delay(1000);
            continue;
        }
    }
    else
    {
        LCD_Clear();
        LCD_String("AUTH error");
        MFRC522_StopCrypto1();
        HAL_Delay(500);
        continue;
    }
}
}

```

```

// Авторизація для читання
blockAddr = 3; // звернення до блоку 3 для авторизації в секторі 0
status = MFRC522_Auth(PICC_AUTHENT1A, blockAddr, sectorKeyA[0],
sn);
if (status == MI_OK)
{
    // Зчитування даних з блоку 1
    blockAddr = 1;
    status = MFRC522_Read(blockAddr, str);
    if (status == MI_OK)
    {
        sprintf(DataChar[0],
"%02X%02X%02X%02X%02X%02X%02X%02X",
str[0],str[1],str[2],str[3],str[4],str[5],str[6],str[7]);
        sprintf(DataChar[1],
"%02X%02X%02X%02X%02X%02X%02X%02X",
str[8],str[9],str[10],str[11],str[12],str[13],str[14],str[15]);
        LCD_Clear();
        LCD_String(DataChar[0]);
        LCD_SetPos(0, 1);
        LCD_String(DataChar[1]);
    }
}
else
{
    LCD_Clear();
    LCD_String("AUTH error");
    MFRC522_StopCrypto1();
    HAL_Delay(500);
    continue;
}
// Завершення циклу опитування картки
// Переведення рідера до режиму сну
MFRC522_Halt();
// Вимкнення антени
MFRC522_AntennaOff();
MFRC522_StopCrypto1();
HAL_Delay(500);
}
/* USER CODE END 3 */

```

4.3. Завантажити програму до мікроконтролеру та перевірити результат. Під час першого піднесення картки до рідера, на неї мають записатися дані, вказані у змінній *Data*. У разі успішного запису, на дисплеї з'явиться повідомлення «Success!», після чого виведеться інформація з блоку 1, який вже буде містити записані дані.

Індивідуальні завдання

1. Розробити повноцінну ID-картку. Під час першого піднесення картки до рідера, на неї має бути записано прізвище, ім'я, дата народження та місто проживання особи, а також зчитано і збережено ідентифікатор (UID) картки. Під час наступних піднесень даної картки до рідера, на дисплеї має виводитись вся записана інформація про її власника. Вивід інформації має відбуватися лише у випадку, якщо UID картки співпадає із раніше запам'ятованим (система не має реагувати на інші картки). Для порівняння UID карток можна використати функцію *MFRC522_Compare(Card1, Card2)*.
2. Розробити транспортну картку. Під час першого піднесення картки до рідера, на неї має записуватись 5 поїздок. Під час наступних піднесень кількість поїздок має зменшуватися на 1 за кожне піднесення, а на дисплеї виводитись повідомлення про успішну оплату проїзду та кількість поїздок, що залишилися на картці. У випадку закінчення кількості поїздок має виводитись повідомлення про заборону проїзду та пропозиція поповнити картку.
3. Розробити депозитну картку. Під час першого піднесення картки до рідера, на неї має бути записана певна грошова сума. Сума вводиться за допомогою кнопки: одне коротке натискання кнопки має записувати на картку 25 грн. У випадку, якщо сума на картці перевищить 250 грн, на дисплей має вивестись повідомлення про перевищення ліміту. Довге натискання на кнопку має вимикати режим внесення коштів. Під час наступних піднесень картки до рідера сума на картці повинна зменшуватись на 50 грн. У випадку закінчення коштів на картці, на дисплеї має виводитись відповідне повідомлення.

Контрольні запитання

1. Опишіть принцип роботи та області застосування інтерфейсу передачі даних SPI.

2. Поясніть апаратну структуру інтерфейсу SPI: опишіть призначення ліній шини та порядок передачі даних між пристроями.
3. Наведіть та поясніть часові діаграми роботи шини SPI в різних режимах.
4. Розкрийте особливості роботи та області використання технології RFID.
5. Опишіть основні характеристики та спосіб підключення модуля RC522 до мікроконтролера.
6. Поясніть призначення та принцип роботи RFID-міток.
7. Опишіть характеристики та структуру організації пам'яті картки MIFARE Classic EV1 1K.
8. Поясніть алгоритм роботи з рідером RC522 для зчитування даних з мітки.
9. Як можна записати інформацію до блоку 2 сектора 3 картки MIFARE?

РОБОТА З ІНТЕРФЕЙСОМ ПЕРЕДАЧІ ДАНИХ I2C, OLED ДИСПЛЕЄМ ТА МОДУЛЕМ ПУЛЬСОКСИМЕТРУ

Мета: Ознайомитись з особливостями роботи інтерфейсу передачі даних I2C. Навчитись створювати програмний код для зчитування даних з датчиків за допомогою інтерфейсу I2C.

Теоретичні відомості

I2C (англ. Inter-Integrated Circuit, – схема внутрішнього зв'язку, 2-wire Serial Interface) – послідовний інтерфейс передачі даних, розроблений компанією Philips. Виробники мікросхем та модулів часто використовують саме I2C для організації зв'язку з зовнішніми пристроями (блоки екранів, дисплеї, деякі камери в смартфонах), а також мікросхемами периферії: АЦП / ЦАП пам'ять, гіроскопи, акселерометри, компаси, драйвери світлодіодів і матриць, ШІМ-контролери, синтезатори частот тощо.

Особливістю шини є можливість одночасного підключення великої кількості пристроїв – від сотень до тисячі, використовуючи при цьому лише два виводи мікроконтролера. В основному шина працює на швидкості 100 Кбіт/с. Також є режим з підвищеною швидкістю – до 3,4 Мбіт/с. Хоча більшість пристроїв спокійно працюють на швидкості 100-500 Кбіт/с.

Фізично шина I2C складається з двох проводів (крім землі і живлення), які підтягнуті до джерела живлення резисторами (1-10 кОм). Проводи інтерфейсу наступні:

- Провід даних (**SDA – Serial DAta**) – ця лінія відповідає безпосередньо за передачу даних.
- Провід тактування (**SCL – Serial CLock**) – ця лінія відповідає за синхронізацію з'єднання.

Як і в інтерфейсі SPI, на шині I2C існує **ведучий** (master) та **підлеглий** (slave). Ведучий генерує тактовий сигнал, підлеглий лише підтверджує прийом байта. В залежності від стандарту, на одній шині може бути від 127 до 1023 пристроїв. Схема підключення пристроїв до шини I2C показана на рис. 5.1.

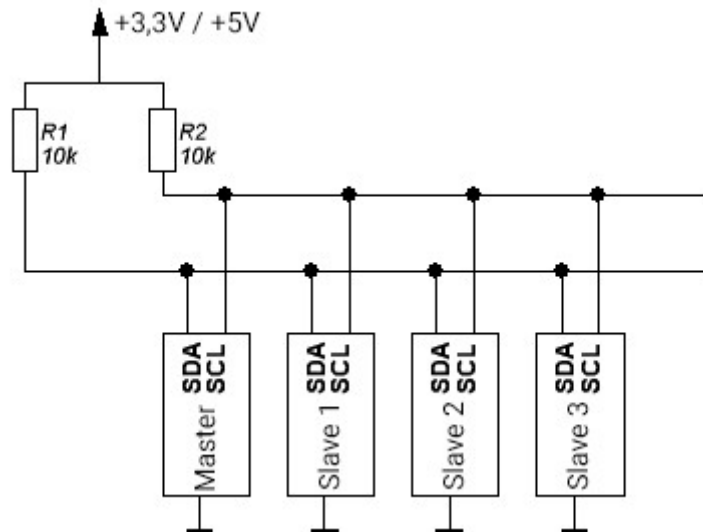


Рис. 5.1. Схема підключення пристроїв до шини I2C

Процес передачі даних включає два етапи: START і STOP. Для організації зв'язку використовуються різні комбінації логічних рівнів на лініях SDA і SCL – вони виставляються в нуль або одиницю в залежності від етапу.

Перед передачею даних виставляється положення START – виконується перехід на низький рівень (з «1» на «0») на SDA за високого рівня на SCL. Якщо дані йдуть від ведучого до підлеглого (рис. 5.2), то підлеглий приймає їх, коли на SCL встановлена одиниця. Причому сигнали на обох лініях виставляє ведучий.

У зворотній ситуації, коли підлеглий передає дані ведучому, лінію SDA встановлює підлеглий (рис. 5.3). Тут біти на лінії даних виставляються тільки за нульового рівня на SCL. Окрім прийому даних, ведучий займається зміною рівнів на лінії синхронізації. Він виставляє на SCL високий рівень, щоб стежити за тим, що робить підлеглий, і водночас зчитує виставлені біти на лінії даних.

Щоб зупинити передачу даних необхідно встановити положення STOP – виконати перехід на високий рівень (з «0» на «1») лінії SDA за високого рівня на SCL. Слід зазначити, що **під час читання даних зміна на лінії SDA можлива лише за низького рівня на лінії синхронізації.**

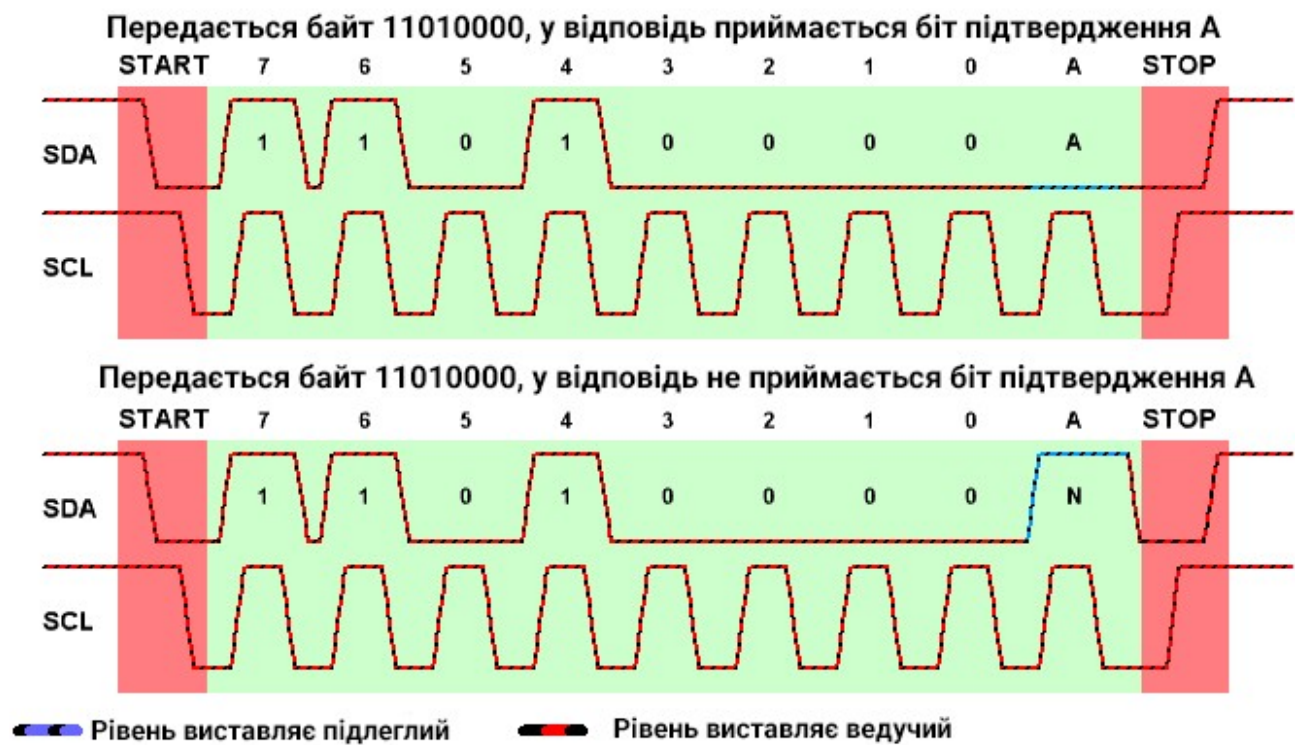


Рис. 5.2. Процес передачі даних від ведучого до підлеглого

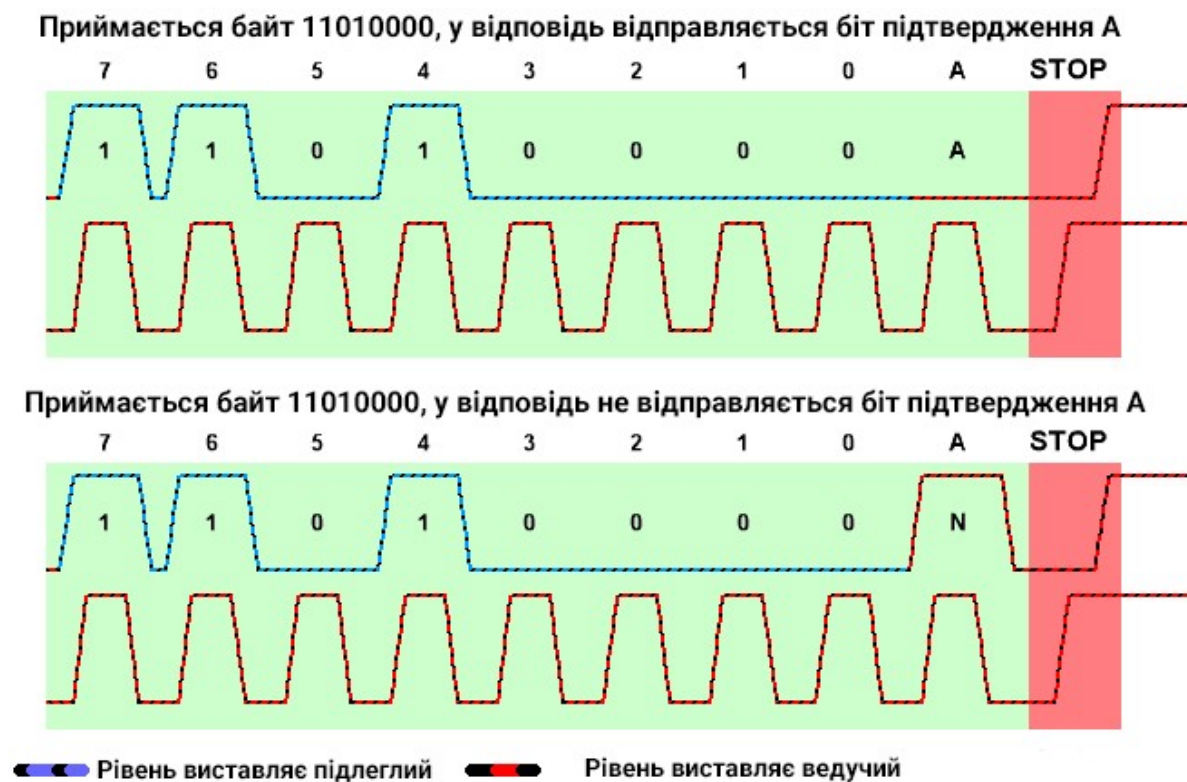


Рис. 5.3. Процес передачі даних від підлеглого до ведучого

Коли на лінії SCL високий рівень – йде читання. Якщо стан лінії SDA змінюється за високого рівня SCL, то це – службові команди START або STOP.

Після старту, передавач одного біта даних йде за тактовим імпульсом. Коли на лінії SCL логічний «0», ведучий або підлеглий виставляють біт даних на SDA (заземлюють лінію, якщо передається «0», або не заземлюють, якщо «1»), після чого SCL переводиться до високого рівня і ведучий / підлеглий зчитують біт. Таким чином, протокол абсолютно не залежить від часових інтервалів, тільки від зміни стану тактових бітів. Тому шину I2C дуже легко налагоджувати.

Отже, повний цикл передачі даних (рис.5.4) складається з таких кроків:

1. Початок передачі визначається START-послідовністю – встановленням низького рівня на лінії SDA за високого рівня на SCL;
2. Під час передачі інформації від ведучого до підлеглого, ведучий генерує тактові імпульси синхронізації на SCL та встановлює біти даних на SDA. Підлеглий зчитує ці біти, коли на лінії SCL встановлюється логічна «1».
3. Під час передачі інформації від підлеглого до ведучого, ведучий генерує такти на SCL і відслідковує інформацію на лінії SDA – зчитує дані. А підлеглий, коли на SCL з'являється логічний «0», виставляє на SDA біт даних, який ведучий зчитує, коли на SCL знову з'являється високий рівень.
4. Процес завершується STOP-послідовністю, тобто коли за високого рівня на SCL лінія SDA переходить з низького на високий рівень.

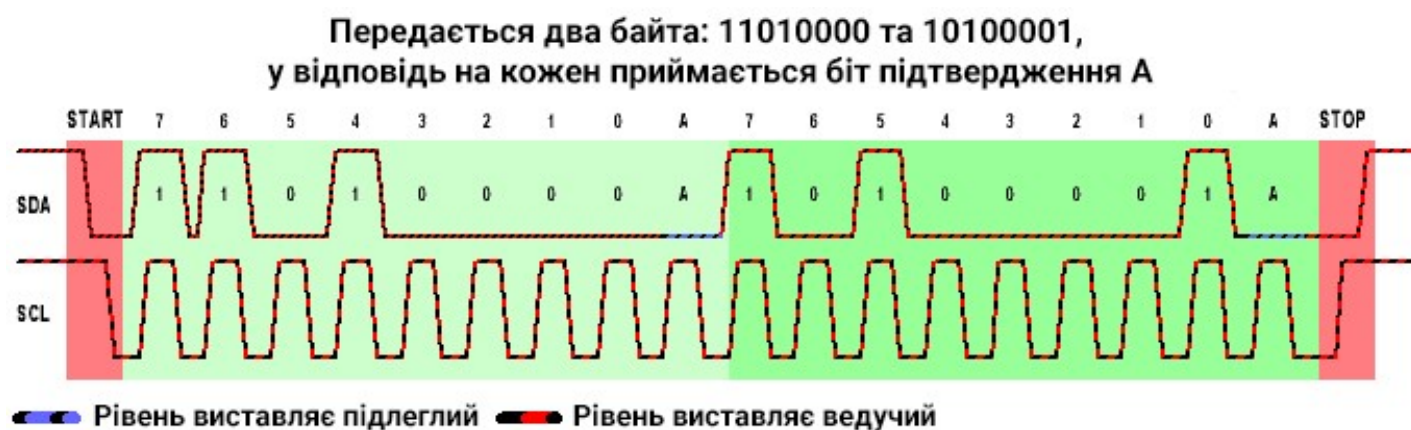


Рис. 5.4. Процес передачі двох байтів даних

Якщо підлеглий є повільним пристроєм та не встигає за ведучим (наприклад, у EEPROM низька швидкість запису), то він може примусово встановити лінію SCL до низького стану і не давати ведучому генерувати нові такти. Ведучий повинен це зрозуміти і дати підлеглому час на зчитування байту.

Тому не можна лише генерувати такти – під час встановлення логічної «1» на SCL треба впевнитись, що на лінії дійсно встановився високий рівень. А якщо не встановився, то треба зупинитися і чекати до тих пір, поки підлеглий її не «відпустить» лінію. Потім продовжити процес з того ж місця (рис. 5.5).

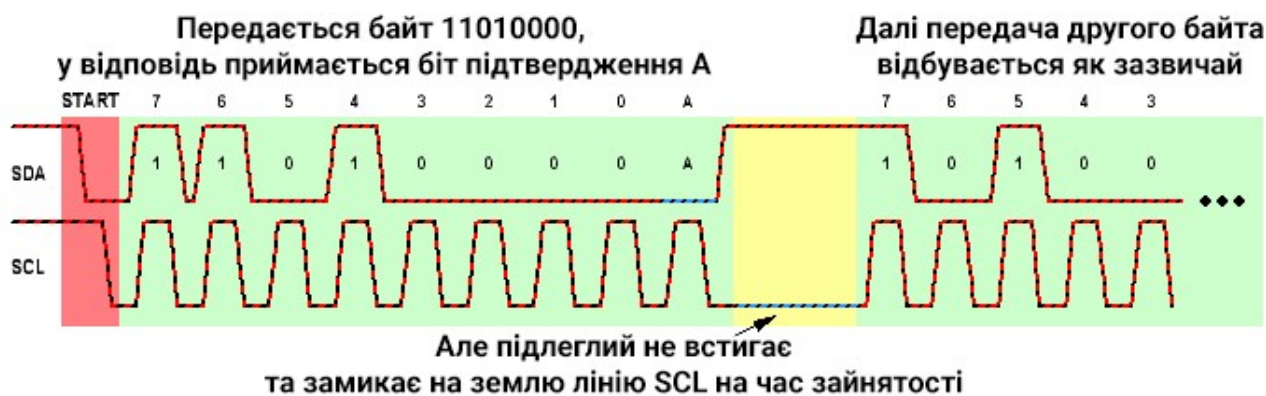


Рис. 5.5. Підлеглий робить паузу в передачі даних

Дані за інтерфейсом I2C відправляються пакетами, кожен пакет складається з дев'яти біт (рис. 5.6), з яких 8 біт даних та 1 біт підтвердження / непідтвердження прийому. Першим пакетом ведучий відправляє підлеглому фізичну адресу пристрою і біт напрямку.

Сама адреса складається з семи біт, а восьмий біт визначає, що буде робити підлеглий на наступному пакеті – приймати або передавати дані. Дев'ятим бітом є біт підтвердження ACK. Якщо підлеглий отримав свою власну адресу і зчитав її повністю, то на дев'ятому такті він замкне лінію SDA в «0», згенерувавши ACK. Ведучий це «помітить» та зрозуміє, що все йде за планом і можна продовжувати. Якщо підлеглого не виявлено або він пропустив адресу, неправильно прийняв байт чи вийшов з ладу, то лінія SDA на дев'ятому такті не буде заземлена та біт ACK не сформується. Замість нього буде інверсний біт – NACK. Після цього ведучий припинить передачу.

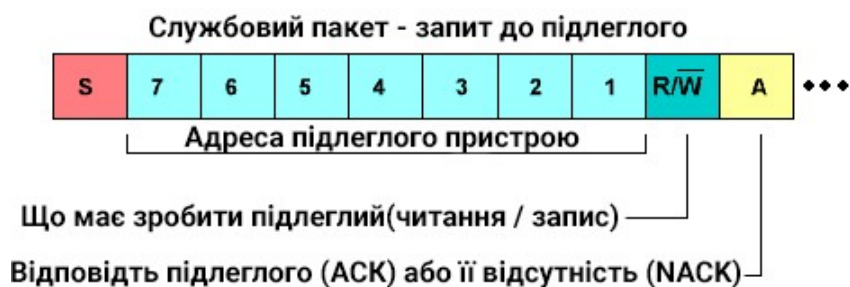


Рис. 5.6. Схема бітів пакету даних

Після пакета адреси відправляються пакети з даними в ту або іншу сторону. Напрямок (читання чи запис) визначається в залежності від стану біта RW в адресному пакеті:

- $RW = 0$ – дані треба записати ($W=0$);
- $RW = 1$ – дані треба зчитати ($R=1$).

Приклад запису двох байтів даних до деякого пристрою за протоколом I2C показано на рис. 5.7.



Рис. 5.7. Пакети запису двох байтів даних

Зчитування даних відбувається практично так само, але з певною відмінністю. Після прийому останнього байта необхідно повідомити підлеглому, що підключення до нього більше не потрібне. Для цього треба встановити біт NACK на пакеті. Якщо ж відправити біт ACK, то після зупинки передачі ведучий не припинить заземлення лінії. Порядок прийому двох байтів показано на рис. 4.8 (встановлено режим читання $R=1$):



Рис. 4.8. Пакети зчитування двох байтів даних

Можлива ще одна ситуація, яка називається **повторний старт**. В такому випадку не оголошується STOP, а одразу подається команда на ще один START. Після цього є можливість звернутися до іншого пристрою, не звільняючи шину. Але частіше йде звернення до того ж самого пристрою, що пов'язано з особливостями організації пам'яті.

Як було видно з протоколу, в першому байті відбувалася адресація цілого пристрою. Але в самому пристрої цілком може бути дуже складна структура, з великою кількістю комірок та регістрів. В такому випадку, для запису даних у першому пакеті передається адреса самого пристрою, а в другому в якості даних – адреса комірки пам'яті, до якої потрібно записати інформацію. Під час продовження запису даних, адреса комірки буде автоматично збільшуватись на 1 з кожним новим пакетом.

У випадку зчитування процедура подібна – спочатку передається адреса пристрою, потім адреса комірки пам'яті, а потім виконується повторний старт. Після цього знову відбувається звернення за адресою всього пристрою, але вже з бітом $R=1$, на читання. Наступним етапом пристрій почне видавати дані, які були записані в комірку, що була вказана в пакеті до повторного старту. Такий ланцюжок запис-повторний старт-зчитування зустрічається під час роботи з переважною більшістю модулів та мікросхем.

Як вже було зазначено раніше, інтерфейс I2C часто застосовується для підключення дисплеїв. Раніше в даних лабораторних роботах використовувався матричний РК-дисплей з роздільною здатністю 16×2 точок, який є частиною більшості проектів для відображення певної інформації для користувача. Але ці LCD мають багато обмежень стосовно функціональності та зручності у використанні. Набагато сучаснішими є дисплеї OLED.

В **OLED (Organic Light-Emitting Diode)** дисплеях використовується технологія, в якій світлодіоди самі випромінюють світло без додаткового підсвічування (на відміну від LCD). Дисплей OLED складається з тонкої багат шарової органічної плівки, розміщеної між анодом і катодом. OLED має високий потенціал застосування і розглядається в якості провідної технології для

наступного покоління плоских та гнучких дисплеїв. За невеликих розмірів і незначного енергоспоживання OLED дисплей забезпечує насичену контрастність. Якість відображення інформації, низька вартість та чудові кути огляду OLED дисплея роблять його лідером сучасного ринку.

Такі дисплеї в основному доступні на чіпі SSD1306, який працює за інтерфейсі I2C, що дозволяє його швидко підключити і почати використовувати. OLED модуль з розширенням 128×64 (0.96 дюйма, рис. 5.9) складається з двох частин. Перша частина – сам дисплей, який в свою чергу можна розділити на дві складові: графічний дисплей і контролер SSD1306, від якого йде гнучкий шлейф на зворотну сторону плати. Друга частина модуля являє собою друковану плату (яка по суті є перехідником), на якій реалізована електрична обв'язка та однорядний роз'єм.



Рис. 5.9. Модуль OLED 128×64 на контролері SSD1306

OLED модулі, випускаються з різною роздільною здатністю (128×64 , 128×32 і 96×16), сам контролер SSD1306 може працювати з OLED матрицями з максимальною роздільною здатністю 128×64 . Модулі бувають білі, сині і синьо-жовті (зверху жовта смуга шириною 16 пікселів). Кожен виробник випускає свою друковану плату з різною компоновкою електронних компонентів і виведеним інтерфейсом. Призначення зовнішніх виводів зазвичай наступне:

- VCC – +5 В / +3 В (живлення)
- GND – GND (земля)
- SDA – лінія SDA інтерфейсу I2C
- SCL – лінія SCL інтерфейсу I2C

Перед тим, як виводити щось на екран, завжди необхідно визначити, як в контролері дисплею організована пам'ять. Вся графічна пам'ять (GDDRAM) являє собою область $128 \times 64 = 8192$ біт = 1 Кбайт. Область розбита на 8 сторінок (рис. 5.10), які представлені у вигляді масиву зі 128-ми 8-бітних сегментів (рис. 5.11). Якщо біт = 1 – піксель світиться, якщо 0 – ні. Адресація пам'яті відбувається за номером сторінки і номером сегмента відповідно.

За такого методу адресації існує неприємна особливість – неможливо записати в пам'ять 1 біт інформації, так як запис відбувається за сегментом (по 8 біт). А оскільки для коректного відображення одиничного пікселя на екрані необхідно знати стан інших пікселів в сегменті, доцільно створити в пам'яті мікроконтролера буфер розміром 1 Кбайт і циклічно завантажувати його в пам'ять дисплея, відповідно, виконуючи його повне оновлення. У разі використанні такого методу можливо перерахувати положення кожного біта в пам'яті на класичні координати x, y .

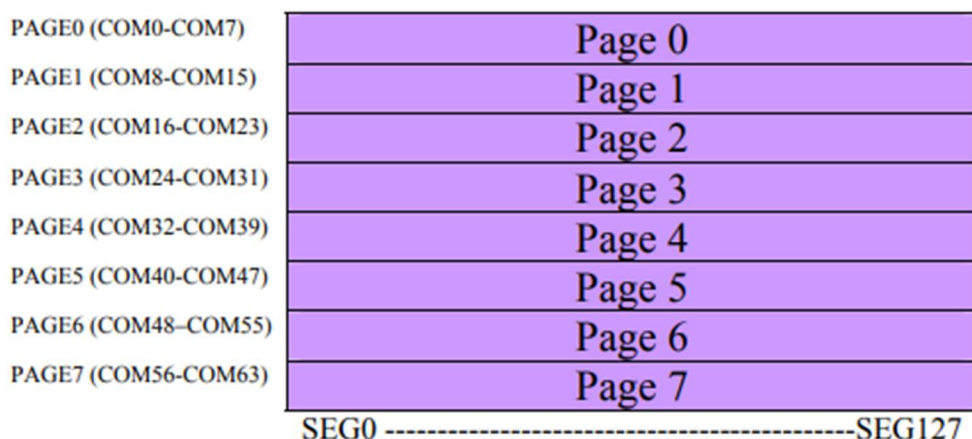


Рис. 5.10. Схема сторінок пам'яті OLED 128×64



Рис. 4.11. Сегменти однієї сторінки пам'яті OLED 128×64

Іншим розповсюдженим прикладом використання інтерфейсу I2C є підключення різноманітних модулів. Як приклад, компанія Maxim Integrated розробила інтегрований сенсорний модуль **MAX30102**, який дозволяє з мінімальними витратами реалізувати портативний і точний **вимірювач пульсу та вмісту кисню в крові**.

Датчики для вимірювання частоти серцевої діяльності і насичення артеріальної крові киснем вже давно знаходять широке застосування в медичних приладах різного призначення. Колись вони використовувалися виключно в складному стаціонарному обладнанні, а з появою нових спеціалізованих інтегральних схем з'явилася можливість створювати зручні портативні прилади – **пульсоксиметри**. Вони дозволяють відстежувати ступінь насичення артеріальної крові киснем (SpO_2) та частоту серцевих скорочень (пульс).

Більш насичена киснем кров має більш яскравий відтінок червоного кольору. Зі зміною насиченості крові киснем (сатурації) змінюється ступінь поглинання і відбиття променів червоного та інфрачервоного світла, спрямованих на капіляри. Проходячи через кров і тканини, світловий сигнал набуває пульсуючий характер під впливом зміни об'єму кровоносних судин.

В основу методу пульсоксиметрії (рис. 5.12) лежить вимірювання ступеня поглинання гемоглобіном крові променів червоного та інфрачервоного світла. Гемоглобін служить свого роду фільтром, причому «колір» фільтра залежить від кількості кисню, пов'язаного з гемоглобіном або, іншими словами, від процентного вмісту кисню в крові. А «товщину» фільтра визначає пульсація артерій, яка виникає під час зміни в них кількості крові.

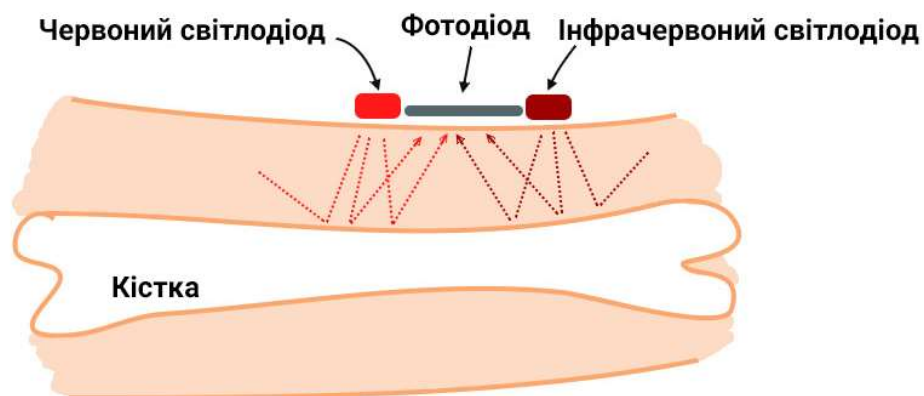


Рис. 5.12. Методика вимірювання пульсу та сатурації

Використовуючи датчики червоного і інфрачервоного світла спільно з фотодетекторами, АЦП і системами обробки даних, можна контролювати вміст кисню в крові. Пульсоксиметр має датчик, в якому знаходяться два джерела світла - 660 нм (червоний) і 940 нм (інфрачервоний, ІЧ). Фотодетектор реєструє рівень світла після поглинання частини потоку тканинами і компонентами крові, а мікропроцесор аналізує отримані результати і визначає насиченість крові киснем і частоту серцевих скорочень. Приклад графіків отриманих сигналів від червоного та інфрачервоного променів наведено на рис. 5.13.

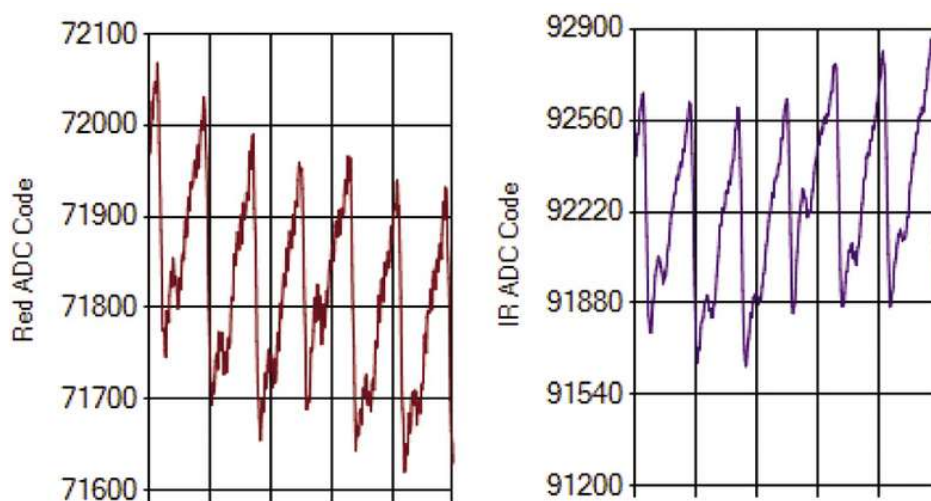


Рис. 5.13. Сигнали пульсоксиметра від червоного та інфрачервоного світла

MAX30102 є інтегральним сенсорним модулем, призначеним для спрощення розробки портативних медичних приладів контролю серцевого ритму і насиченості крові киснем. До складу цієї мікросхеми входять інтегровані світлодіоди (червоний та ІЧ) і фотоприймач, а також вбудовані оптичні елементи. Наявна в складі MAX30102 електронна схема обробки сигналів забезпечує компенсацію зовнішнього засвічення. В процесі вимірювань використовується канал червоного та інфрачервоного світла з програмно регульованою інтенсивністю світіння і тривалістю сеансів вимірювання.

MAX30102 (рис. 5.14) працює від джерела живлення з напругою 1,8 В. Окреме джерело живлення +3,3...+5,0 В потрібне для роботи світлодіодів. Модуль MAX30102 може бути програмно переведений в режим очікування з практично нульовим струмом споживання.



Рис. 5.14. Датчик MAX30102

У корпусі MAX30102 реалізована повнофункціональна схема сенсорного модуля для створення портативних систем пульсоксиметрії з високими вимогами до точності вимірювань. Пристрій має мініатюрні розміри, домогтися яких вдалося без шкоди для оптичних або електричних характеристик.

На рис. 5.15 зображено структурну схему MAX30102. Система живлення включає окремі джерела для основної схеми і для світлодіодів. Управління роботою MAX30102 здійснюється через внутрішні програмні регістри. Цифрові вихідні дані можуть бути збережені в 32-бітному буфері FIFO, який дозволяє через загальну шину послідовно передавати цифровий потік на зовнішній мікроконтролер.

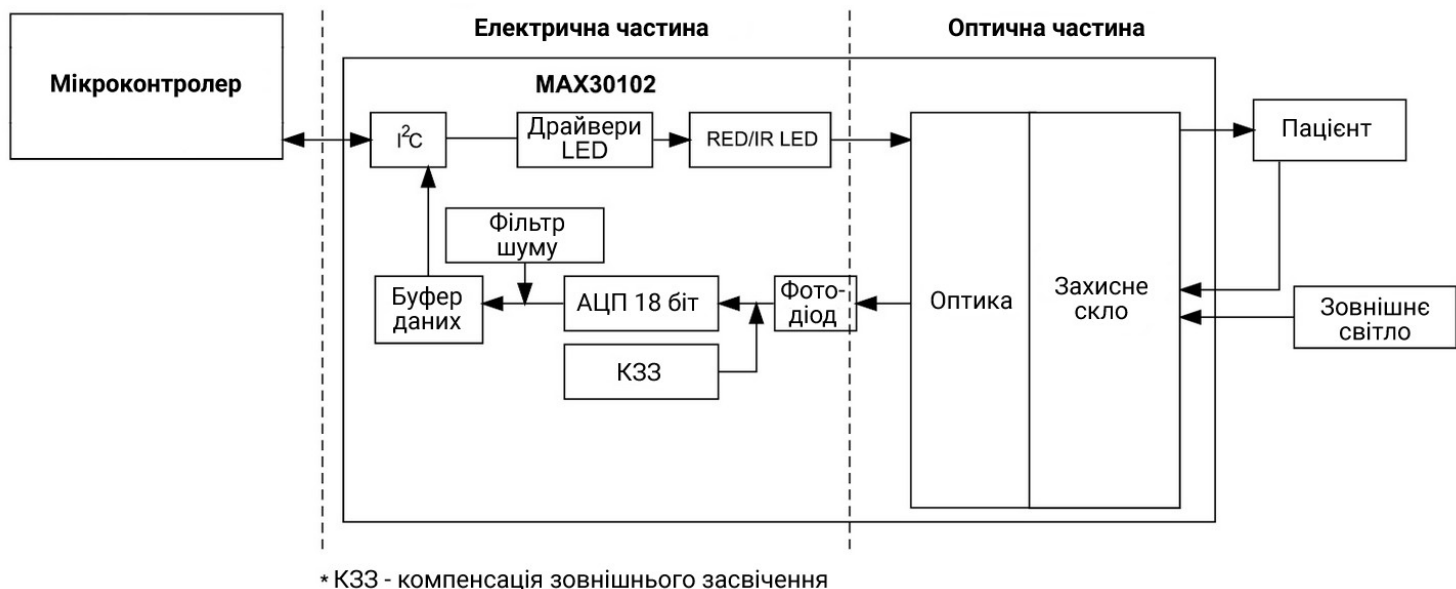


Рис. 5.15. Структурна схема модуля MAX30102

Процентний вміст кисню в крові визначається неінвазійним методом через шкіру (про що свідчить позначення «Sp»), як процентне відношення насиченого киснем гемоглобіну (HbO_2) до загального вмісту гемоглобіну ($\text{HbO}_2 + \text{RHb}$), які визначаються за допомогою фотодетектора, ІЧ та червоного світлодіода.

Підсистема вимірювання SpO_2 включає схему компенсації зовнішнього засвічення (КЗЗ), сигма-дельта-АЦП та цифровий фільтр. КЗЗ має внутрішню схему блокування сигналу для усунення зовнішнього засвічення і розширення ефективного динамічного діапазону. Внутрішній АЦП виконує безперервну дискретизацію, використовуючи сигма-дельта 18-бітний конвертор. Швидкість виведення даних АЦП програмується в діапазоні 50 ... 3200 вибірок в секунду.

Вихідні дані MAX30102 порівняно нечутливі до довжини хвилі ІЧ-світлодіода, тоді як довжина хвилі червоного світлодіода має вирішальне значення для правильної інтерпретації результатів вимірювань. Тому модуль має вбудовану систему компенсації температури оточуючого середовища. Також MAX30102 використовує функцію контролю присутності (близькості) пацієнта з метою скорочення випромінювання світла і енергозбереження, коли біля датчика немає пальця пацієнта.

Червоний та ІЧ-світлодіоди управляються за допомогою внутрішніх драйверів, які модулюють тривалість імпульсів і величину струму. Це дозволяє оптимізувати режим роботи в залежності від конкретної ситуації. Внутрішній буфер зберігає до 32 вимірювань, тому зовнішньому процесору немає необхідності зчитувати показання після кожного вимірювання. Використовувані для корекції показань дані про температуру знімаються один раз в секунду.

Послідовність подій в процесі вимірювання SpO_2 і обміну даними:

1. Активізація режиму SpO_2 , ініціалізація вимірювання температури;
2. Завершення вимірювання температури, генерація переривання;
3. Зчитування даних температури, очищення прапорця переривання;
4. Генерація переривання при критичному наповненні буфера FIFO;
5. Зчитування даних FIFO, очищення прапорця переривання;
6. Збереження даних наступного вимірювання.

Хід виконання роботи

1. Виконати попередні налаштування плати «STM32 Blue pill» для підключення OLED дисплея.
 - 1.1. Запустити середовище програмування STM32CubeIDE та створити новий проект.
 - 1.2. Встановити джерело тактування від зовнішнього кварцового резонатора та режим відлагодження «*Serial Wire*».
 - 1.3. На вкладці *Clock Configuration* налаштувати тактування мікроконтролера. Встановити тактування від зовнішнього кварцового резонатора та основну частоту тактування 72 МГц.
 - 1.4. На вкладці *Pinout & Configuration* у розділі *Connectivity* активувати інтерфейс I2C1 та налаштувати його для роботи у високошвидкісному режимі (рис. 5.16). При цьому на схемі мікроконтролера автоматично активуються зарезервовані для I2C1 виводи PB6 та PB7.

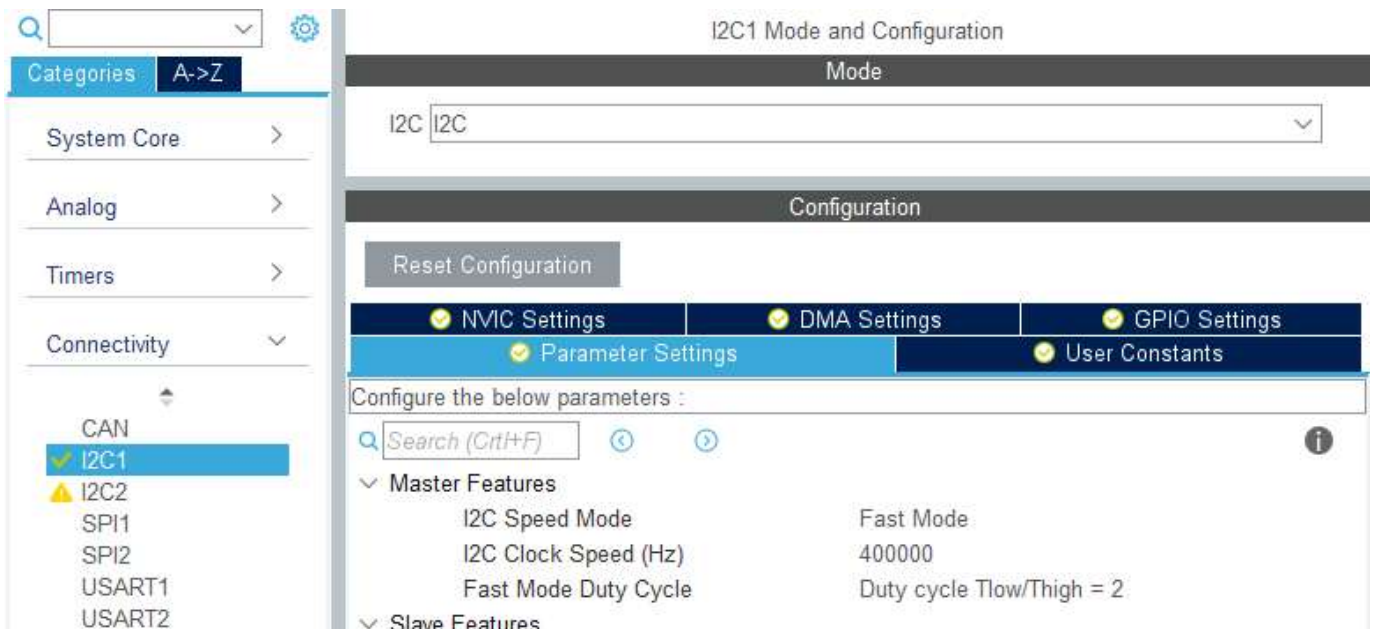


Рис. 5.16. Налаштування інтерфейсу I2C1

- 1.5. Зберегти проект та провести автоматичну генерацію коду.

1.6. В дереві структури проекту додати файли необхідних бібліотек для керування OLED дисплеєм. У папці *Inc* розмістити заголовковий файл бібліотеки *ssd1306.h* та файл бібліотеки шрифтів *fonts.h*. У папці *Src* підключити відповідні вихідні файли *ssd1306.c* та *fonts.c*.

1.7. У файлі *main.c* підключити відповідні заголовкові файли:

```
/* USER CODE BEGIN Includes */
#include "fonts.h"
#include "ssd1306.h"
#include "stdio.h"
/* USER CODE END Includes */
```

1.8. Провести ініціалізацію дисплею за допомогою функції *SSD1306_Init()*, а також вивести на дисплей напис «Kafedra ASNK». Для встановлення курсору на певну позицію використовується функція *SSD1306_GotoXY()*, аргументами якої є координати необхідної позиції. Для виведення рядку символьних даних використовується функція *SSD1306_Puts()*, аргументами якої є власне рядок даних, необхідний шрифт (7x10, 11x18 або 16x26 пікселів) та колір (1 – піксель активний, 0 – піксель чорний). Функція *SSD1306_UpdateScreen()* оновлює вміст на екрані.

```
/* USER CODE BEGIN 2 */
// Ініціалізація дисплею
SSD1306_Init();

// Встановити курсор на позицію 0,0
SSD1306_GotoXY (0,0);
// Вивести напис Kafedra (шрифт 11x18 пікселів)
SSD1306_Puts ("Kafedra", &Font_11x18, 1);

SSD1306_GotoXY (0,20);
SSD1306_Puts("ASNK", &Font_11x18, 1);
SSD1306_UpdateScreen();

/* USER CODE END 2 */
```

1.9. Підключити дисплей до мікроконтролера за схемою, показаною на рис. 5.17.

1.10. Завантажити програму до мікроконтролера та перевірити правильність її роботи. Критерієм правильності буде виведення жовтого напису «Kafedra» та у наступному рядку синього напису «ASNK».

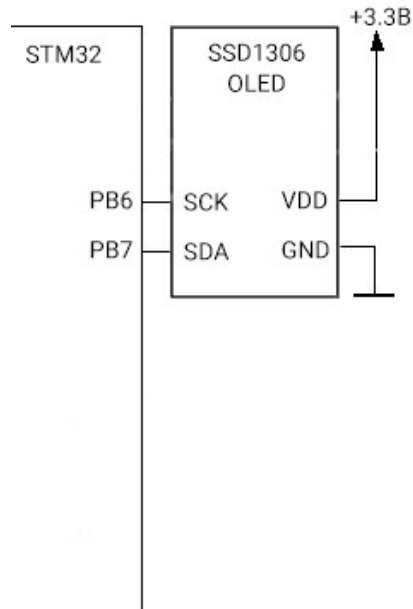


Рис. 5.17. Підключення OLED SSD1306 до мікроконтролера за I2C інтерфейсом

1.11. Перетворити створений напис на біжучий рядок. Для цього використати функцію *SSD1306_ScrollLeft()*, першим аргументом якої є індекс початкової сторінки дисплею для переміщення, другим – кінцевої сторінки. Для переміщення одразу всього вмісту екрану можна вказати індекси (0,7). Після виклику даної функції необхідно організувати затримку і у потрібний момент зупинити рух вмісту екрану за допомогою функції *SSD1306_Stopscroll()*.

```

/* USER CODE BEGIN 2 */
// Ініціалізація дисплею
SSD1306_Init();

// Встановити курсор на позицію 0,0
SSD1306_GotoXY (0,0);
// Вивести напис Kafedra (шрифт 11x18 пікселів)
SSD1306_Puts("Kafedra", &Font_11x18, 1);

SSD1306_GotoXY (0,20);
SSD1306_Puts("ASNK", &Font_11x18, 1);
SSD1306_UpdateScreen();

SSD1306_ScrollLeft(0,7);
HAL_Delay(6000);
SSD1306_Stopscroll();

/* USER CODE END 2 */

```

1.12. Модифікувати програму таким чином, щоб після завершення переміщення біжучого рядку на дисплей виводився заповнений трикутник. Для цього необхідно спочатку очистити дисплей за допомогою функції *SSD1306_Clear()*, а потім використати функцію *SSD1306_DrawFilledTriangle()*, перші шість аргументів якої є координатами вершин трикутника, а сьомий аргумент – колір. Після цього потрібно оновити вміст дисплею.

```
/* USER CODE BEGIN 2 */
SSD1306_Init();

// Встановити курсор на позицію 0,0
SSD1306_GotoXY (0,0);
// Вивести напис Kafedra (шрифт 11x18 пікселів)
SSD1306_Puts("Kafedra", &Font_11x18, 1);

SSD1306_GotoXY (0,20);
SSD1306_Puts("ASNK", &Font_11x18, 1);
SSD1306_UpdateScreen();

SSD1306_ScrollLeft(0,7);
HAL_Delay(6000);
SSD1306_Stopscroll();

HAL_Delay(500);
SSD1306_Clear();
SSD1306_DrawFilledTriangle(60, 16, 100, 60, 20, 60, 1);
SSD1306_UpdateScreen();

/* USER CODE END 2 */
```

2. Налаштувати мікроконтролер для підключення датчика пульсоксиметра MAX30102 та підготувати програму до роботи.

2.1. Перейти до налаштувань мікроконтролера. На вкладці *Pinout & Configuration* у розділі *Connectivity* активувати інтерфейс I2C2 та налаштувати його для роботи у високошвидкісному режимі. При цьому на схемі мікроконтролера автоматично активуються зарезервовані для I2C2 виводи PB10 та PB11.

2.2. Самостійно активувати вивід PB12 для роботи в якості входу з підтягуваннями до високого рівня та вивід користувацького світлодіода PC13 для роботи в якості активного виходу.

2.3. Зберегти проект та провести генерацію коду.

2.4. Підключити датчик MAX30102 до мікроконтролера за схемою, показаною на рис. 5.18.

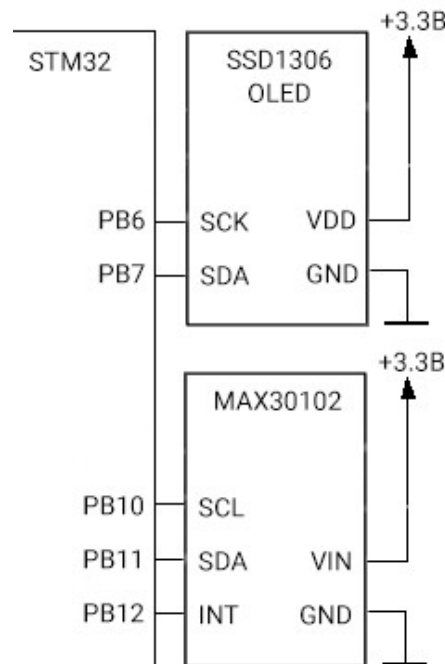


Рис. 5.18. Підключення OLED SSD1306 та датчика MAX30102 до мікроконтролера за I2C інтерфейсами

2.5. В дереві структури проекту додати файли необхідних бібліотек для керування датчиком MAX30102. У папці *Inc* розмістити заголовковий файл бібліотеки *max30102.h*. У папці *Src* підключити відповідний вихідний файл *max30102.c*.

2.6. У файлі *main.c* додати підключення відповідного заголовкового файлу:

```
/* USER CODE BEGIN Includes */
#include "fonts.h"
#include "ssd1306.h"
#include "stdio.h"
#include "max30102.h"
/* USER CODE END Includes */
```

2.7. Створити необхідні змінні для зберігання значень SpO2, частоти серцевих скорочень (Heart Rate) та службової інформації *diff* для подальшої візуалізації серцевого ритму. Також передбачити змінні для зберігання значень SpO2 та частоти серцевих скорочень у форматі рядкових даних.

```

/* USER CODE BEGIN PV */
extern uint8_t spo2;
extern uint8_t heartRate;
int16_t diff;

char SP02[8];
char HR[8];
/* USER CODE END PV */

```

2.8. У блоці ініціалізації додати код для ініціалізації датчика пульсоксиметра за допомогою функції *max30102_init()*, а також модифікувати код виведення інформації на дисплей для створення шаблону інтерфейсу пристрою:

```

/* USER CODE BEGIN 2 */
// Ініціалізація дисплею
SSD1306_Init();
// Ініціалізація MAX30102
max30102_init();

SSD1306_GotoXY (0,20);
SSD1306_Puts("HRate:", &Font_11x18, 1);
SSD1306_GotoXY (0,45);
SSD1306_Puts("SpO2:", &Font_11x18, 1);
SSD1306_UpdateScreen();

/* USER CODE END 2 */

```

2.9. Завантажити програму до мікроконтролера та перевірити правильність її роботи. Критерієм правильності будуть написи «HRate:» та «SpO2:» на дисплеї та увімкнений червоний світлодіод на платі датчика MAX30102, а також увімкнений користувацький світлодіод.

3. Написати програму для зчитування інформації з датчика MAX30102 та виведення її на дисплей.

3.1. Інформація з датчику повинна зчитуватися тоді, коли на його виводі переривань INT буде виставлено низький рівень – сигнал про те, що датчик готовий передати дані. При цьому з діагностичною метою рекомендується змінити стан користувацького світлодіода на протилежний.

- 3.2. Наступним кроком відбувається калібрування датчика із використанням функції *max30102_cal()*.
- 3.3. Після калібрування, за допомогою функцій *max30102_getSpO2()*, *max30102_getHeartRate()* та *max30102_getDiff()* з регістрів датчика за інтерфейсом I2C зчитуються відповідні параметри.
- 3.4. Далі дані виводяться на дисплей. Фінальний код програми розміщується у нескінченному циклі *while(1)*:

```
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */

    if (HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_12) == GPIO_PIN_RESET)
    {
        HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);
        max30102_cal();

        spo2 = max30102_getSpO2();
        sprintf(SPO2, "%3d", spo2);
        heartRate = max30102_getHeartRate();
        sprintf(HR, "%3d", heartRate);

        diff = max30102_getDiff();
    }

    SSD1306_GotoXY (66,20);
    SSD1306_Puts (HR, &Font_11x18, 1);
    SSD1306_GotoXY (99,20);
    SSD1306_Puts ("bpm", &Font_7x10, 1);

    SSD1306_GotoXY (66,45);
    SSD1306_Puts (SPO2, &Font_11x18, 1);
    SSD1306_GotoXY (99,45);
    SSD1306_Puts ("%", &Font_11x18, 1);
    SSD1306_UpdateScreen();

}
/* USER CODE END 3 */
```

- 3.5. Завантажити програму до мікроконтролера та перевірити результат. У разі піднесення пальця до захисного скла датчика (в районі червоного світлодіоду) на екрані з'являться визначені біометричні показники.

4. Модифікувати програму для виведення на верхню частину екрану графіку динаміки серцевих скорочень.

4.1. В налаштуваннях мікроконтролеру активувати таймер TIM2 в режимі генерації циклічних переривань з періодом 0,03 с (максимальний період оновлення дисплею). Для цього встановити значення регістру подільника частоти таймера *Prescaler*: «7199» та значення регістру перезавантаження *Counter Period*: «299». Дозволити переривання від таймеру в NVIC.

4.2. Зберегти проект та провести генерацію коду.

4.3. У файлі *main.c* у блоці ініціалізації додати функцію старту таймера:

```
/* USER CODE BEGIN 2 */
HAL_TIM_Base_Start_IT(&htim2);

// Ініціалізація дисплею
SSD1306_Init();
// Ініціалізація MAX30102
max30102_init();

SSD1306_GotoXY (0,20);
SSD1306_Puts("HRate:", &Font_11x18, 1);
SSD1306_GotoXY (0,45);
SSD1306_Puts("SpO2:", &Font_11x18, 1);
SSD1306_UpdateScreen();

/* USER CODE END 2 */
```

4.4. В нескінченному циклі прибрати або закоментувати функцію оновлення дисплею (тепер оновлення буде відбуватись у перериванні):

```
...

SSD1306_GotoXY (66,45);
SSD1306_Puts (SP02, &Font_11x18, 1);
SSD1306_GotoXY (99,45);
SSD1306_Puts ("%", &Font_11x18, 1);
//SSD1306_UpdateScreen();

}
/* USER CODE END 3 */
```

4.5. Відкрити файл *stm32f1xx_it.c* для опису функції-обробника переривань.

4.6. Підключити заголовковий файл бібліотеки дисплею:

```
/* USER CODE BEGIN Includes */
#include "ssd1306.h"
/* USER CODE END Includes */
```

4.7. Оголосити видимою змінною *diff* та додати змінну *diff_past*, яка буде зберігати попереднє значення *diff*, а також створити змінну лічильника:

```
/* USER CODE BEGIN PV */
extern int16_t diff;
int16_t diff_past;
uint8_t i;
/* USER CODE END PV */
```

4.8. В тілі функції-обробника переривань таймеру TIM2 реалізувати алгоритм побудови графіка. Графік будується у першій рядковій секції дисплею (піксельні індекси $y = 0 \dots 15$). Відповідно, значення *diff*, які будуть відкладатись по осі *y*, не повинні виходити за вказані межі. Побудова графіка відбувається послідовно за двома точками – *diff_past* та *diff*. Точки з'єднуються лініями за допомогою функції *SSD1306_DrawLine()*, в якості аргументів якої передаються координати точок та колір. Таким чином, під час кожного виклику функції-обробника переривання на графік буде додаватися одна точка, яка буде з'єднуватись лінією з попередньою. Коли лінія графіку дійде до правого краю дисплею (піксельний індекс 124, оскільки краще залишити ще 3 пікселі в запасі), старий графік повинен стертися та почати будуватись знову. Для цього використовується функція *SSD1306_DrawFilledRectangle()*, яка малює чорний заповнений прямокутник на полотні графіка. В якості аргументів до функції передаються координати діагональних точок прямокутника та колір (в нашому випадку – 0, чорний). Після виконання всіх операцій відбувається відповідне оновлення змінної лічильника *i* та оновлення дисплею. Програмний код, який реалізує даний алгоритм, має наступний вигляд:

```
/* USER CODE BEGIN TIM2_IRQn 1 */
if(i==0) diff_past = diff;

if(diff > 15) diff = 15;
if(diff < 0) diff = 0;

SSD1306_DrawLine(i, 15-diff_past, i+1, 15-diff, 1);

if(i>124)
{
    SSD1306_DrawFilledRectangle(0, 0, 127, 16, 0);
```

```

        i=0;
    }
else
{
    i++;
    diff_past=diff;
}

SSD1306_UpdateScreen();

/* USER CODE END TIM2_IRQn 1 */

```

4.9. Завантажити програму до мікроконтролера та перевірити результат. Критерієм правильності буде поява графіку серцевого ритму у верхній частині дисплею. Графік буде коректним у разі піднесення пальцю до датчику (інакше буде пряма лінія – датчик неактивний).

Індивідуальні завдання

1. Модифікувати програму таким чином, щоб перед запуском пульсоксиметра на екран виводились логотип та назва вашої бригади. Інформація має постійно переміщуватись («плавати») від лівого до правого краю дисплею. Дані мають затримуватись на декілька секунд на дисплеї, після чого має з'явитись головний інтерфейс з вимірюваннями, розроблений у роботі. Для розробки логотипу використовувати функції малювання фігур, з якими можна ознайомитись у файлі *ssd1306.h*. Логотип має складатися не менш ніж із трьох різних елементів.
2. Реалізувати виведення на дисплей результатів вимірювань лише тоді, коли датчик стабільно «захоплює» серцевий ритм. У випадку, коли інтервал між сусідніми піками на графіку нерівномірний (коливання відстані між декількома піками відбувається більш ніж на 20%), або коли значення пульсу аномально високе (більше 220 уд/хв) або низьке (менше 30 уд/хв), на екрані має виводитись повідомлення «Searching for pulse...». Результати вимірювань мають з'являтися на екрані лише тоді, коли серцевий ритм відповідає зазначеним вимогам.

3. Провести редизайн інтерфейсу програми у відповідності до рисунку нижче (окрім індикатора рівня заряду). Висота стовпчику в правій частині екрану має динамічно змінюватись в залежності від значення пульсу. Чим вище пульс, тим вище стовпчик (більша кількість прямокутних блоків, з яких він складається).



Контрольні запитання

1. Опишіть принцип роботи та області застосування інтерфейсу передачі даних I2C.
2. Наведіть приклад монтажної схеми для підключення пристроїв за інтерфейсом I2C, поясніть переваги та недоліки такої схеми.
3. Розкрийте процес передачі адреси та даних від ведучого до підлеглого пристрою I2C.
4. Опишіть порядок зчитування даних ведучим від підлеглого пристрою за інтерфейсом I2C.
5. Поясніть процес обміну даних за I2C між ведучим і підлеглим пристроями у випадку складної (вкладеної) організації пам'яті підлеглого.
6. Опишіть будову та особливості OLED дисплею SSD1306.
7. Поясніть структуру організації пам'яті дисплеїв на контролері SSD1306.
8. Опишіть методику неінвазійного вимірювання пульсу та сатурації.
9. Опишіть принцип роботи та внутрішню будову датчика MAX30102.
10. Поясніть алгоритм виведення на дисплей вимірних значень сатурації та пульсу, а також принцип побудови графіку серцевого ритму.

РОБОТА ЗІ СТАНДАРТОМ БЕЗДРОВОЇ ПЕРЕДАЧІ ДАНИХ BLUETOOTH ТА ГОДИННИКОМ РЕАЛЬНОГО ЧАСУ

Мета: Ознайомитись з особливостями роботи стандарту бездротової передачі даних Bluetooth. Навчитись створювати програмний код для обміну даних зі смартфоном за допомогою Bluetooth. Ознайомитись та навчитись використовувати годинник реального часу.

Теоретичні відомості

Bluetooth – стандарт бездротової передачі даних, розроблений спільно компаніями Ericsson, IBM, Nokia, Intel та Toshiba. Стандарт Bluetooth знайшов широке застосування для реалізації зв'язку різних пристроїв – смартфонів, ноутбуків, та периферійних пристроїв – навушників, мишок, клавіатур. Протоколи стандарту Bluetooth дозволили різним пристроям зв'язуватися за допомогою процедури сполучення (англ. pairing) та передавати дані між собою.

Принцип дії ґрунтується на використанні радіохвиль. Радіозв'язок Bluetooth здійснюється в ISM-діапазоні (англ. Industry, Science and Medicine), який використовується в різних побутових приладах та бездротових мережах (вільний від ліцензування діапазон 2,4-2,4835 ГГц). У Bluetooth застосовується метод розширення спектру зі стрибкоподібною перебудовою частоти (англ. Frequency Hopping Spread Spectrum, FHSS). Метод FHSS простий у реалізації, забезпечує стійкість до завад, а обладнання недороге.

Основою архітектури Bluetooth є пікомережа (англ. piconet). Вона являє собою централізовану мережу з одного головного вузла та до семи підлеглих вузлів у радіусі 10 метрів (рис. 6.1а). Пікомережі можна пов'язувати між собою за допомогою спеціального вузла – мосту. Такі об'єднані мережі створюють розсіяну мережу (англ. scatternet), схема якої показана на рис. 6.1б.

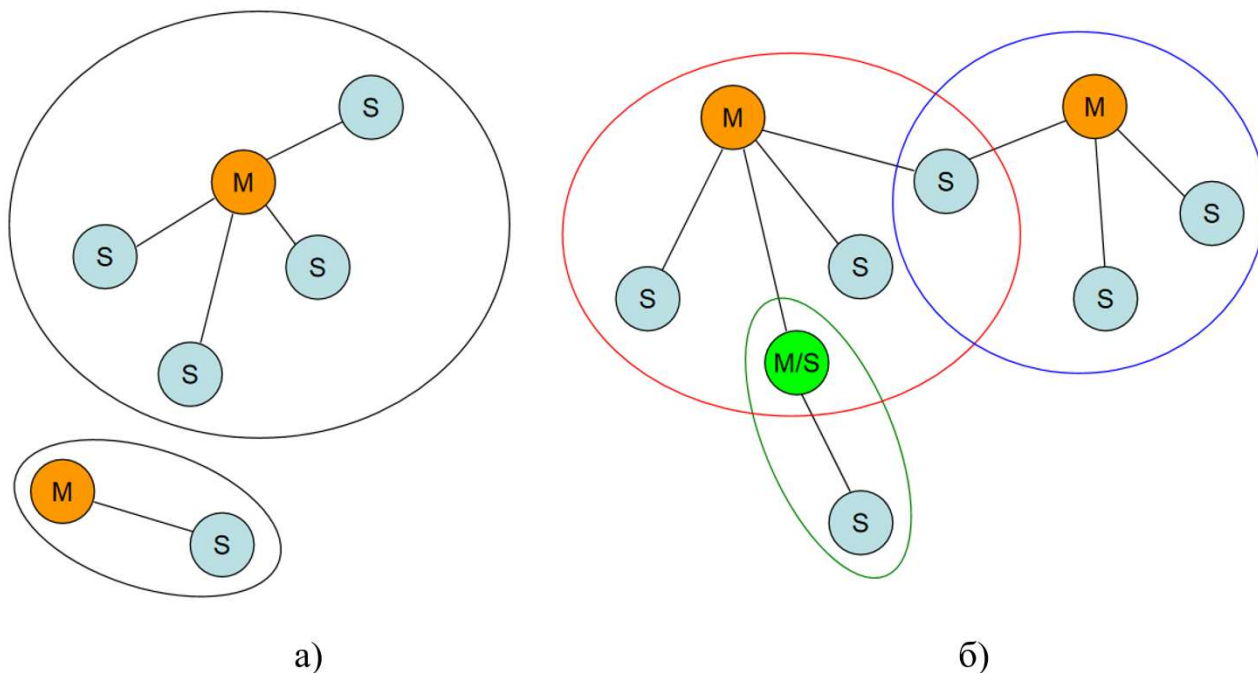
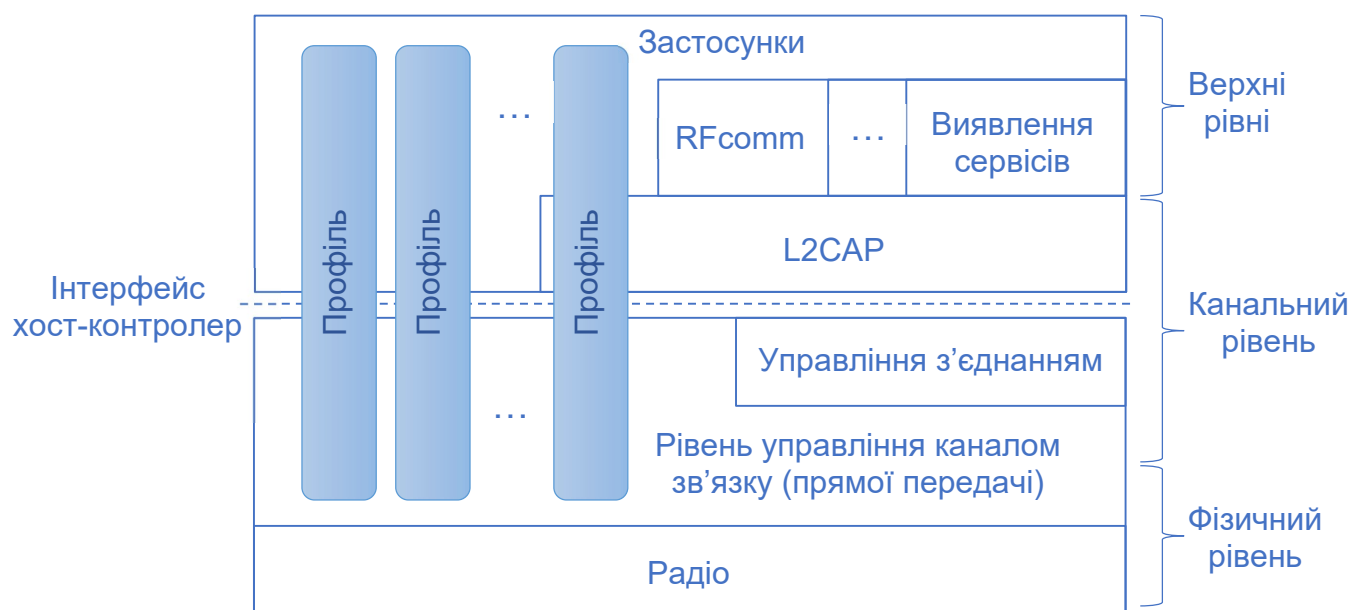


Рис. 6.1. Архітектура мереж Bluetooth: а) – пікомережі, б) – розсіяна мережа

Окрім семи активних підлеглих вузлів, ведучий вузол може підтримувати до 255 вузлів, що відпочивають – це пристрої, які працюють в режимі зниженого енергоспоживання. В архітектурі Bluetooth також реалізовано часове ущільнення – головний вузол контролює та розподіляє часові інтервали між підлеглими вузлами. Безпосередньо підлеглі вузли не пов'язані, зв'язок здійснюється виключно через ведучий вузол. Схема організації набору (стеку) протоколів Bluetooth показана на рис. 6.2.



Нижній, **фізичний рівень** визначає характеристики радіозв'язку та методи модуляції. Параметри цього рівня стека Bluetooth були вибрані таким чином, щоб максимально знизити вартість модулів.

Наступний рівень – **рівень управління каналом зв'язку**. Цей рівень визначає розподіл головним вузлом часових інтервалів та їх групування у кадри. Далі слідують два протоколи, які використовують протокол управління каналом зв'язку: протокол управління з'єднанням та протокол управління логічними каналами та погодження L2CAP. **Протокол управління з'єднанням** встановлює логічні канали між пристроями, керує режимами енергоспоживання, створенням пари, шифруванням та якістю обслуговування. Цей протокол знаходиться нижче за **хост-контролер** – інтерфейс, протоколи нижче якого зазвичай реалізуються на чіпі Bluetooth, а протоколи вище реалізуються на самому пристрої. **Протокол каналного рівня L2CAP** (англ. Logical Link Control And Adaptation Protocol) збирає повідомлення змінної довжини і забезпечує надійність зв'язку.

Для визначення місцезнаходження служб у межах мережі використовується **протокол виявлення сервісів**. **Протокол RFComm** емулює роботу стандартного послідовного (COM) порту.

На найвищому рівні знаходяться **застосунки**. Різноманітні **профілі** (англ. profiles) застосунків представлені вертикальними прямокутниками, тому що кожен із них визначає частину стека протоколу для конкретної мети. Специфічні профілі, наприклад, профілі для пристроїв типу гарнітур, використовують лише ті протоколи, які необхідні для їх роботи.

На момент написання цього посібника існувало 25 таких профілів. На жаль, це призводить до сильного ускладнення системи. Шість профілів призначені для різного використання аудіо та відео. Наприклад, профіль intercom дозволяє двом телефонам з'єднуватися один з одним на кшталт рацій. Інші профілі призначені для передачі стереозвуку та відео. Профіль HID призначений для пристроїв взаємодії з людиною, наприклад, з'єднання з комп'ютером клавіатур та мишок. Інші профілі дозволяють смартфону або комп'ютеру отримувати зображення від камери, підключатись до мережі Інтернет тощо.

Рівень радіозв'язку передає інформацію біт за бітом від головного вузла до підлеглих і назад. Це малопотужна приймальна система з радіусом дії близько 10 метрів. Вона працює у ISM діапазоні 2,4 ГГц. Діапазон поділено на 79 каналів по 1 МГц у кожному. Щоб співіснувати з іншими мережами, що використовують ISM діапазон, застосовується розширений спектр зі стрибкоподібною перебудовою частоти. Можливі до 1600 стрибків частоти за секунду, тривалість одного часового інтервалу (слота чи такту) – 625 мкс. Всі вузли пікомереж змінюють частоти одночасно, відповідно до синхронізації тактів і псевдовипадкової послідовності стрибків, яка генерується головним вузлом.

У найпростішому випадку головний вузол кожної пікомережі видає послідовності часових інтервалів по 625 мкс, причому передача даних з боку головного вузла починається з парних тактів, а з боку підлеглих вузлів – непарних. Кадри можуть бути довжиною 1, 3 чи 5 тактів. У кожному кадрі йде 126 службових біт на код доступу і заголовок. Корисні дані кадру можуть бути захищені для конфіденційності за допомогою ключа, який вибирається, коли ведучий пристрій з'єднується з підлеглим. Перемикання частоти відбувається лише між кадрами, але не під час передачі кадру.

Протокол керування з'єднаннями встановлює логічні канали, які називаються з'єднаннями (англ. links), щоб переносити кадри між головними та підлеглими пристроями. Перш ніж використовувати з'єднання, два пристрої проходять процедуру **спряження** (англ. pairing). Безпечний простий метод спряження (англ. secure simple pairing) дозволяє користувачам підтвердити, що обидва пристрої показують один і той же ключ, або бачити ключ на одному пристрої та ввести його на другому. Звичайно, цей метод не може використовуватися на деяких пристроях з обмеженим введенням/виводом, таких як бездротові гарнітури. Коли пару створено, протокол встановлює з'єднання.

Існує два основних **типи з'єднань**. Перший тип називається SCO (англ. Synchronous Connection Oriented – синхронний із встановленням зв'язку). Він призначений для передачі даних у реальному масштабі часу – це потрібно,

наприклад, під час телефонних розмов. Такий тип каналу отримує фіксований часовий інтервал передачі у кожному з напрямів.

Інший тип з'єднання називається ACL (англ. Asynchronous Connectionless – асинхронний без встановлення зв'язку). Цей тип зв'язку використовується для передачі пакетів даних, які можуть з'явитись у довільний момент часу. Жодних гарантій успішної передачі не надається. Кадри можуть губитися та пересилатися повторно.

Дані, надіслані ACL-з'єднанням, надходять з рівня **L2CAP**. Цей рівень виконує чотири основні функції:

1. Приймає пакети розміром до 64 Кбайт із верхніх рівнів і розбиває їх на кадри для передачі фізичним каналом. На протилежному кінці цей рівень використовується для зворотної дії – об'єднання кадрів у пакети.
2. Займається розподіленням множини джерел пакетів. Після збирання пакета він визначає, куди слід направити пакет (наприклад, протоколу Rfcomm або протоколу виявлення сервісів).
3. Управляє контролем помилок та повторним пересиланням кадрів. Він визначає помилки та пересилає пакети, які не були розпізнані.
4. Забезпечує якість обслуговування, необхідну кільком з'єднанням.

Розглянемо формат кадрів. На початку кадру (рис. 6.3) вказується код доступу, який зазвичай слугує ідентифікатором головного вузла. Це дозволяє двом головним вузлам, які розташовані досить близько, щоб «чути» один одного, розрізняти, кому з них призначаються дані. Потім слідує заголовок з 54 біт, в якому містяться службові поля. Якщо кадр відправляється з базовою швидкістю (1 Мбіт/с), далі розташоване поле даних. Його розмір обмежений 2744 бітами (для передачі за п'ять тактів). Якщо кадр посилається на збільшеній швидкості (2-3 Мбіт/с), частина даних може бути в два або три рази більше, оскільки кожен символ передає 2 або 3 біти замість одного біта. Перед цими даними розміщується захисний інтервал та зразок синхронізації, який використовується, щоб перейти на більш високу швидкість передачі даних. Кадри з більшою швидкістю закінчуються короткою міткою кінця.

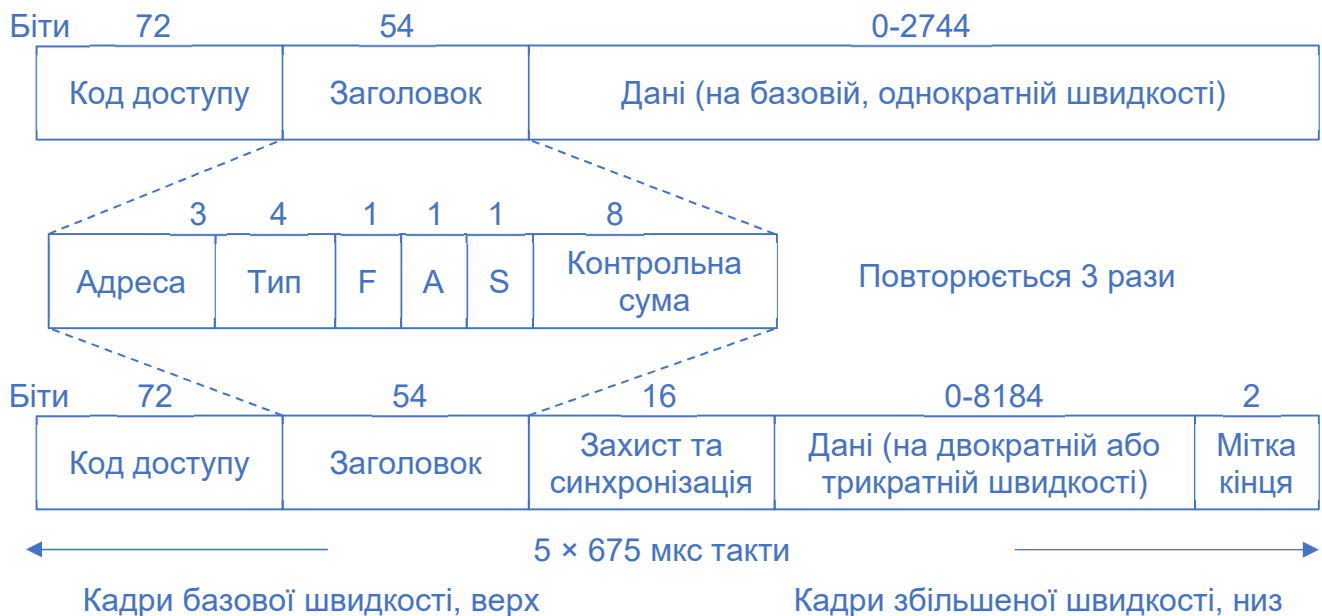


Рис. 6.3. Структура кадрів Bluetooth

Далі розглянемо, із чого складається простий заголовок кадру. Поле **Адреса** ідентифікує один із восьми пристроїв, якому призначена інформація. Поле **Тип** визначає тип кадру, що передається (ACL, SCO, опитування або порожній кадр), метод корекції помилок і кількість часових інтервалів, з яких складається кадр. Біт **F** (англ. Flow – потік) виставляється підлеглим вузлом і повідомляє про те, що його буфер заповнений. Цей біт забезпечує примітивну форму керування потоком. Біт **A** (англ. Acknowledgement – підтвердження) є підтвердженням (ACK), що відсилається разом із кадром. Біт **S** (англ. Sequence – послідовність) використовується для нумерації кадрів, що дозволяє виявляти повторні передачі. Далі слідує 8-бітна **контрольна сума** заголовка. Весь 18-бітний заголовок кадру повторюється тричі, що у підсумку становить 54 біти. На стороні, що приймає, нескладна схема аналізує всі три копії кожного біта. Якщо вони збігаються, біт приймається таким, яким він є. Інакше все вирішує більшість.

У кадрах SCO з базовою швидкістю кадри влаштовані легко: довжина поля даних завжди дорівнює 240 біт. У ACL можливі три варіанти: 80, 160 чи 240 біт. Біти поля даних, що залишилися, використовуються для виправлення помилок.

У найнадійнішої версії (80 біт корисної інформації) один і той же вміст повторюється три рази (що і становить 240 біт), як і в заголовку кадру.

Для зручної роботи мікроконтролерів зі стандартом Bluetooth часто використовуються сторонні модулі. Найбільш популярними є модулі лінійки HC-03 – HC-09. Для практичного використання найзручнішим є модуль HC-05 (рис. 6.4), оскільки він легко налаштовується як для роботи в режимі ведучого, так і в якості підлеглого.

Дані модулі дозволяють передавати дані без дротів, водночас не замислюючись про те, як взагалі влаштований протокол Bluetooth, про його профілі та інші тонкощі. Ці модулі за правильних налаштувань забезпечують передачу даних таким чином, що програміст пише алгоритм так, ніби він працює з дротовим інтерфейсом UART. Тому писати код стає максимально зручно.

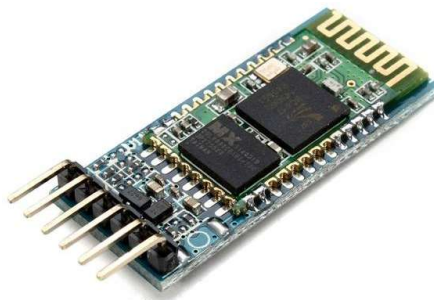


Рис. 6.4. Зовнішній вигляд модуля Bluetooth-UART HC-05

Часто під час вимірювань необхідно фіксувати час або працювати із датою. Блок **годинника реального часу** (англ. real-time clock, RTC) в мікроконтролерах – це незалежний двійково-десятковий таймер/лічильник. В загальному випадку, в мікроконтролерах STM32 блок RTC надає апаратний функціонал годинника/календаря, двох програмованих переривань будильника і прапорець періодичного пробудження з можливістю виклику переривання. RTC також включає блок автоматичного пробудження, щоб керувати режимами зниженого енергоспоживання.

Два 32-бітові регістри містять інформацію про секунду, хвилину, годину (12-годинний або 24-годинний формат), день (день тижня), дату (день місяця), місяць, рік. Ця інформація закодована у форматі binary coded decimal (BCD, двійково-десятковий формат).

Компенсація для чисел 28 лютого, 29 (високосний рік), 30 і 31 місяців виконується автоматично. Також може бути виконаний перехід на літній час і назад. Додаткові 32-бітні регістри будильника містять програмовані значення субсекунди, секунди, хвилини, години, дня та дати. Є функція цифрового калібрування компенсації відхилень частоти кварцового генератора. Поки напруга живлення залишається в допустимому діапазоні, RTC ніколи не зупиняється незалежно від загального стану MCU.

Серед STM32 зустрічаються мікроконтролери з RTC, але без функції календаря. Їх RTC вміє працювати з часом, але не з датами. Як приклад можна навести наш мікроконтролер STM32F103C8T6. По суті, у ньому RTC перетворюється просто на 32-розрядний таймер з розширеними можливостями. Тактування RTC відбувається від зовнішнього кварцового резонатора з частотою 32,768 кГц, який вже розміщений на платі «STM32 Blue pill».

Хід виконання роботи

1. Виконати попередні налаштування плати «STM32 Blue pill» для використання інтерфейсу USART для підключення Bluetooth модуля HC-05.
 - 1.1. Запустити середовище програмування STM32CubeIDE та створити новий проект.
 - 1.2. Встановити джерело тактування від зовнішнього кварцового резонатора та режим відлагодження «*Serial Wire*».
 - 1.3. На вкладці *Clock Configuration* налаштувати тактування мікроконтролера. Встановити тактування від зовнішнього кварцового резонатора та основну частоту тактування 72 МГц.
 - 1.4. В розділі *Connectivity* активувати інтерфейс USART2, обравши *Mode*: «*Asynchronous*». Після цього на графічній моделі мікроконтролера автоматично активуються виводи PA2-PA3, призначені для роботи з USART2.

- 1.5. В параметрах USART2 встановити швидкість передачі даних на рівні 9600 біт/с, обравши *Baud Rate*: «9600». Встановити довжину слова *Word Length*: «8 Bits (including Parity)», вимкнути необхідність використання біту парності *Parity*: «None» та обрати кількість *Stop Bits*: «1».
- 1.6. В налаштуваннях USART2 на вкладці NVIC Settings дозволити переривання від USART2, поставивши прапорець «Enabled».
- 1.7. Зберегти проект та провести генерацію коду.
2. Написати програму для прийому даних зі смартфона та передачі їх назад в луна-режимі, використовуючи стандарт Bluetooth.
- 2.1. У файлі *main.c* підключити стандартні бібліотеки для роботи з рядками:

```
/* USER CODE BEGIN Includes */
#include <string.h>
#include <stdio.h>
/* USER CODE END Includes */
```

- 2.2. Створити змінну, яка буде приймати рядок, що передається зі смартфона.
Також оголосити додаткову структуру для зручної роботи з UART.

```
/* USER CODE BEGIN PV */
char str1[60]={0};

typedef struct UART_prop
{
    uint8_t uart_buf[60];
    uint8_t uart_cnt;
}

UART_prop_ptr;
UART_prop_ptr uartprop;

/* USER CODE END PV */
```

- 2.3. У блоці *USER CODE BEGIN 0* створити функцію, призначену для відправлення рядка за інтерфейсом UART до модуля HC-05 (і, як наслідок, подальшої передачі цього рядка на смартфон по Bluetooth):

```
/* USER CODE BEGIN 0 */
void string_parse(char* buf_str)
{
    HAL_UART_Transmit(&huart2,(uint8_t*)buf_str,strlen(buf_str),0x1000);
}
```

2.4. Нижче у тому самому блоці створити функцію-обробник переривання від UART. Даний обробник приймає символ з порту UART, заносить його в буфер за адресою, що знаходиться в лічильнику та перевіряє на символ перенесення рядка. Якщо такий символ є, то прийом рядка припиняється, набраний у буфері рядок передається до функції відправлення, а лічильник обнулюється. Якщо ж це інший символ, то лічильник нарощується. Також разом з цим у функцію було включено код перевірки на перевищення рядком допустимого розміру буфера (у нашому випадку він становить 60 символів):

```
void UART2_RxCpltCallback(void)
{
    //перевірка на перевищення обсягу буфера
    if (uartprop.uart_cnt>59)
    {
        uartprop.uart_cnt=0;
        HAL_UART_Receive_IT(&huart2,(uint8_t*)str1,1);
        return;
    }

    //заносимо символ до буфера
    uartprop.uart_buf[uartprop.uart_cnt] = str1[0];

    //якщо це символ перенесення рядка
    if(str1[0]==0x0A)
    {
        uartprop.uart_buf[uartprop.uart_cnt+1]=0;
        string_parse((char*)uartprop.uart_buf);
        uartprop.uart_cnt=0;
        HAL_UART_Receive_IT(&huart2,(uint8_t*)str1,1);
        return;
    }

    uartprop.uart_cnt++;
    HAL_UART_Receive_IT(&huart2,(uint8_t*)str1,1);
}

/* USER CODE END 0 */
```

2.5. Додати офіційний обробник переривань від прийому UART, в якому перевіряється, з якого саме порту прийшло переривання та викликається попередньо створений власний обробник:

```

/* USER CODE BEGIN 4 */

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if(huart==&huart2)
    {
        UART2_RxCpltCallback();
    }
}

/* USER CODE END 4 */

```

2.6. Ініціалізувати UART в режимі генерації переривань:

```

/* USER CODE BEGIN 2 */

HAL_UART_Receive_IT(&huart2,(uint8_t*)str1,1);

/* USER CODE END 2 */

```

2.7. Підключити модуль HC-05 до мікроконтролера за схемою, показаною на рис. 6.5.

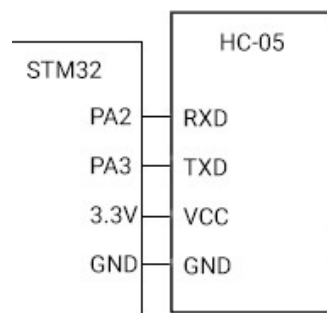


Рис. 6.5. Схема підключення модуля HC-05 до мікроконтролера

2.8. Завантажити програму до мікроконтролера.

2.9. Встановити застосунок «Bluetooth Terminal HC-05» на смартфон.

Створити пару з модулем HC-05, після чого відкрити термінал для обміну даними з мікроконтролером.

2.10. У полі для введення ASCII команд ввести будь-який текст (наприклад, Hello!). Натиснути кнопку *Send ASCII*. Після цього, у випадку правильних налаштувань, у консолі застосунку повинне продублюватись щойно введене повідомлення (рис. 6.6). Це означатиме, що мікроконтролер успішно прийняв рядок по Bluetooth та відправив його назад на смартфон.

2.11. У застосунку «Bluetooth Terminal HC-05» здійснити довге натискання на кнопку Btn 1. Після цього відкриється меню налаштувань кнопки, де можна змінити її назву та задати команду, яка буде надсилатись по Bluetooth у випадку натискання даної кнопки у додатку. Ввести налаштування, показані на рис. 6.7, та зберегти їх.

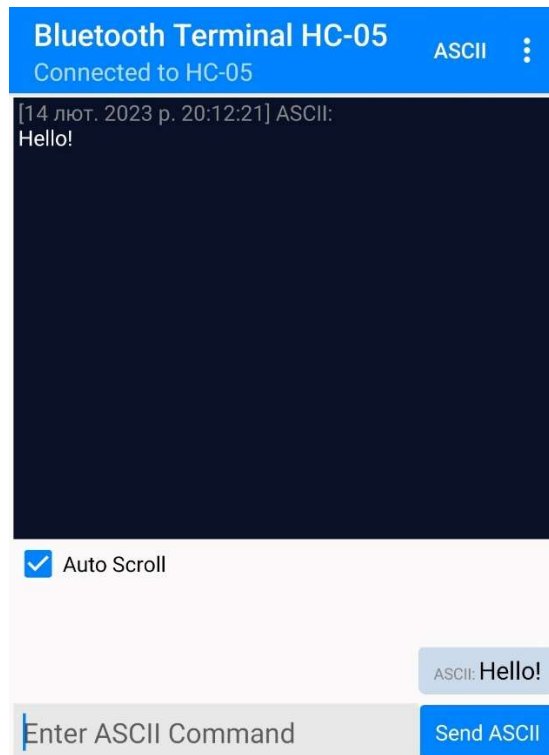


Рис. 6.6. Успішно прийнятий і повернений рядок Hello!

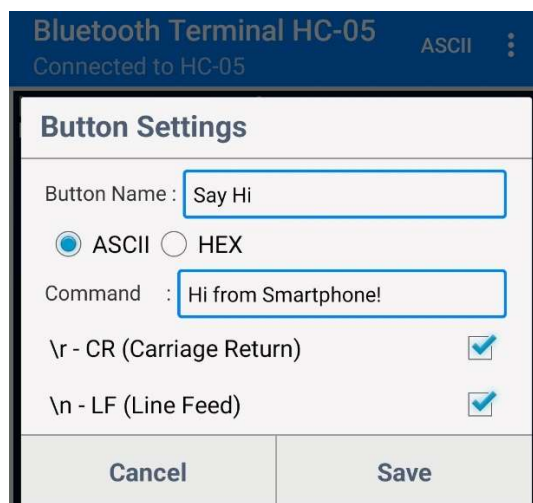


Рис. 6.7. Налаштування кнопки користувача для відправки рядка «Hi from Smartphone!»

- 2.12. У застосунку «Bluetooth Terminal HC-05» натиснути на щойно налаштовану кнопку *Say Hi*. Мікропроцесор має повернути на смартфон повідомлення «Hi from Smartphone!», яке з'явиться у консолі застосунку.
3. Налаштувати мікроконтролер для періодичної (раз на 2 секунди) відправки на смартфон довільних рядків та написати відповідну програму.
- 3.1. В налаштуваннях мікроконтролера увімкнути таймер TIM2 в режимі генерації переривань. Основні налаштування таймера показані на рис. 6.8. Самостійно дозволити переривання від таймера TIM2 в NVIC.

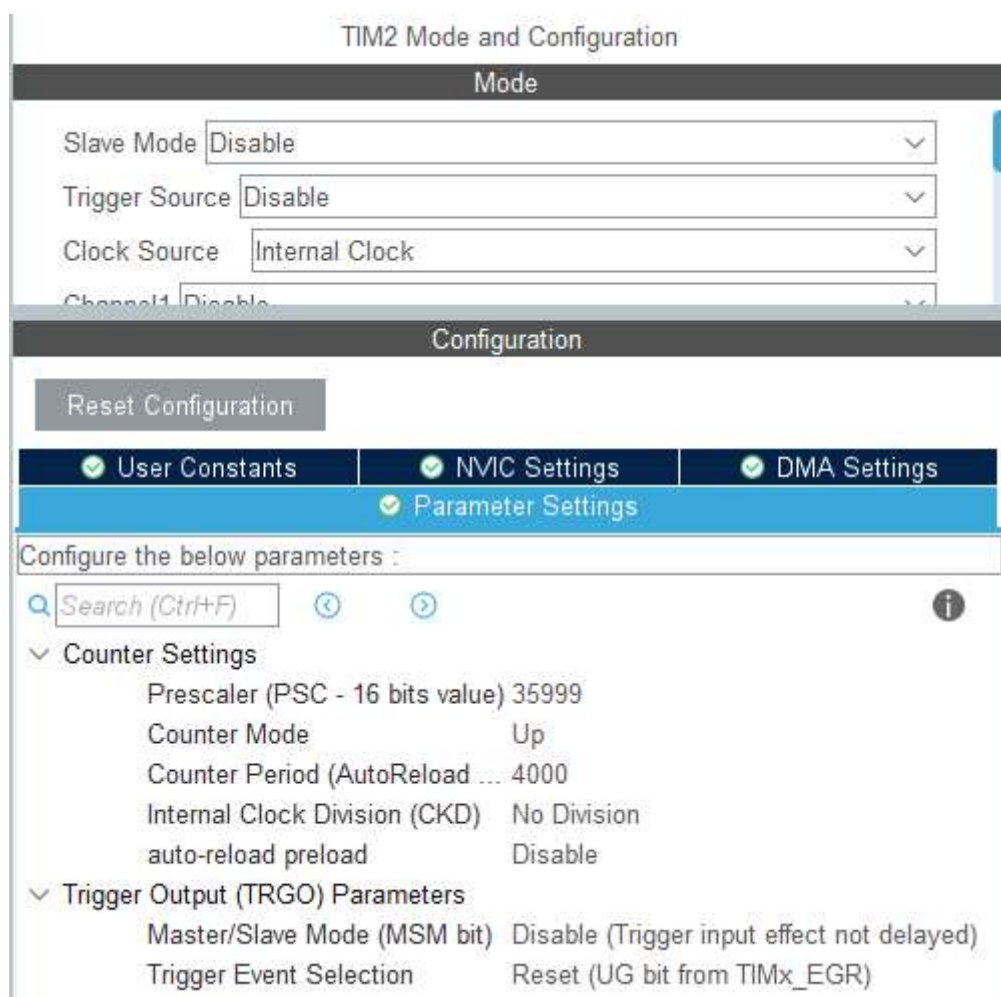


Рис. 6.8. Налаштування таймера для генерації переривань кожні 2 секунди

3.2. Зберегти проект та згенерувати код.

3.3. У файлі *main.c* додати змінну *i* для керування лічильником та створити масив рядків, які будуть передаватись на смартфон:

```
/* USER CODE BEGIN PV */
char str1[60]={0};
```

```
typedef struct UART_prop
{
    uint8_t uart_buf[60];
    uint8_t uart_cnt;
}
```

```
UART_prop_ptr;
UART_prop_ptr uartprop;
```

```
uint8_t i=0;
char *str2[] =
{
    "String1\r\n",
    "String2\r\n",
    "String3\r\n",
    "String4\r\n",
    "String5\r\n"
};
```

```
/* USER CODE END PV */
```

3.4. Додати ініціалізацію таймера:

```
/* USER CODE BEGIN 2 */
```

```
HAL_UART_Receive_IT(&huart2,(uint8_t*)str1,1);
HAL_TIM_Base_Start_IT(&htim2);
```

```
/* USER CODE END 2 */
```

3.5. Написати функцію-обробник переривань від таймера, яка буде по чергові відправляти рядки з масиву *str2* на смартфон:

```
/* USER CODE BEGIN 4 */
```

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if(huart==&huart2)
    {
        UART2_RxCpltCallback();
    }
}
```

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if(htim==&htim2)
    {
        HAL_UART_Transmit(&huart2,(uint8_t*)str2[i],strlen(str2[i]),0x1000);
        i++;
    }
}
```

```

    if(i>4)
    {
        i=0;
    }
}

```

```

/* USER CODE END 4 */

```

3.6. Завантажити програму до мікроконтролера. В результаті, у консолі застосунку «Bluetooth Terminal HC-05» почнуть почергово (з інтервалом у дві секунди) відображатись прийняті з мікроконтролера рядки.

4. Активувати годинник реального часу та модифікувати програму таким чином, щоб разом з текстом рядка на смартфон надсилався і час його відправлення.

4.1. В налаштуваннях мікроконтролера на вкладці *Pinout & Configuration* в розділі *Timers* активувати годинник реального часу RTC та встановити налаштування, показані на рис. 6.9.

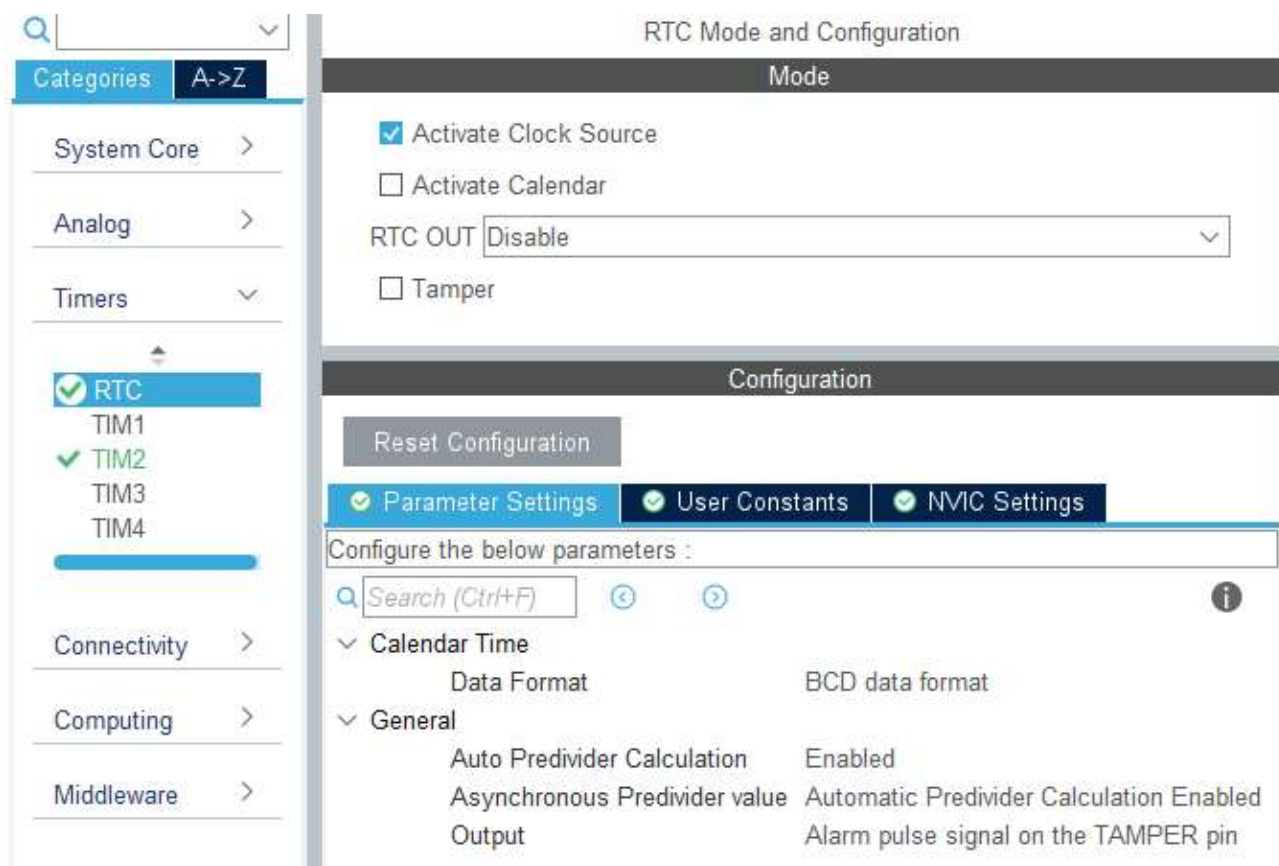


Рис. 6.9. Налаштування годинника реального часу

- 4.2. В розділі *System Core* перейти на вкладку *RCC* та увімкнути тактування від зовнішнього кварцового резонатора, обравши в опції *Low Speed Clock (LSE)*: «Crystal/Ceramic Resonator».
- 4.3. В розділі *Clock Configuration* задати джерело тактування *RTC* від зовнішнього кварцового резонатора та встановити частоту тактування 37,768 кГц (рис. 6.10).

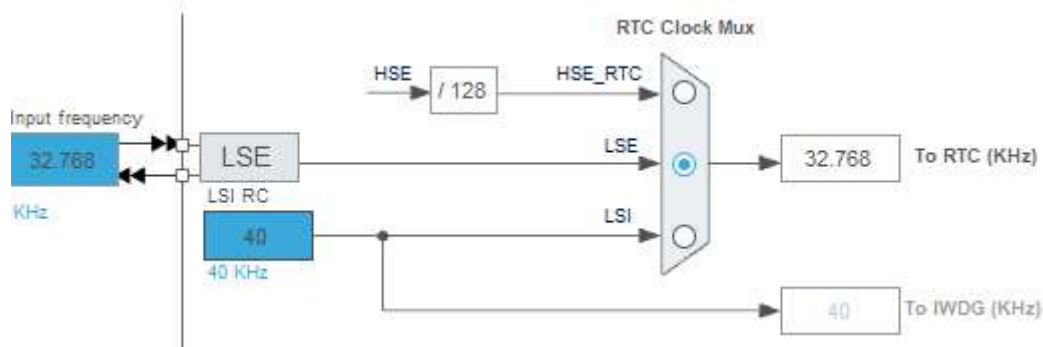


Рис. 6.10. Налаштування тактування годинника реального часу

- 4.4. Зберегти проект та згенерувати код.
- 4.5. У файлі *main.c* додатково створити структуру для зберігання часу та змінну для зберігання поточного значення лічильника *RTC*:

```
/* USER CODE BEGIN PV */
char str1[60]={0};

typedef struct UART_prop
{
    uint8_t uart_buf[60];
    uint8_t uart_cnt;
}
UART_prop_ptr;
UART_prop_ptr uartprop;

uint8_t i=0;
char *str2[] =
{
    "String1\r\n",
    "String2\r\n",
    "String3\r\n",
    "String4\r\n",
    "String5\r\n"
};

RTC_TimeTypeDef sTime;
uint8_t aShowTime[16] = {0};
/* USER CODE END PV */
```

4.6. Створити функцію для встановлення часу:

```
/* USER CODE BEGIN 0 */

void string_parse(char* buf_str)
{
    HAL_UART_Transmit(&huart2,(uint8_t*)buf_str,strlen(buf_str),0x1000);
}

void UART2_RxCpltCallback(void)
{
    //перевірка на перевищення обсягу буфера
    if (uartprop.uart_cnt>59)
    {
        uartprop.uart_cnt=0;
        HAL_UART_Receive_IT(&huart2,(uint8_t*)str1,1);
        return;
    }
    //вносимо символ до буфера
    uartprop.uart_buf[uartprop.uart_cnt] = str1[0];
    //якщо це символ перенесення рядка
    if(str1[0]==0x0A)
    {
        uartprop.uart_buf[uartprop.uart_cnt+1]=0;
        string_parse((char*)uartprop.uart_buf);
        uartprop.uart_cnt=0;
        HAL_UART_Receive_IT(&huart2,(uint8_t*)str1,1);
        return;
    }

    uartprop.uart_cnt++;
    HAL_UART_Receive_IT(&huart2,(uint8_t*)str1,1);
}

/*=====RTC=====*/
static void setTime(uint8_t hour,uint8_t min,uint8_t sec,
                    uint32_t format)
{
    //Ініціалізація RTC та встановлення часу
    sTime.Hours = hour;
    sTime.Minutes = min;
    sTime.Seconds = sec;

    if (HAL_RTC_SetTime(&hrtc, &sTime, format) != HAL_OK)
    {
        Error_Handler();
    }
}

/* USER CODE END 0 */
```

4.7. Додати код для ініціалізації RTC та встановлення початкового значення часу:

```
/* USER CODE BEGIN 2 */

HAL_UART_Receive_IT(&huart2, (uint8_t*)str1, 1);
HAL_TIM_Base_Start_IT(&htim2);

HAL_RTC_WaitForSynchro(&hrtc);
setTime(0x12, 0x20, 0x00, FORMAT_BCD);

/* USER CODE END 2 */
```

4.8. Модифікувати функцію-обробник переривань від таймеру таким чином, щоб до рядку, який відправляється на смартфон, додавався поточний час:

```
/* USER CODE BEGIN 4 */

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if(huart==&huart2)
    {
        UART2_RxCpltCallback();
    }
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    char *toSmartphone[60] = {0};

    if (htim == &htim2)
    {
        HAL_RTC_GetTime(&hrtc, &sTime, FORMAT_BIN);
        sprintf((char*)aShowTime, "%02d:%02d:%02d\t", sTime.Hours,
            sTime.Minutes, sTime.Seconds);
        strcpy((char*)toSmartphone, (char*)aShowTime);
        strcat((char*)toSmartphone, str2[i]);

        HAL_UART_Transmit(&huart2, (uint8_t*) toSmartphone,
            strlen((char*)toSmartphone), 0x1000);
        i++;

        if (i > 4)
        {
            i = 0;
        }
    }
}

/* USER CODE END 4 */
```

4.9. Завантажити код до мікроконтролера. Якщо все виконано вірно, то у консолі застосунку «Bluetooth Terminal HC-05» окрім самих рядків буде вказаний і час їх відправлення (рис. 6.11).

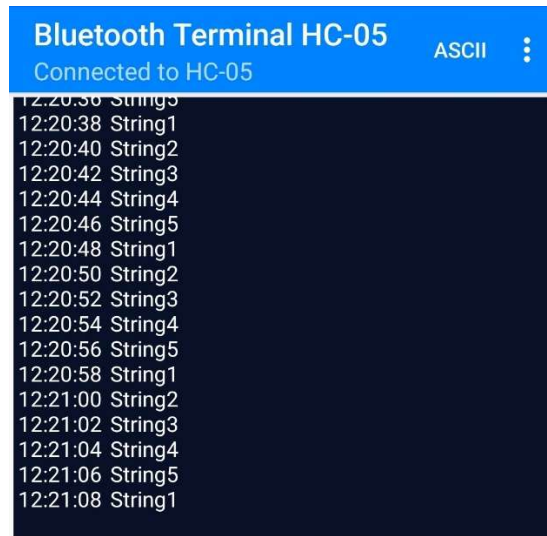


Рис. 6.11. Отримання на смартфон рядків з вказаним часом відправлення

Індивідуальні завдання

1. Підключити до мікроконтролера комп'ютерний кулер. На смартфоні у застосунку «Bluetooth Terminal HC-05» створити кнопки для запуску та зупинки кулера. Реалізувати запуск/зупинку кулера зі смартфона за натисненням відповідних кнопок. У разі запуску кулера мікропроцесор додатково повинен надсилати на смартфон повідомлення «Cooler started at [HH:MM:SS]», а у разі зупинки – «Cooler stopped at [HH:MM:SS]», де [HH:MM:SS] – поточний час.
2. Підключити до мікроконтролера RGB світлодіод. На смартфоні у застосунку «Bluetooth Terminal HC-05» створити кнопки для запалення світлодіода у режимі червоного, зеленого та синього кольорів, а також кнопку для вимкнення світлодіода. Будь яка зміна стану світлодіода має супроводжуватись відправленням на смартфон повідомлення із зазначенням поточного часу та нового стану (наприклад, «[HH:MM:SS] LED turned to red color», [HH:MM:SS] – поточний час).

3. Підключити до мікроконтролера температурний датчик LM-35. Реалізувати циклічну передачу (раз на 10 секунд) наступного рядка даних на смартфон: «[HH:MM:SS] Temperature: XX °C», де [HH:MM:SS] – поточний час, XX – виміряне значення температури.

Контрольні запитання

1. Опишіть архітектуру мереж стандарту Bluetooth.
2. Поясніть структуру стеку протоколів Bluetooth.
3. Розкрийте особливості використання профілів Bluetooth.
4. Поясніть фізичні особливості роботи радіоканалу Bluetooth.
5. Обґрунтуйте необхідність використання стрибкоподібної зміни частоти радіоканалу Bluetooth.
6. Проаналізуйте відмінності між різними типами з'єднань Bluetooth.
7. Опишіть призначення та основні завдання рівня L2CAP.
8. Розкрийте структуру кадрів Bluetooth.
9. Поясніть відмінності між форматами кадрів Bluetooth для різних швидкостей передачі даних.
10. Опишіть призначення та особливості роботи модуля HC-05.
11. Опишіть архітектуру та особливості роботи годинника реального часу в мікроконтролерах STM32.
12. Поясніть порядок налаштувань мікроконтролера STM32 для роботи з RTC.
13. Поясніть, з яких міркувань обираються налаштування необхідного інтерфейсу мікроконтролера STM32 для роботи з модулем HC-05.
14. Опишіть призначення та області використання структур у мові C.
15. Поясніть відмінності між функціями *strcpy* та *strcat*.
16. Обґрунтуйте необхідність використання колбеків (callbacks) для створення функцій-обробників переривань в STM32.
17. Поясніть, чому робоча частота зовнішнього кварцового резонатора для тактування годинника реального часу дорівнює саме 32,768 кГц.

РОБОТА З СЕНСОРНИМ TFT ДИСПЛЕЄМ

Мета: Ознайомитись з особливостями роботи TFT дисплеїв роздільною здатністю 320×240 пікселів на контролері ILI9341 та сенсорних модулів на контролері ХРТ2046. Навчитись створювати програмний код для виведення графічної інформації на TFT дисплей. Ознайомитись та навчитись використовувати сенсорний модуль для взаємодії з графічними елементами на дисплеї.

Теоретичні відомості

TFT (Thin-Film Transistor) дисплеї є різновидом рідкокристалічних дисплеїв (LCD) з активною матрицею, де тонкоплівкові транзистори (TFT) використовуються для керування кожним пікселем індивідуально. TFT дисплеї широко застосовуються в моніторах, телевізорах, смартфонах та мікроконтролерних проєктах завдяки своїй компактності та низькій вартості.

TFT дисплей складається з кількох шарів (рис. 7.1):

- LED-підсвітка, оскільки TFT є пасивними дисплеями і не випромінюють світло самостійно, а лише модулюють світло від підсвітки;
- поляризаційні фільтри (один з горизонтальною поляризацією, другий – з вертикальною), які контролюють проходження світла;
- шар рідких кристалів;
- шар тонкоплівкових транзисторів (TFT), в яких кожен піксель (або субпіксель – R, G, B) має свій транзистор і конденсатор. Транзистори розміщені на скляній підкладці (зазвичай аморфний кремній або полікристалічний кремній);

- колірні фільтри з червоними, зеленими та синіми субпікселями для створення кольорів (RGB);
- захисне скло.

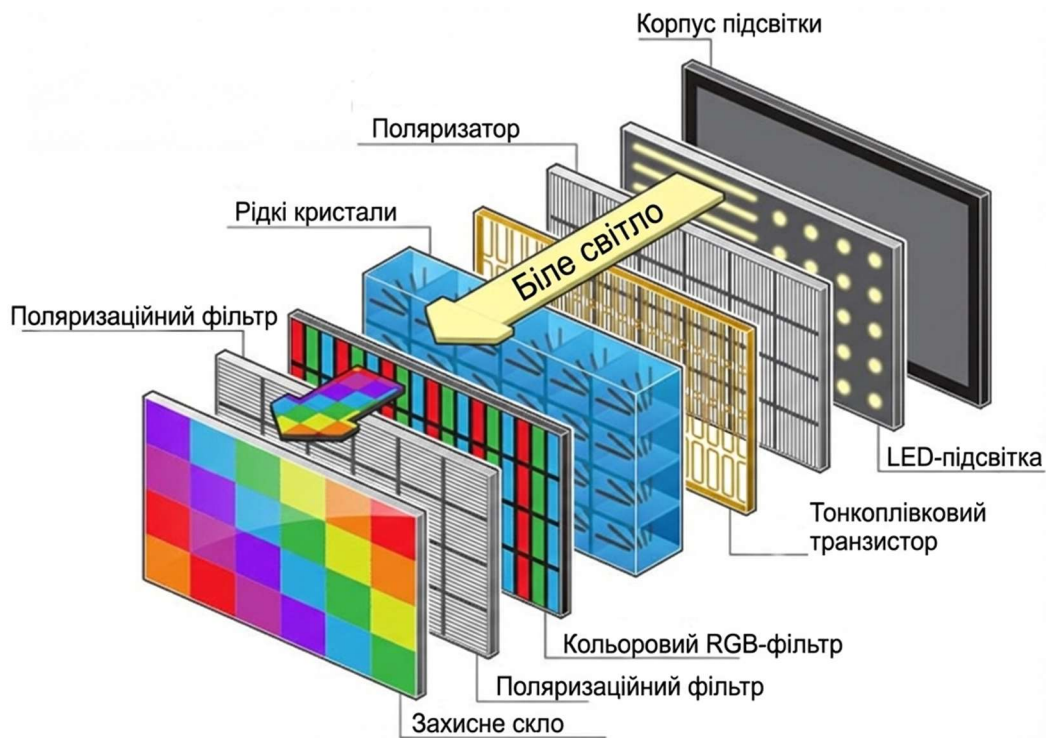


Рис. 7.1. Узагальнена структура TFT дисплею

Робота TFT базується на електрооптичних властивостях рідких кристалів і активному керуванні транзисторами. Без напруги (у стані спокою) молекули рідких кристалів скручені (у TN-структурі на 90°). Світло від підсвітки проходить через перший поляризатор, скручується разом з молекулами і виходить через другий поляризатор. Піксель "увімкнений" (світлий).

У разі прикладення напруги електричне поле вирівнює молекули рідких кристалів перпендикулярно до скла. Світло блокується другим поляризатором, оскільки його поляризація не змінюється. Піксель "вимкнений" (темний).

Такий дисплей зазвичай має матрицю рядків і стовпців. Контролер дисплею активує рядок через ключ на основі транзистора, а потім надсилає заряд по стовпцю до конденсатора пікселя. Конденсатор утримує заряд до наступного оновлення, дозволяючи незалежне керування кожним пікселем без витoku заряду. Напруга на транзисторі контролює ступінь скручування молекул, регулюючи інтенсивність світла (від 0% до 100% для градацій сірого/кольору).

Для оновлення зображення контролер дисплею сканує екран рядок за рядком (line-by-line). Для кодування кольору використовується формат RGB565 (16-біт). Швидкість оновлення – до 60 Гц або більше, залежно від розміру екрану.

TFT дисплеї споживають мало енергії, оскільки транзистори дозволяють оновлювати тільки змінені пікселі. Такі дисплеї можуть мати порівняно високу роздільну здатність та швидкий відгук (5-10 мс). Серед недоліків можна виділити обмежені кути огляду (близько 70°), потребу в підсвітці, нижчий контраст порівняно з OLED.

Резистивні сенсорні дисплеї (або тачскріни) є одним з найстаріших і найпростіших типів сенсорних технологій, які використовуються для виявлення дотику на поверхні екрану. Вони базуються на принципі зміни електричного опору при фізичному контакті. Ці дисплеї не реагують на дотик пальця безпосередньо (як ємнісні), а вимагають певного тиску, тому вони добре працюють зі стилусами, рукавичками чи будь-якими предметами, що створюють тиск.

Типовий резистивний тачскрін складається з таких основних шарів (рис. 7.2):

- верхній шар (гнучкий) – зазвичай поліестерова плівка, покрита зсередини тонким резистивним матеріалом. Цей шар має електроди (контакти) по вертикальній осі (Y+ і Y-).
- нижній шар (жорсткий) – це скляна або пластикова основа, також покрита резистивним матеріалом, з електродами по горизонтальній осі (X+ і X-).
- між шарами розміщені маленькі ізоляційні роздільні точки (спейсери), які запобігають випадковому контакту. Повітряний зазор становить близько 0.1-0.2 мм.
- зовнішня поверхня верхнього шару може мати антиблікове або захисне покриття.

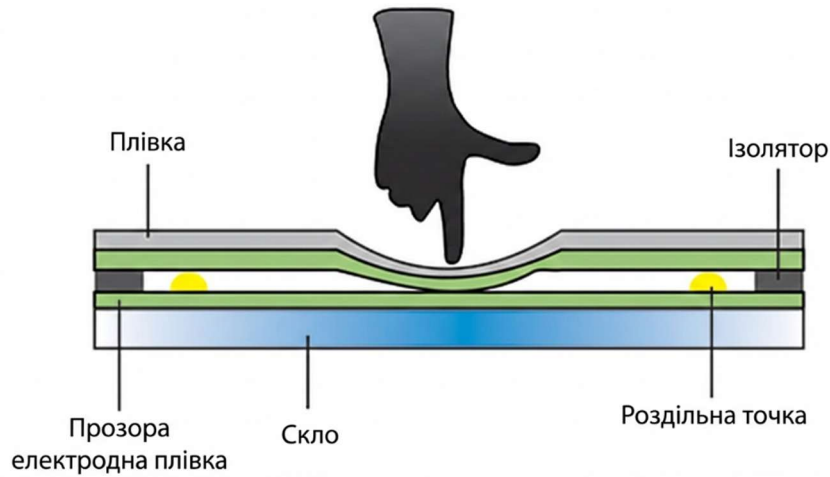


Рис. 7.2. Узагальнена структура резистивного сенсорного екрану

Ця конструкція називається 4-дротовою (4-wire), яка є найпоширенішою для простих дисплеїв. Існують також 5-, 6-, 7- і 8-дротові варіанти для підвищення точності (наприклад, за рахунок компенсації опору електродів), але вони складніші.

Робота резистивного тачскріну базується на створенні електричного контакту між шарами при натисканні. Без дотику шари розділені, електричний ланцюг розімкнутий. Контролер тачскріну може відслідковувати поточне значення опору (наприклад, через pull-up резистор) для виявлення стану "спокою". При дотику верхній шар прогинається, торкаючись нижнього в точці контакту. Це створює змінний опір, який залежить від положення дотику. Точка контакту по суті діє як потенціометр (подільник напруги) в двовимірній площині. Вимірювання координат дотику відбувається по чергово для осей X і Y, а також для Z (для визначення тиску – сили натискання).

Ці вимірювання є аналоговими, тому потрібен аналого-цифровий перетворювач для конвертації в цифрові значення. Для підвищення точності використовується раціометричний метод (відношення до опорної напруги), що компенсує нерівномірність напруги живлення. Також, контролер сенсорного екрану може генерувати переривання при виявленні дотику, щоб зменшити навантаження на процесор.

Точність визначення координат дотику залежить від рівномірності резистивного-покриття. Для відповідності координатам (пікселям) дисплея потрібно проводити калібрування. Сенсорні екрани, що використовуються у цій лабораторній роботі, уже відкалібровані.

Основними перевагами резистивних екранів є низька вартість, простота конструкції та порівняно висока точність для одnodотикових систем. Недоліки: менша чутливість, знос з часом, обмежена підтримка мультитач (зазвичай тільки один дотик) та менша прозорість порівняно з ємнісними екранами. Такі екрани частіше використовуються в промислових панелях та медичних пристроях, де потрібна робота в рукавичках, а у секторі споживчих гаджетів витісняються ємнісними екранами.

Модуль дисплею 2.8" 240x320 TFT LCD SPI (рис. 7.3) є компактним кольоровим екраном з сенсорним вводом. Модуль використовує послідовний інтерфейс SPI для зв'язку з мікроконтролером, що робить його простим у підключенні та ефективним у роботі з обмеженими ресурсами. Цей модуль часто застосовується в проектах IoT, інтерфейсах користувача, відображенні даних з датчиків, портативних пристроях та освітніх експериментах.

Основні характеристики:

- Розмір екрану: 2.8 дюйма (діагональ);
- Роздільна здатність: 240x320 пікселів (QVGA);
- Кольорова глибина: 262K (18-біт) або 65K (16-біт) кольорів, з підтримкою RGB-форматів.
- Інтерфейс: SPI (один для дисплея та сенсорного екрану одночасно);
- Живлення: 3.3V, сумісність з 5V.
- Сенсор: резистивний, роздільність 12 біт, підтримка одного дотику.
- Споживання струму: до 100 мА з підсвіткою.
- Додаткові функції: підтримка SD-карт (FAT16/32), орієнтація екрану (портретна/альбомна), кут огляду 70°.
- Продуктивність: оновлення екрану до 60 FPS при оптимізованому коді; затримка сенсорного екрану <10 мс.



Рис. 7.3. Модуль дисплею 2.8" 240x320 TFT з сенсорним екраном

Модуль підтримує роботу з SD-картами для завантаження зображень (наприклад, у форматі BMP), що дозволяє відображати графіку без постійної передачі даних від мікроконтролера.

Дисплей ILI9341 дозволяє відображати різноманітні типи інформації завдяки гнучкій графічній системі. Він підтримує шрифти різного розміру та стилю через бібліотеки, які генерують символи з бітмар або векторних шрифтів. Бітмар-шрифти являють собою растрові зображення символів фіксованого розміру (наприклад, 8x8 або 16x16 пікселів), що забезпечують швидке відображення, але обмежують масштабування – на кожен розмір тексту потрібен свій окремий шрифт. Також на дисплеї можна відображати прості фігури (лінії, прямокутники, кола), графіки, іконки та UI-елементи (кнопки, меню).

Можна виводити і кольорові зображення. Статичні зображення можуть бути завантажені безпосередньо через SPI (піксель за пікселем) або з SD-карти. Основний формат для виведення зображень на екран – RGB565, який є 16-бітним представленням кольору (5 біт на червоний компонент, 6 біт на зелений, 5 біт на синій), що забезпечує 65 536 можливих відтінків. Цей формат оптимізовано для ефективної передачі даних через SPI, але з деяким обмеженням глибини кольору. Бібліотеки конвертують зображення з BMP чи інших форматів у RGB565 перед відправкою. Доступні і прості анімації шляхом швидкого оновлення кадрів (наприклад, рухомі іконки, прогрес-бари чи базові ефекти. Ці можливості роблять модуль універсальним для відображення реального часу даних (наприклад, графіки сенсорів), статичних меню чи навіть простих ігор.

Контролер дисплея ILI9341 є основним чіпом, відповідальним за керування TFT-екраном. Цей 16-бітний контролер графіки, розроблений компанією ILI Technology Corp., обробляє команди для відображення пікселів, управління кольорами та оновлення екрану. Контролер інтегрує в себе RAM для буферизації даних (GRAM - Graphic RAM), що забезпечує швидке оновлення екрану без постійного доступу до мікроконтролера. Алгоритм роботи дисплея з ILI9341 полягає в тому, що мікроконтролер надсилає команди через SPI. Лінія DC (Data/Command) визначає тип передачі: низький рівень для команд (наприклад, ініціалізація, встановлення вікна для малювання), високий – для даних (пікселі, кольори). ILI9341 інтерпретує команди, записує дані в GRAM і оновлює екран. Підсвітка LED активується окремо для освітлення LCD-матриці.

Контролер сенсорного вводу XPT2046, що також входить до складу модуля дисплея, є мікросхемою для обробки сигналів від резистивного сенсорного шару з 4-дротовою схемою підключення. Він забезпечує виявлення дотиків з роздільною здатністю до 12 біт. Цей контролер перетворює аналогові сигнали від сенсорної панелі на цифрові координати (X, Y) та рівень тиску (Z). Контролер працює через той самий SPI-інтерфейс, що і дисплей, але з окремим виводом чіп-селект (CS). Він підтримує режим генерування переривань для ефективного виявлення подій дотику, що зменшує навантаження на мікроконтролер.

Опис основних виводів модуля дисплею 2.8" 240x320 TFT з сенсорним екраном наведено в таблиці 5.1.

Таблиця 7.1. Опис виводів модуля дисплею TFT з сенсорним екраном

Вивід	Призначення
T_IRQ	Переривання від сенсорного контролера XPT2046. Цей вивід переходить у низький рівень (логічний 0) у разі виявлення дотику.
T_DO	Вихід даних від сенсорного екрану (Touch Data Out, еквівалент MISO для XPT2046). Передає координати дотику та інші дані від XPT2046 до мікроконтролера через SPI.
T_DIN	Вхід даних для сенсорного екрану (Touch Data In, еквівалент MOSI для XPT2046). Приймає команди від мікроконтролера для опитування або конфігурації XPT2046.
T_CS	Вибір мікросхеми для сенсорного екрану (Touch Chip Select). Керує активацією XPT2046: низький рівень активує сенсор для SPI-зв'язку, високий – деактивує, уникаючи конфліктів з дисплеєм.

Вивід	Призначення
T_CLK	Тактовий сигнал для сенсора (Touch Clock). Спільний з дисплеєм, використовується для синхронізації даних по SPI для XPT2046.
SDO <MISO>	Вихід даних від дисплея (Serial Data Out, MISO для ILI9341). Передає дані від ILI9341 до мікроконтролера. Спільний з T_DO. Дисплей не завжди використовує MISO, тому читання може бути обмеженим.
LED	Керування підсвіткою. Підключається безпосередньо до 3.3V для постійного максимального рівня яскравості; для регулювання яскравості можна використовувати ШІМ.
SCK	Тактовий сигнал для дисплея (Serial Clock). Спільний з T_CLK, забезпечує синхронізацію SPI для ILI9341.
SDI <MOSI>	Вхід даних для дисплея (Serial Data In, MOSI для ILI9341). Приймає команди та дані від мікроконтролера. Спільний з T_DIN.
DC	Визначення типу даних для дисплея (Data/Command). Низький рівень – команда (наприклад, ініціалізація), високий – дані (пікселі). Критичний для правильної роботи ILI9341.
RESET	Скидання дисплея. Короткий негативний імпульс скидає ILI9341 до початкового стану, необхідного для ініціалізації.
CS	Вибір мікросхеми для дисплея (Chip Select). Низький рівень активує ILI9341 для SPI-зв'язку, високий – деактивує.
GND	Загальна земля.
VCC	Живлення модуля. 3.3V живить логіку ILI9341, XPT2046 та підсвітку. Можна використовувати 5V.

Схема підключення даного модуля до плати STM32 “Blue Pill” показана на рис. 7.4.

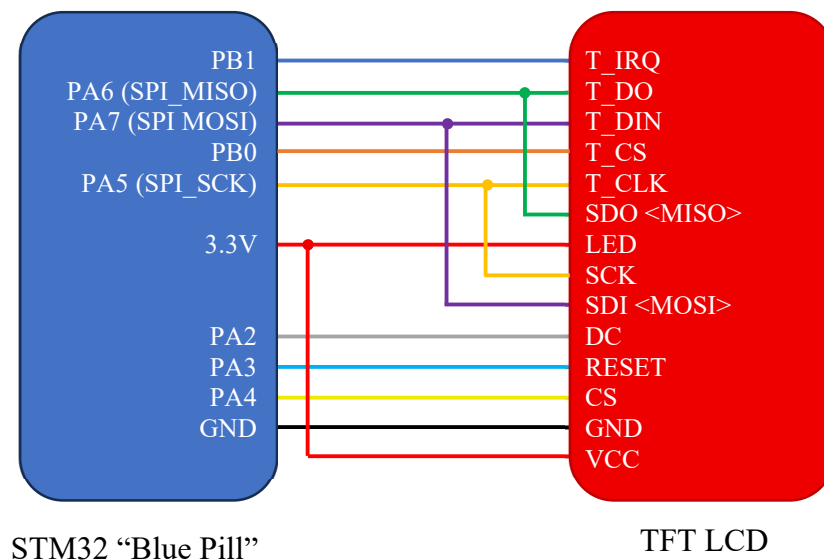


Рис. 7.4. Схема підключення модуля дисплею 2.8" 240x320 TFT з сенсорним екраном до плати STM32 “Blue Pill”

У наведеній схемі підключення модуль використовує спільний SPI-інтерфейс (SPI1 на STM32 “Blue Pill”) для дисплея та сенсора, з окремими лініями керування (T_CS для тачскріну та CS для дисплею). Це стандартна конфігурація для зменшення кількості задіяних виводів. Для початку роботи потрібно налаштувати SPI на STM32 в режимі майстра та ініціалізувати контролер дисплею ILI9341 і контролер сенсорного екрану XPT2046. Іноді може знадобитись додаткове калібрування сенсорного екрану, щоб його показання узгоджувались із координатами пікселів дисплею.

Хід виконання роботи

1. Виконати попередні налаштування плати «STM32 Blue Pill» для підключення TFT LCD модуля з сенсорним екраном.
 - 1.1. Запустити середовище програмування STM32CubeIDE та створити новий проект.
 - 1.2. Встановити джерело тактування від зовнішнього кварцового резонатора та режим відлагодження «*Serial Wire*».
 - 1.3. На вкладці *Clock Configuration* налаштувати тактування мікроконтролера. Встановити тактування від зовнішнього кварцового резонатора та основну частоту тактування 72 МГц.
 - 1.4. В розділі *Connectivity* активувати інтерфейс SPI1, обравши *Mode*: «Full-Duplex Master» та *Hardware NSS Signal*: «Disable». Після цього на графічній моделі мікроконтролера автоматично активуються виводи PA5-PA7, призначені для роботи з SPI1.
 - 1.5. В параметрах SPI1 встановити довжину слова даних 8 біт, обравши *Data Size*: «8-bit». Встановити значення подільника частоти *Prescaler*: «32», налаштувати режими *CPOL*: «Low» та *CPHA*: «1 Edge» та обрати програмне джерело сигналу синхронізації *NSS Signal Type*: «Software». Інші параметри залишити за замовчуванням.

- 1.6. На вкладці *Pinout & Configuration* активувати виводи PA2-PA4 в режимі активних виходів для підключення TFT LCD, а також вивід PB0 в режимі активного виходу та вивід PB1 в якості входу з підтягуванням до високого рівня для підключення сенсорного екрану.
- 1.7. У розділі *System Core* → *GPIO* встановити налаштування щойно активованих виводів у відповідності до рис. 7.5.
- 1.8. Зберегти проєкт та провести генерацію коду.

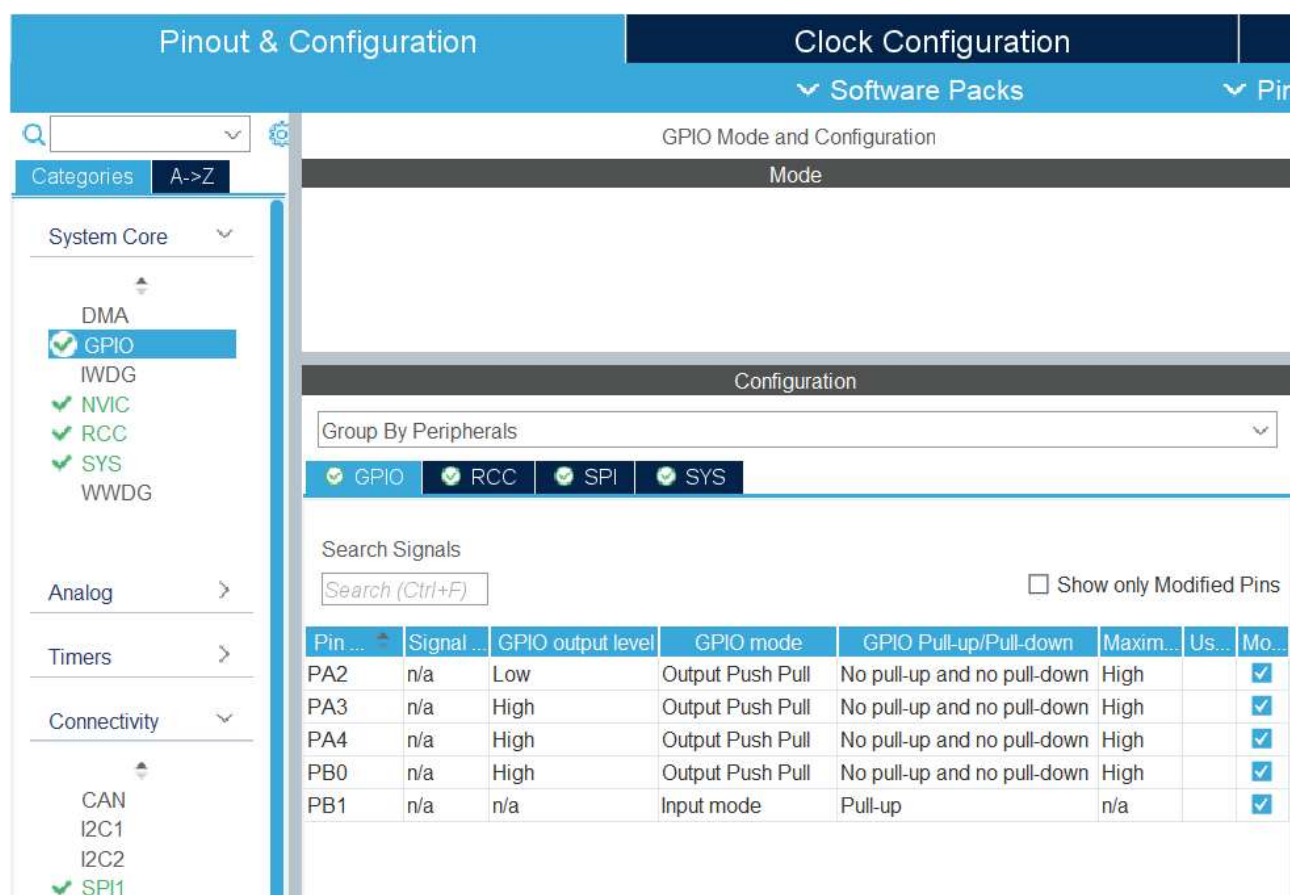


Рис. 7.5. Налаштування виводів GPIO для підключення TFT LCD з сенсорним екраном

2. Написати програму для виведення на TFT LCD напису «Hello, world!».
- 2.1. В дереві структури проєкту додати файли необхідних бібліотек для керування TFT LCD з сенсорним екраном. У папці *Inc* розмістити заголовкові файли бібліотеки *ili9341.h*, *ili9341_touch.h* та файл бібліотеки

шрифтів *fonts.h* і тестове зображення *testing.h*. У папці *Src* підключити відповідні вихідні файли *ili9341.c*, *ili9341_touch.c* та *fonts.c*.

2.2. У файлі *main.c* підключити необхідні бібліотеки:

```
/* USER CODE BEGIN Includes */
#include "ili9341.h"
#include "fonts.h"
/* USER CODE END Includes */
```

2.3. У блоці *USER CODE BEGIN 2* використати функції для ініціалізації дисплею, заповнення його чорним кольором та виведення на дисплей напису «Hello, world!». Для ініціалізації дисплею використовується функція *ILI9341_Init()*.

Функція *ILI9341_FillScreen()* повністю заповнює екран одним кольором (очищає/зафарбовує весь дисплей) і приймає один вхідний аргумент *color* – 16-бітний колір у форматі RGB565.

Функція *ILI9341_WriteString()* виводить текстовий рядок на дисплей у заданій позиції. Інформація про функцію:

- Формат: *void ILI9341_WriteString(uint16_t x, uint16_t y, const char* str, FontDef font, uint16_t color, uint16_t bgcolor)*.
- Аргументи:
 - *x* – координата по горизонталі (пікселі).
 - *y* – координата по вертикалі (пікселі).
 - *str* – рядок тексту.
 - *font* – шрифт (структура *FontDef*, наприклад *Font_11x18*).
 - *color* — колір тексту (RGB565).
 - *bgcolor* — колір фону під текстом (RGB565).

Використаємо ці функції:

```
/* USER CODE BEGIN 2 */
// Переконайтесь що контролер тачскріну не активний на SPI шині
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_SET);
```

```
ILI9341_Init();
ILI9341_FillScreen(ILI9341_BLACK);
ILI9341_WriteString(10, 10, "Hello world!", Font_11x18, ILI9341_WHITE,
ILI9341_BLACK);

/* USER CODE END 2 */
```

2.4. Головний цикл `while(1)` в даному випадку залишається пустим.

2.5. Підключити дисплей до мікроконтролеру за схемою, показаною на рис. 5.4.

2.6. Завантажити програму до мікроконтролеру та перевірити правильність її роботи. Критерієм правильності буде виведення білого напису «Hello, world!» на чорному фоні у верхньому лівому куті дисплею (якщо дисплей розміщено в альбомній орієнтації).

3. Модифікувати поточний код програми так, щоб напис "Hello world!" не виводився, а замість нього виводився напис "Kafedra ASNK" у вигляді біжучого рядка (зліва направо) по центру екрана.

3.1. Додати підключення бібліотеки *string* для роботи з рядковими даними у файлі *main.c*:

```
/* USER CODE BEGIN Includes */
#include "ili9341.h"
#include "fonts.h"
#include <string.h>
/* USER CODE END Includes */
```

3.2. Створити змінні для роботи з біжучим рядком. Змінні мають бути наступними:

- *g_text_w* – ширина всього рядка тексту в пікселях;
- *g_scroll_text* – текст, який відображається і «рухається» в рядку;
- *g_scroll_font* – вказівник на шрифт, яким малюється біжучий рядок;
- *g_x* – поточна координата X (лівий край) позиції, де виводиться рядок у поточному кадрі;
- *g_x_max* – максимально допустиме значення *g_x*, щоб текст не виходив за межі екрана.
- *g_y* – фіксована координата Y для рядка.

Реалізація в коді:

```
/* USER CODE BEGIN PV */
static uint16_t g_text_w = 0;
static const char g_scroll_text[] = "Kafedra ASNK";
static const FontDef *g_scroll_font = &Font_11x18;

static int32_t g_x = 0;
static int32_t g_x_max = 0;
static uint16_t g_y = 0;
/* USER CODE END PV */
```

3.3. Модифікувати код в блоці ініціалізації наступним чином:

```
/* USER CODE BEGIN 2 */

// Переконались що контролер тачскріну не активний на SPI шині
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_SET);

ILI9341_Init();
ILI9341_FillScreen(ILI9341_BLACK);

// Обчислити координату центру дисплея для обраного шрифту
g_y = (uint16_t)((ILI9341_HEIGHT - g_scroll_font->height) / 2u);

// Визначити довжину тексту у пікселях
const uint32_t text_w = (uint32_t)strlen(g_scroll_text) *
(uint32_t)g_scroll_font->width;
g_text_w = (uint16_t)text_w;

// Обчислити максимальне значення X щоб текст вміщався на дисплеї
g_x_max = (int32_t)ILI9341_WIDTH - (int32_t)text_w - 1;
if (g_x_max < 0) {
    g_x_max = 0;
}
g_x = 0;

/* USER CODE END 2 */
```

3.4. В головному циклі while(1) реалізувати алгоритм біжучого рядка:

```
/* USER CODE BEGIN 3 */
// Вивести текст на новій позиції
ILI9341_WriteString((uint16_t)g_x, g_y, g_scroll_text,
*g_scroll_font, ILI9341_WHITE, ILI9341_BLACK);

// очистити стовпець зліва від тексту після його зсуву на 1 крок
if (g_x == 0) {
    // Для руху справа-наліво
    if (g_x_max > 0) {
        ILI9341_FillRectangle((uint16_t)g_x_max, g_y, g_text_w,
g_scroll_font->height, ILI9341_BLACK);
    }
}
```

```

    } else {
        // Для руху зліва-направо
        ILI9341_FillRectangle((uint16_t)(g_x - 1), g_y, 1,
g_scroll_font->height, ILI9341_BLACK);
    }

    // Виконати рух зліва-направо
    g_x++;
    if (g_x > g_x_max) {
        g_x = 0;
    }

    // Затримка для керування швидкістю руху тексту
    HAL_Delay(20);

}
/* USER CODE END 3 */

```

3.5. Завантажити програму до мікроконтролера та перевірити правильність її роботи.

4. Модифікувати програму таким чином, щоб на дисплей виводились зелений трикутник, синє коло та червоний квадрат. Всі три фігури повинні розміщуватись на різних позиціях на дисплеї, мати різний розмір та не перетинатись.

4.1. У блоці *USER CODE BEGIN 4* створити функції для малювання заданих фігур та деякі допоміжні функції. Призначення функцій наступне:

- *FillHLineClipped(int32_t x, int32_t y, int32_t w, uint16_t color)* – малює горизонтальну заповнену лінію (фактично прямокутник висотою 1 піксель) у кольорі *color*, обрізаючи її по межах екрана; приймає аргументи: початок *x*, рядок *y*, довжина *w*.
- *FillCircle(int32_t x0, int32_t y0, int32_t r, uint16_t color)* – малює заповнене коло з центром (*x0*, *y0*), радіусом *r* і кольором *color*.
- *FillTriangle(int32_t x0, int32_t y0, int32_t x1, int32_t y1, int32_t x2, int32_t y2, uint16_t color)* – малює заповнений трикутник за трьома вершинами (*x0*,*y0*), (*x1*,*y1*), (*x2*,*y2*) у кольорі *color* з попереднім сортуванням вершин за *y* і обмеженням по висоті екрана.
- *iabs32(int32_t v)* – повертає модуль (абсолютне значення) цілого *v*.

- *SwapI32(int32_t *a, int32_t *b)* – міняє місцями значення двох змінних, на які вказують *a* і *b*.

Реалізація в коді:

```
/* USER CODE BEGIN 4 */

static inline int32_t iabs32(int32_t v) { return (v < 0) ? -v : v; }

static void FillHLineClipped(int32_t x, int32_t y, int32_t w, uint16_t
color)
{
    if (w <= 0) return;
    if (y < 0 || y >= (int32_t)ILI9341_HEIGHT) return;

    int32_t x0 = x;
    int32_t x1 = x + w - 1;

    if (x1 < 0 || x0 >= (int32_t)ILI9341_WIDTH) return;

    if (x0 < 0) x0 = 0;
    if (x1 >= (int32_t)ILI9341_WIDTH) x1 = (int32_t)ILI9341_WIDTH - 1;

    ILI9341_FillRectangle((uint16_t)x0, (uint16_t)y, (uint16_t)(x1 - x0 +
1), 1, color);
}

static void FillCircle(int32_t x0, int32_t y0, int32_t r, uint16_t color)
{
    if (r <= 0) return;

    int32_t x = r;
    int32_t y = 0;
    int32_t err = 1 - x;

    while (x >= y) {
        // Заповнення по горизонталі
        FillHLineClipped(x0 - x, y0 + y, 2 * x + 1, color);
        FillHLineClipped(x0 - x, y0 - y, 2 * x + 1, color);
        FillHLineClipped(x0 - y, y0 + x, 2 * y + 1, color);
        FillHLineClipped(x0 - y, y0 - x, 2 * y + 1, color);

        y++;
        if (err < 0) {
            err += 2 * y + 1;
        } else {
            x--;
            err += 2 * (y - x) + 1;
        }
    }
}
```



```

    }
}

static void SwapI32(int32_t *a, int32_t *b)
{
    int32_t t = *a; *a = *b; *b = t;
}

static void FillTriangle(int32_t x0, int32_t y0,
                        int32_t x1, int32_t y1,
                        int32_t x2, int32_t y2,
                        uint16_t color)
{
    // Сортуння за y: (x0,y0) верх, (x1,y1) середина, (x2,y2) низ
    if (y0 > y1) { SwapI32(&y0, &y1); SwapI32(&x0, &x1); }
    if (y1 > y2) { SwapI32(&y1, &y2); SwapI32(&x1, &x2); }
    if (y0 > y1) { SwapI32(&y0, &y1); SwapI32(&x0, &x1); }

    // Якщо неможливо намалювати - виходимо
    if (y0 == y2) return;

    // Масштабування діапазону Y в залежності від розміру дисплею
    int32_t yStart = y0;
    int32_t yEnd = y2;
    if (yStart < 0) yStart = 0;
    if (yEnd >= (int32_t)ILI9341_HEIGHT) yEnd = (int32_t)ILI9341_HEIGHT -
1;

    for (int32_t y = yStart; y <= yEnd; y++) {

        // xA на грані v0-v2
        int32_t xA;
        if (y2 != y0) {
            xA = x0 + (int32_t)((((int64_t)(x2 - x0) * (int64_t)(y - y0))
/ (int64_t)(y2 - y0)));
        } else {
            xA = x0;
        }

        // xB на грані v0-v1 для верхньої частини, або v1-v2 для нижньої
        int32_t xB;
        if (y < y1) {
            if (y1 != y0) {
                xB = x0 + (int32_t)((((int64_t)(x1 - x0) * (int64_t)(y -
y0)) / (int64_t)(y1 - y0)));
            } else {
                xB = x0;
            }
        } else {

```

```

        if (y2 != y1) {
            xB = x1 + (int32_t)((((int64_t)(x2 - x1) * (int64_t)(y -
y1)) / (int64_t)(y2 - y1)));
        } else {
            xB = x1;
        }
    }

    // Намалювати заповнювач між xA та xB
    int32_t xMin = (xA < xB) ? xA : xB;
    int32_t xMax = (xA > xB) ? xA : xB;
    FillHLineClipped(xMin, y, (xMax - xMin + 1), color);
}
}

/* USER CODE END 4 */

```

4.2. Додати прототипи даних функцій у відповідному блоці:

```

/* USER CODE BEGIN PFP */
static void FillCircle(int32_t x0, int32_t y0, int32_t r, uint16_t
color);
static void FillTriangle(int32_t x0, int32_t y0,
                        int32_t x1, int32_t y1,
                        int32_t x2, int32_t y2,
                        uint16_t color);

/* USER CODE END PFP */

```

4.3. Модифікувати код в блоці ініціалізації наступним чином:

```

/* USER CODE BEGIN 2 */

// Переконатись що контролер тачскріну не активний на SPI шині
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_SET);

ILI9341_Init();
ILI9341_FillScreen(ILI9341_BLACK);

// Червоний квадрат
ILI9341_FillRectangle(20, 30, 60, 60, ILI9341_RED);
// Синє коло
FillCircle(180, 80, 30, ILI9341_BLUE);
// Зелений трикутник
FillTriangle(60, 220, 180, 220, 120, 140, ILI9341_GREEN);

/* USER CODE END 2 */

```

4.4. Завантажити програму до мікроконтролеру та перевірити правильність її роботи.

5. Модифікувати програму таким чином, щоб на дисплей спочатку виводився білий напис "Kafedra ASNK" посередині на темно-синьому фоні, а потім через 3 секунди виводилось тестове зображення (*testing.h*).

5.1. Включити файл *testing.h* з бінарним кодом текстового зображення до файлу *main.c*:

```
/* USER CODE BEGIN Includes */
#include "ili9341.h"
#include "fonts.h"
#include <string.h>
#include "testing.h"
/* USER CODE END Includes */
```

5.2. Оголосити константу для визначення темно-синього кольору:

```
/* USER CODE BEGIN PD */
#define COLOR_DARK_BLUE ILI9341_COLOR565(0, 0, 96)
/* USER CODE END PD */
```

5.3. Замінити код в блоці ініціалізації на наступний:

```
/* USER CODE BEGIN 2 */

// Переконайтесь що контролер тачскріну не активний на SPI шині
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_SET);

ILI9341_Init();

// Темно-синій фон та напис по центру
ILI9341_FillScreen(COLOR_DARK_BLUE);

const FontDef font = Font_11x18;
const char *text = "Kafedra ASNK";

// Центрування
const uint16_t text_w = (uint16_t)(strlen(text) * font.width);
const uint16_t text_h = font.height;

const uint16_t x = (text_w < ILI9341_WIDTH) ?
(uint16_t)((ILI9341_WIDTH - text_w) / 2u) : 0u;
const uint16_t y = (text_h < ILI9341_HEIGHT) ?
(uint16_t)((ILI9341_HEIGHT - text_h) / 2u) : 0u;

ILI9341_WriteString(x, y, text, font, ILI9341_WHITE, COLOR_DARK_BLUE);

// Пауза 3 секунди
HAL_Delay(3000);

// Показ тестового зображення
ILI9341_FillScreen(ILI9341_BLACK);
```

```
// testing.h містить зображення 120x120 у форматі RGB565
const uint16_t img_w = 120;
const uint16_t img_h = 120;

const uint16_t img_x = (img_w < ILI9341_WIDTH) ?
(uint16_t)((ILI9341_WIDTH - img_w) / 2u) : 0u;
const uint16_t img_y = (img_h < ILI9341_HEIGHT) ?
(uint16_t)((ILI9341_HEIGHT - img_h) / 2u) : 0u;

ILI9341_DrawImage(img_x, img_y, img_w, img_h, (const
uint16_t*)&test_img_120x120[0][0]);

/* USER CODE END 2 */
```

5.4. Головний цикл while(1) в даному випадку залишити пустим.

5.5. Завантажити програму до мікроконтролеру та перевірити правильність її роботи.

6. Змінити програму таким чином, щоб на дисплей виводились координати останнього місця дотику до сенсорного екрану. Координати мають виводитись у лівому верхньому куті дисплею. Місце останнього дотику до екрану має відмічатись символом "X" зеленого кольору. При новому дотику попередня позначка "X" має стиратись і відмічатись нова, а також оновлюватись координати.

6.1. Додати підключення бібліотеки *ili9341_touch.h* для роботи з сенсорним екраном та бібліотеки *stdio.h* у файлі *main.c*:

```
/* USER CODE BEGIN Includes */
#include "ili9341.h"
#include "fonts.h"
#include <string.h>
#include "ili9341_touch.h"
#include <stdio.h>

/* USER CODE END Includes */
```

6.2. Створити змінні для роботи з сенсорним екраном. Змінні мають бути наступними:

- *g_prev_pressed* – зберігає попередній стан натискання сенсорного екрану (0/1), щоб виявляти момент нового дотику;

- *g_has_marker* – прапорець, що на екрані вже намальована позначка (маркер), яку треба стерти перед новою;
- *g_marker_x* – X-координата (верхній лівий кут) попередньо намальованого маркера на дисплеї;
- *g_marker_y* – Y-координата (верхній лівий кут) попередньо намальованого маркера на дисплеї.

Реалізація в коді:

```
/* USER CODE BEGIN PV */
static uint8_t g_prev_pressed = 0;
static uint8_t g_has_marker = 0;
static uint16_t g_marker_x = 0;
static uint16_t g_marker_y = 0;
/* USER CODE END PV*/
```

6.3. В боці ініціалізації замінити код таким чином, щоб створити інтерфейс для виведення значень координат натискання:

```
/* USER CODE BEGIN 2 */

// Переконайтесь що контролер тачскріну не активний на SPI шині
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_SET);

ILI9341_Init();
ILI9341_FillScreen(ILI9341_BLACK);

// Початкові написи в лівому верхньому куті дисплею
ILI9341_FillRectangle(0, 0, ILI9341_WIDTH, Font_11x18.height,
ILI9341_BLACK);
ILI9341_WriteString(0, 0, "X:--- Y:---", Font_11x18, ILI9341_WHITE,
ILI9341_BLACK);
/* USER CODE END 2 */
```

6.4. Реалізувати алгоритм опрацювання дотику до сенсорного екрану в головному циклі. Для детектування дотику використовується функція ***ILI9341_TouchPressed()***. Координати натискання можна дізнатись із використанням функції ***ILI9341_TouchGetCoordinates()***.

```
/* USER CODE BEGIN 3 */
uint8_t pressed = ILI9341_TouchPressed();
if (pressed && !g_prev_pressed)
{
    // Зачитуємо координати дотику
    uint16_t x_raw, y_raw;
```

```

    if (ILI9341_TouchGetCoordinates(&x_raw, &y_raw)) {
        // Врахуємо альбомну орієнтацію дисплею:
        uint16_t x = x_raw;
        uint16_t y = y_raw;

        // Стираємо попередній маркер
        if (g_has_marker) {
            ILI9341_WriteString(g_marker_x, g_marker_y, "+",
Font_11x18, ILI9341_BLACK, ILI9341_BLACK);
        }
        // Оновлюємо координати в лівому верхньому куті
        char buf[32];
        snprintf(buf, sizeof(buf), "X:%03u Y:%03u", x, y);

        ILI9341_FillRectangle(0, 0, ILI9341_WIDTH,
Font_11x18.height, ILI9341_BLACK);
        ILI9341_WriteString(0, 0, buf, Font_11x18,
ILI9341_WHITE, ILI9341_BLACK);

        // Малюємо новий маркер
        if (x < ILI9341_WIDTH && y < ILI9341_HEIGHT) {
            uint16_t mx = x;
            uint16_t my = y;

            const uint16_t fw = Font_11x18.width;
            const uint16_t fh = Font_11x18.height;

            // Обмежуємо значення X
            if ((uint16_t)(mx + fw) >= ILI9341_WIDTH) {
                mx = (uint16_t)(ILI9341_WIDTH - fw - 1u);
            }

            // Обмежуємо значення Y
            if ((uint16_t)(my + fh) > ILI9341_HEIGHT) {
                my = (uint16_t)(ILI9341_HEIGHT - fh);
            }

            ILI9341_WriteString(mx, my, "X", Font_11x18,
ILI9341_GREEN, ILI9341_BLACK);

            g_marker_x = mx;
            g_marker_y = my;
            g_has_marker = 1;
        }
    }
    g_prev_pressed = pressed;
    // Невелика затримка
    HAL_Delay(10);
}
/* USER CODE END 3 */

```

6.5. Завантажити програму до мікроконтролера та перевірити правильність її роботи. Координати маркеру можуть незначно відрізнятись від фактичного місця натискання, це нормально. Однак, якщо відхилення значні – необхідно провести калібрування сенсорного екрану.

7. Змінити програму таким чином, щоб дотик і проведення пальцем або стилусом по сенсорному екрану дозволяло малювати лінію червоного кольору. Також має бути передбачена червона кнопка з білим написом "Очистити", яка очищує все що було попередньо намальовано на дисплеї. Кнопка повинна розміщуватись у правому верхньому куті дисплею.

7.1. Оголосити константи зі значеннями відступів кнопки від краю дисплею:

```
/* USER CODE BEGIN PD */
#define BTN_MARGIN    2u
#define BTN_PAD_X     6u
#define BTN_PAD_Y     4u
/* USER CODE END PD */
```

7.2. Оновити блок з оголошенням змінних, створивши деякі нові:

- *g_prev_pressed* – зберігає попередній стан натискання сенсорного екрану (0/1), щоб виявляти момент нового дотику;
- *g_drawing* – прапорець режиму малювання: 1, коли стилус/палець зараз притиснутий і треба продовжувати лінію; 0, коли відпущено;
- *g_prev_x* – попередня X-координата останньої точки малювання (на дисплеї), використовується як початок відрізка для з'єднання з новою точкою;
- *g_prev_y* – попередня Y-координата останньої точки малювання (на дисплеї), використовується разом із *g_prev_x* для побудови безперервної лінії;
- *g_btn_label* – текст мітки кнопки очищення;
- *g_btn_x*, *g_btn_y*, *g_btn_w*, *g_btn_h* – координати лівого верхнього кута (*x*, *y*) кнопки та її розміри (ширина *w*, висота *h*) у пікселях, потрібні щоб малювати кнопку та перевіряти потрапляння дотику в її область;

- *g_btn_text_x*, *g_btn_text_y* – координати виводу тексту кнопки, тобто позиція, з якої друкується текст на кнопці (обчислюється для правильного центрування напису всередині кнопки).

Реалізація в коді:

```
/* USER CODE BEGIN PV */
static uint8_t g_prev_pressed = 0;
static uint8_t g_drawing = 0;
static uint16_t g_prev_x = 0;
static uint16_t g_prev_y = 0;

static const char g_btn_label[] = "Clear";
static uint16_t g_btn_x = 0, g_btn_y = 0, g_btn_w = 0, g_btn_h = 0;
static uint16_t g_btn_text_x = 0, g_btn_text_y = 0;
/* USER CODE END PV*/
```

7.3. В блоці *USER CODE BEGIN 4* створити функції для малювання заданих фігур та деякі допоміжні функції. Призначення функцій наступне:

- *App_DrawClearButton(void)* – малює контур кнопки, заповнює прямокутник червоним і виводить білий напис у межах кнопки.
- *App_PointInClearButton(uint16_t x, uint16_t y)* – перевіряє, чи потрапляє точка дотику (*x*, *y*) у прямокутну область кнопки.
- *App_ClearCanvas(void)* – очищує екран (заливає чорним) і одразу перемальовує кнопку, щоб вона залишалась видимою після очищення.
- *App_IAbs32(int32_t v)* – швидкий цілочисельний модуль числа *v*, допоміжна функція для алгоритму малювання лінії.
- *App_DrawLine(int32_t x0, int32_t y0, int32_t x1, int32_t y1, uint16_t color)* – малює відрізок між двома точками за алгоритмом Брезенгема (піксель за пікселем із використанням функції *ILI9341_DrawPixel()* з бібліотеки дисплею), з перевіркою виходу за межі екрана і заборонаю малювання поверх кнопки (щоб кнопка завжди читалась).

Реалізація в коді:


```

/* USER CODE BEGIN 4 */
static void App_DrawClearButton(void)
{
    // Червона кнопка з білим текстом
    ILI9341_FillRectangle(g_btn_x, g_btn_y, g_btn_w, g_btn_h,
    ILI9341_RED);
    ILI9341_WriteString(g_btn_text_x, g_btn_text_y, g_btn_label,
    Font_11x18, ILI9341_WHITE, ILI9341_RED);
}

static uint8_t App_PointInClearButton(uint16_t x, uint16_t y)
{
    return (x >= g_btn_x) && (x < (uint16_t)(g_btn_x + g_btn_w)) &&
    (y >= g_btn_y) && (y < (uint16_t)(g_btn_y + g_btn_h));
}

static void App_ClearCanvas(void)
{
    ILI9341_FillScreen(ILI9341_BLACK);
    App_DrawClearButton();
}

static int32_t App_IAbs32(int32_t v) { return (v < 0) ? -v : v; }

// Алгоритм Брезенгема (малювання цілочисельної лінії)
static void App_DrawLine(int32_t x0, int32_t y0, int32_t x1, int32_t y1,
uint16_t color)
{
    int32_t dx = App_IAbs32(x1 - x0);
    int32_t sx = (x0 < x1) ? 1 : -1;
    int32_t dy = -App_IAbs32(y1 - y0);
    int32_t sy = (y0 < y1) ? 1 : -1;
    int32_t err = dx + dy;

    while (x0 != x1 && y0 != y1)
    {
        if (x0 >= 0 && y0 >= 0 && x0 < (int32_t)ILI9341_WIDTH && y0 <
(int32_t)ILI9341_HEIGHT)
        {
            // Не малюємо поверх кнопки – вона має бути завжди видима
            if (!App_PointInClearButton((uint16_t)x0, (uint16_t)y0))
            {
                ILI9341_DrawPixel((uint16_t)x0, (uint16_t)y0, color);
            }
        }

        const int32_t e2 = 2 * err;
        if (e2 >= dy) { err += dy; x0 += sx; }
        if (e2 <= dx) { err += dx; y0 += sy; }
    }
}

/* USER CODE END 4 */

```

7.4. Додати прототипи даних функцій у відповідному блоці:

```
/* USER CODE BEGIN PFP */
static void App_DrawClearButton(void);
static uint8_t App_PointInClearButton(uint16_t x, uint16_t y);
static void App_ClearCanvas(void);
static int32_t App_IABs32(int32_t v);
static void App_DrawLine(int32_t x0, int32_t y0, int32_t x1, int32_t y1,
uint16_t color);
/* USER CODE END PFP */
```

7.5. Модифікувати код в блоці ініціалізації наступним чином:

```
/* USER CODE BEGIN 2 */
// Переконайтесь що контролер тачскріну не активний на SPI шині
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO\_PIN\_SET);

ILI9341_Init();
ILI9341_FillScreen(ILI9341_BLACK);

// Розрахунок геометрії кнопки у правому верхньому куті /
{
    const FontDef font = Font_11x18;
    const uint16_t text_w = (uint16_t)(strlen(g_btn_label) *
font.width);
    const uint16_t text_h = font.height;

    g_btn_w = (uint16_t)(text_w + 2u * BTN_PAD_X);
    g_btn_h = (uint16_t)(text_h + 2u * BTN_PAD_Y);

    // Залишаємо запас справа, щоб WriteString не переносив текст
    g_btn_x = (uint16_t)(ILI9341_WIDTH - BTN\_MARGIN - g_btn_w);
    g_btn_y = (uint16_t)(BTN_MARGIN);

    g_btn_text_x = (uint16_t)(g_btn_x + BTN_PAD_X);
    g_btn_text_y = (uint16_t)(g_btn_y + BTN_PAD_Y);
}

App_DrawClearButton();

/* USER CODE END 2 */
```

7.6. Реалізувати алгоритм запуску програми та опитування сенсорного екрану в головному циклі:

```

/* USER CODE BEGIN 3 */
const uint8_t pressed = ILI9341_TouchPressed();

if (pressed)
{
    uint16_t x = 0, y = 0;

    if (ILI9341_TouchGetCoordinates(&x, &y))
    {
        // Нове натискання
        if (!g_prev_pressed)
        {
            if (App_PointInClearButton(x, y))
            {
                App_ClearCanvas();
                g_drawing = 0;
            }
            else
            {
                g_drawing = 1;
                g_prev_x = x;
                g_prev_y = y;

                // Перша точка штриха
                ILI9341_DrawPixel(x, y, ILI9341_RED);

                // Очищуємо кнопку "наверх", якщо раптом торкались її
                App_DrawClearButton();
            }
        }
        else
        {
            // Утримання (малювання)
            if (g_drawing)
            {
                if (App_PointInClearButton(x, y))
                {
                    // Якщо під час малювання зайшли на кнопку
                    g_drawing = 0;
                    App_DrawClearButton();
                }
                else
                {
                    // Малюємо штрих (сегмент)
                    if (x != g_prev_x || y != g_prev_y)
                    {
                        App_DrawLine((int32_t)g_prev_x,
                                      (int32_t)g_prev_y, (int32_t)x, (int32_t)y,
                                      ILI9341_RED);
                        g_prev_x = x;
                        g_prev_y = y;
                    }
                }
            }
        }
    }
}

```

```

// Кнопка має залишатись видимою
App_DrawClearButton();
    }
    }
    }
}
else
{
    // Відпускання
    g_drawing = 0;
}

g_prev_pressed = pressed;

// Невелика пауза
HAL_Delay(5);
}
/* USER CODE END 3 */

```

7.7. Завантажити програму до мікроконтролера та перевірити правильність її роботи.

Індивідуальні завдання

- Створити програму з графічним інтерфейсом користувача, в якій із використанням сенсорних кнопок можна здійснити наступні дії:
 - Вивести на дисплей геометричну фігуру (коло, квадрат або трикутник) розміри якої задаються користувачем;
 - Вивести на дисплей поточну дату та час (задаються програмно у коді прошивки мікроконтролера з використанням годинника реального часу).
- Створити міні-гру у стилі «Хто хоче стати мільйонером?». Після запуску гри на дисплей має виводитись випадкове запитання (запитання зберігаються у флеш-пам'яті процесора) з чотирма варіантами відповіді. Гравець має обрати правильну відповідь. Якщо він відповідає вірно, то сума набраних за гру балів

збільшується, і на дисплей виводиться наступне запитання. Якщо відповідь було невірною, гра зупиняється з виведенням на дисплей напису «You lose». Якщо користувач правильно відповідає на 5 запитань поспіль, то гра завершується з написом «You win!».

3. Створити міні-гру, в якій на дисплей виводиться назва кольору, зафарбована у колір, що не відповідає даному слову (наприклад, **YELLOW**, **BLUE**, **RED** тощо). Задача гравця – за 5 секунд натиснути кнопку, колір якої відповідає тому, який написаний на дисплеї (а не тому, в який колір пофарбоване дане слово). Всього для натискання доступні 2 різні кнопки різного кольору, одна з яких відповідає написаному слову, а ще одна – кольору, яким це слово написано. У разі помилки гра зупиняється. У разі правильної відповіді користувач отримує бали, а на екрані з'являється нове кольорове слово та відповідні кнопки. Гра націлена на дослідження ефекту Струпа.
4. Створити мініатюрний графічний планшет для малювання за допомогою стилуса. Він має забезпечувати наступний функціонал:
 - 1) Малювання лінії одним із доступних кольорів, які обираються користувачем на окремій палітрі;
 - 2) Малювання кольорових квадратів одного із трьох доступних розмірів на позиції дисплею, яку користувач вказує стилусом;
 - 3) Режим «Гумки», в якому стилус не малює лінію, а стирає відповідні пікселі.
 - 4) Наявність кнопки для очищення полотна.

Контрольні запитання

1. Чому для модуля дисплея на контролері ILI9341 і сенсорного контролера XPT2046 важливо коректно керувати сигналами CS (CS і T_CS) при спільному використанні одного SPI, і що станеться, якщо активними будуть обидва пристрої одночасно?

2. За яких умов під час роботи з сенсорним екраном доцільно перейти з опитування в головному циклі на переривання по T_IRQ, і які ризики/складнощі це додасть у програмну логіку?
3. Чому «сирі» значення з контролеру сенсорного екрану не можна напряму трактувати як піксельні координати дисплею, і які перетворення потрібні, щоб отримати коректні координати в системі координат дисплею?
4. Для чого під час зчитування даних сенсорного екрану часто роблять усереднення кількох вимірів та вводять додаткові затримки?
5. Як обрати товщину «лінії» при малюванні на TFT, якщо базова функція малює лише один піксель, і як це вплине на швидкодію?
6. Які граничні умови треба врахувати при виведенні символів/тексту біля правого та нижнього краю екрана, щоб уникнути «переносу» або виходу за межі буфера/області відображення?
7. Як правильно організувати область кнопки «Очистити» в режимі малювання так, щоб малювання не «перекривало» кнопку, але водночас взаємодія з полотном була максимальною?
8. Якщо потрібно мінімізувати миготіння і прискорити відображення під час малювання, які два-три технічні підходи ви б розглянули?
9. Які параметри SPI є критичними для коректної роботи ILI9341 і ХРТ2046?
10. Як при зчитуванні дотику можна розрізнити «новий дотик» і «утримання»?
11. Для чого, на вашу думку, в алгоритмі малювання лінії використовуються лише цілочисельні операції (на кшталт алгоритму Брезенгема)?

РОЗРОБКА МОДУЛЯ ІНЕРЦІЙНОЇ НАВІГАЦІЇ

Мета: Ознайомитись з особливостями роботи модулів MEMS-акселерометрів, MEMS-гіроскопів та магнітометрів. Навчитись створювати прості системи аналізу та візуалізації даних з акселерометрів та гіроскопів.

Теоретичні відомості

MEMS-акселерометр є компактним мікроелектромеханічним пристроєм, призначеним для вимірювання лінійного прискорення об'єкта, до якого він прикріплений. Це робить його незамінним у системах моніторингу руху, орієнтації та вібрацій. Цей датчик знаходить широке застосування в портативних пристроях, таких як смартфони, фітнес-трекери та автомобільні системи безпеки, де він допомагає виявляти зміни положення, удари чи нахили.

У загальному випадку, основне призначення акселерометра полягає в перетворенні механічних сил на електричні сигнали, що дозволяє точно відстежувати динамічні та статичні стани об'єкта. Завдяки мініатюрним розмірам і низькому енергоспоживанню, MEMS-акселерометри легко інтегруються в складні вбудовані системи для аналізу даних в реальному часі. Вони часто комбінуються з іншими сенсорами, такими як гіроскопи, для більш повного уявлення про реальний рух об'єкта, що підвищує точність у задачах на кшталт стабілізації зображення або навігації.

Принцип роботи MEMS-акселерометра базується на ємнісному методі вимірювання прискорення. Його ключовим елементом є рухома маса, підвішена на гнучких пружинах усередині мікроскопічної структури (рис. 8.1), виготовленої з кремнію. Ця маса, часто вагою всього кілька мікрограмів, оточена фіксованими електродами, утворюючи конденсатори з повітряними зазорами між рухомими та нерухомими пластинами.

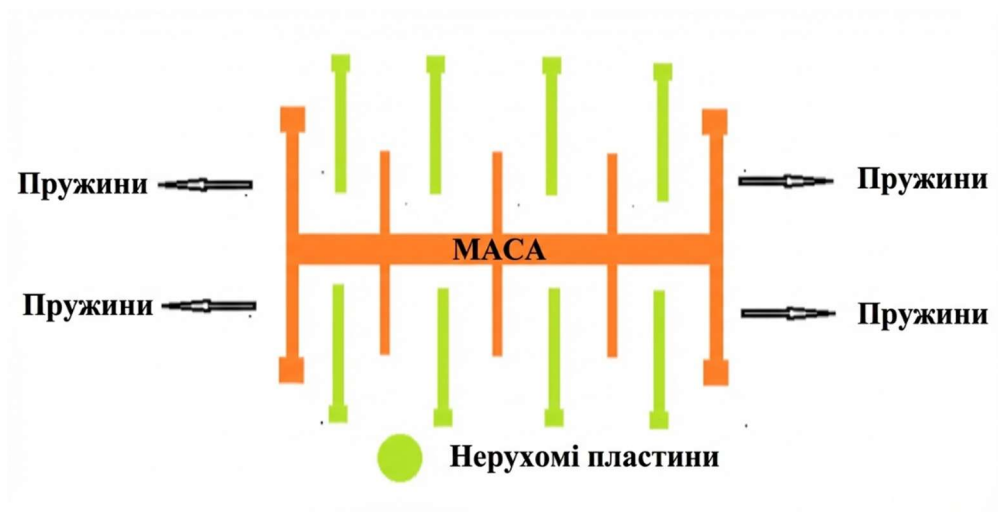


Рис. 8.1. Спрощена схема структури MEMS-акселерометра

Коли на датчик діє прискорення, маса зміщується відносно корпусу за законом інерції, змінюючи відстані в зазорах: з одного боку зазор зменшується, збільшуючи ємність конденсатора, а з іншого – збільшується, зменшуючи її. Ця зміна ємності пропорційна величині та напрямку прискорення, і вбудована електроніка перетворює її на аналоговий або цифровий сигнал. Для підвищення точності використовується диференціальний метод, в якому вимірюється різниця ємностей двох протилежних конденсаторів, що компенсує зовнішні впливи.

У моделях трьохосьових акселерометрів така структура повторюється для кожної осі – X, Y та Z, дозволяючи вимірювати прискорення в 3D-просторі. При статичному нахилі акселерометр реєструє компоненти гравітаційного прискорення вздовж осей, перетворюючи їх на кути нахилу за допомогою тригонометричних обчислень. Додаткові елементи, такі як фільтри або алгоритми корекції, допомагають усунути шум і дрейф, забезпечуючи стабільність показів.

Загалом, акселерометри такого класу чутливі до змін від кількох мілі-g до десятків g, з роздільною здатністю до 16 біт, що робить їх універсальним засобом для точних вимірювань. Зокрема, без них неможливо уявити сучасний дрон, ракету або інший автономний літальний апарат.

MEMS-гіроскоп являє собою компактний мікроелектромеханічний пристрій, призначений для вимірювання кутової швидкості обертання об'єкта навколо однієї або декількох осей, що робить його ключовим компонентом у системах орієнтації, стабілізації та навігації. Цей датчик широко застосовується в портативних пристроях, таких як смартфони, дрони та ігрові контролери, де він допомагає виявляти повороти, нахили та обертові рухи, забезпечуючи точне відстеження динаміки об'єкта в реальному часі.

Основне призначення гіроскопа полягає в перетворенні механічних ефектів обертання на електричні сигнали. Завдяки мініатюрним розмірам, низькому енергоспоживанню та можливості інтеграції з іншими сенсорами, такими як акселерометри, MEMS-гіроскопи сприяють створенню інерціальних вимірювальних модулів, які широко використовуються в автоматичних системах орієнтації та навігації.

Принцип роботи MEMS-гіроскопа ґрунтується на ефекті Коріоліса. Ключовим елементом такого сенсора є рухома маса, підвішена на гнучких структурах усередині мікроскопічної кремнієвої конструкції (рис. 8.2).

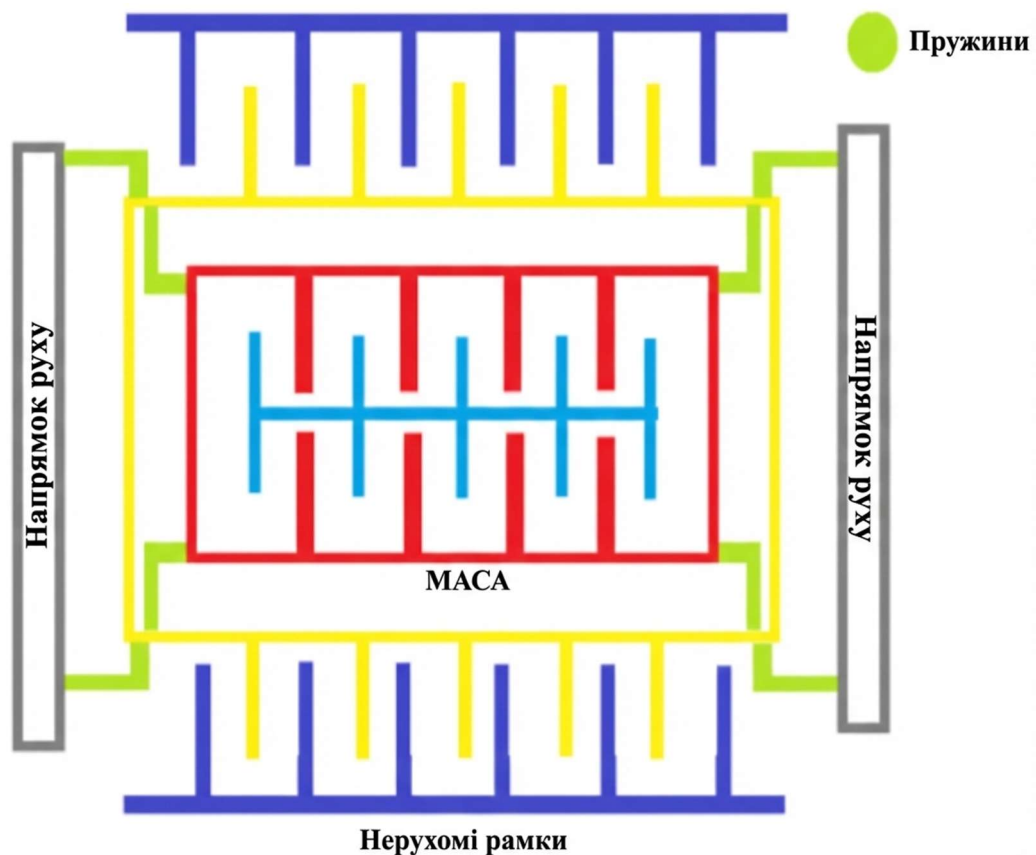


Рис. 8.2. Спрощена схема структури MEMS-гіроскопа

Ця маса, часто у формі рухомої рамки або пластинки вагою кілька мікрограм, примусово коливається вздовж однієї осі з постійною амплітудою та частотою, створюючи опорний рух. Коли об'єкт обертається навколо осі, перпендикулярної до напрямку коливань маси, виникає сила Коріоліса, яка пропорційна кутовій швидкості та масі елемента та викликає відхилення рухомої структури в перпендикулярному напрямку. Це відхилення змінює відстані між рухомими та фіксованими електродами, які утворюють конденсатори, що призводить до зміни ємності (подібно до механізму в акселерометрах). Вбудована електроніка безперервно вимірює цю зміну ємності, яка прямо корелює з величиною та напрямком кутової швидкості, та перетворює сигнал на цифрові дані.

У трьохосових моделях гіроскопів така структура дублюється для осей X, Y та Z, забезпечуючи повне просторове вимірювання. Для компенсації шумів, що викликають дрейф показів, часто застосовуються алгоритми фільтрації, наприклад фільтр Калмана, які прогнозують та коригують відхилення на основі попередніх даних. Загалом, цей принцип дозволяє датчику досягати чутливості від кількох градусів на секунду до тисяч, з роздільною здатністю до 16 біт, роблячи його ефективним для точних вимірювань. Оскільки гіроскоп вимірює кутові швидкості обертання, для визначення реального положення об'єкта у просторі потрібні додаткові тригонометричні обчислення.

Існує багато різноманітних модулів, які містять у своєму складі MEMS акселерометри, гіроскопи та навіть магнітometri (компаси). Модулі серії MPU (рис. 8.3) є компактними мікроелектромеханічними системами, призначеними для точного вимірювання напрямків руху та орієнтації в портативних пристроях. Вони забезпечують відстеження лінійного прискорення, кутової швидкості та, в деяких моделях, магнітного поля. Подібні моделі модулів часто мають вбудований цифровий процесор руху (DMP, Digital Motion Processor) для обробки даних незалежно від мікроконтролера, зменшує вимоги до потужності мікроконтролера та значно економить його ресурси.

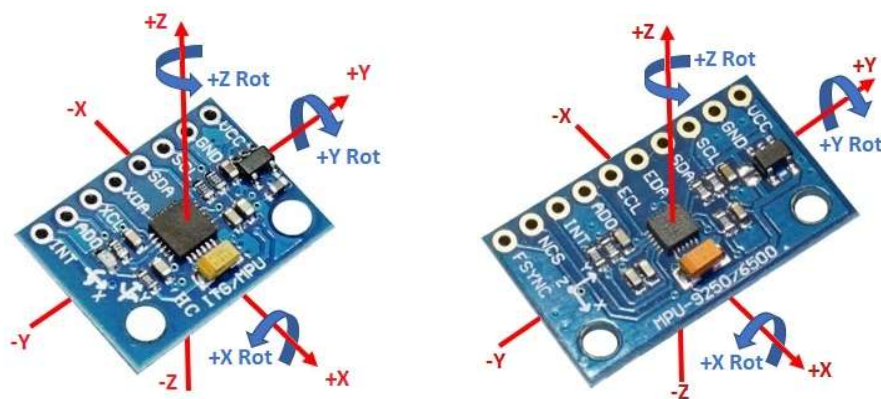


Рис. 8.3. Модулі на основі MPU: MPU6050 (зліва), MPU6500/9250 (справа)

Модуль MPU6050 є базовою 6-осьовою моделлю (3 осі гіроскопа + 3 осі акселерометра), ідеальною для простих задач з обмеженими вимогами до точності, таких як відстеження руху в гаджетах. MPU6500 є її покращеною версією з вищою продуктивністю, зменшеним шумом і підтримкою SPI-інтерфейсу, що робить її придатною для використання у високошвидкісних системах, таких як дрони чи VR-пристрої. MPU9250 розширює можливості до 9 осей, додаючи 3-осьовий магнітометр для вимірювання магнітного поля, що дозволяє створювати компаси і визначати абсолютну орієнтацію. Вони є оптимальними для створення точних навігаційних систем, автономних транспортних засобів чи систем доповненої реальності, де комбінація інерціальних та магнітних даних забезпечує повну картину орієнтації. Технічні характеристики модулів наведено в таблиці 8.1.

Таблиця 8.1. Основні характеристики модулів серії MPU

Специфікація	MPU6050	MPU6500	MPU9250
Осі	6 (3 гір. + 3 акс.)	6 (3 гір. + 3 акс.)	9 (3 гір. + 3 акс. + 3 маг.)
Діапазон акселерометра	$\pm 2g, \pm 4g, \pm 8g, \pm 16g$		
Діапазон гіроскопа	$\pm 250, \pm 500, \pm 1000, \pm 2000$ °/с		
Діапазон магнітометра	-	-	± 4800 μT
Інтерфейси	I ² C (до 400 кГц)	I ² C (до 400 кГц), SPI (1–20 МГц)	I ² C (до 400 кГц), SPI (1–20 МГц)
Розмір пам'яті	1024 байти	512 байти	512 байти
Напруга живлення	3.3 В		

Підключення цих модулів до мікроконтролера здійснюється через цифрові інтерфейси I²C (для всіх) або SPI (для MPU6500 та MPU9250). Живлення подається на VCC (3.3 В або 5 В через вбудований стабілізатор на деяких модулях) та GND. Вивід INT може використовуватись для формування зовнішніх переривань мікроконтролера для сповіщень про події. Допоміжні виводи ECL/EDA дозволяють додавати зовнішні датчики до системи. Схема підключення модуля MPU6050 до плати STM32 “Blue Pill” з підключеним TFT LCD показана на рис. 8.4.

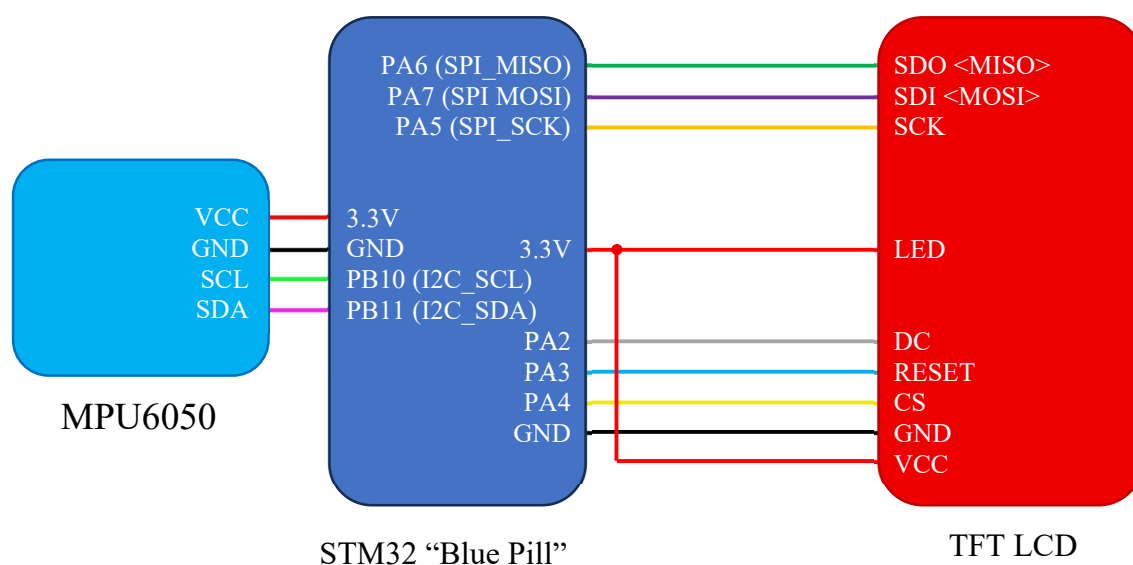


Рис. 8.4. Схема підключення модуля MPU6050 до плати STM32 “Blue Pill” з TFT LCD

Хід виконання роботи

1. Виконати попередні налаштування плати «STM32 Blue Pill» для підключення TFT LCD модуля та модуля MPU6500.
 - 1.1. Запустити середовище програмування STM32CubeIDE та створити новий проект.
 - 1.2. Встановити джерело тактування від зовнішнього кварцового резонатора та режим відлагодження «*Serial Wire*».

- 1.3. На вкладці *Clock Configuration* налаштувати тактування мікроконтролера.
Встановити тактування від зовнішнього кварцового резонатора та основну частоту тактування 72 МГц.
- 1.4. В розділі *Connectivity* активувати інтерфейс SPI1, обравши *Mode*: «Full-Duplex Master» та *Hardware NSS Signal*: «Disable». Після цього на графічній моделі мікроконтролера автоматично активуються виводи PA5-PA7, призначені для роботи з SPI1.
- 1.5. В параметрах SPI1 встановити довжину слова даних 8 біт, обравши *Data Size*: «8-bit». Встановити значення подільника частоти *Prescaler*: «4», налаштувати режими *CPOL*: «Low» та *CPHA*: «1 Edge» та обрати програмне джерело сигналу синхронізації *NSS Signal Type*: «Software». Інші параметри залишити за замовчуванням.
- 1.6. В конфігурації SPI1 на вкладці *DMA Settings* активувати режим запитів DMA для передачі даних на дисплей. Для цього додати новий запит за допомогою кнопки *Add*, після чого встановити *DMA Request*: «SPI1_TX», *Channel*: «DMA1 Channel 3», *Direction*: «Memory To Peripheral», *Priority*: «Very High», *Mode*: «Normal», *Increment Address*: «Memory», *Data Width*: «Byte».
- 1.7. В конфігурації SPI1 на вкладці *NVIC Settings* дозволити переривання за каналом «DMA1 channel3 global interrupt».
- 1.8. На вкладці *Pinout & Configuration* активувати виводи PA2-PA4 в режимі активних виходів для підключення TFT LCD.
- 1.9. У розділі *System Core* → *GPIO* встановити налаштування щойно активованих виводів у відповідності до рис. 8.5.
- 1.10. В розділі *Connectivity* активувати інтерфейс I2C2, обравши *Mode*: «I2C». Встановити стандартний режим швидкості передачі даних, обравши *I2C Speed Mode*: «Standard Mode», та значення швидкості передачі даних *I2C Clock Speed*: «100000». Інші налаштування залишити за замовчуванням.
- 1.11. Зберегти проєкт та провести генерацію коду.
- 1.12. Підключити модулі згідно зі схемою на рис. 8.4.



Рис. 8.5. Налаштування виводів GPIO для підключення TFT LCD

2. Написати програму для виведення на TFT LCD показань акселерометра, гіроскопа, а також орієнтації модуля у просторі у форматі pitch/yaw/roll.

2.1. В дереві структури проєкту додати файли необхідних бібліотек для керування TFT LCD з використанням DMA та модулем на основі MPU6050. У папці *Inc* розмістити заголовкові файли бібліотек *ili9341_dma.h*, *fonts.h* та *mpu6050.h*. У папці *Src* підключити відповідні вихідні файли *ili9341_dma.c*, *fonts.c* та *mpu6050.c*.

2.2. У файлі *main.c* підключити необхідні бібліотеки:

```
/* USER CODE BEGIN Includes */
#include <stdio.h>
#include "fonts.h"
#include "ili9341_dma.h"
#include "mpu6050.h"
/* USER CODE END Includes */
```

2.3. У блоці *USER CODE BEGIN PV* оголосити необхідні змінні:

- *MPU6050* – структура з поточними показаннями датчиків та обчисленими параметрами, які зчитуються з MPU6050 (прискорення, кутові швидкості, температура тощо);
- *g_yaw_deg* – накопичений кут рискання (yaw) у градусах, який обчислюється шляхом інтегрування кутової швидкості навколо осі Z (Gz);

- *g_yaw_last_ms* – часова мітка у мс з попереднього оновлення уaw; потрібна для обчислення *dt* між вимірюваннями під час інтегрування.

Реалізація в коді:

```
/* USER CODE BEGIN PV */
MPU6050_t MPU6050;
static float g_yaw_deg = 0.0f;
static uint32_t g_yaw_last_ms = 0;

/* USER CODE END PV*/
```

2.4. В блоці *USER CODE BEGIN 4* створити функції для опрацювання показань датчиків та деякі допоміжні функції. Призначення функцій наступне:

- *fmt_milli3(char *out, size_t out_sz, int32_t milli)* – форматує ціле значення в тисячних частках (*milli*) у рядок фіксованої ширини з знаком.
- *fmt_decil(char *out, size_t out_sz, int32_t deci)* – форматує ціле значення в десятих частках (*deci*) у рядок фіксованої ширини з знаком.
- *UI_DrawStatic(void)* – малює статичну частину інтерфейсу (заголовок і підписи секцій) один раз, без оновлення чисел.
- *UI_UpdateValues(const MPU6050_t *m)* – зчитує поточні дані з MPU6050_t, рахує/масштабує їх у фіксовані формати, викликає оновлення уaw і виводить рядки значень на дисплей.
- *Yaw_UpdateFromGyroZ(const MPU6050_t *m)* – інтегрує кутову швидкість Gz (deg/s) за часом dt, накопичуючи *g_yaw_deg*, і нормалізує результат у діапазон [-180; +180).

Реалізація в коді:

```
/* USER CODE BEGIN 4 */
static void fmt_milli3(char *out, size_t out_sz, int32_t milli)
{
```

```

char sign = '+';
int32_t a = milli;

if (a < 0)
{
    sign = '-';
    a = -a;
}

int32_t ip = a / 1000;
int32_t fp = a % 1000;

// фіксована довжина: s iii.fff
(void)snprintf(out, out_sz, "%c%3ld.%03ld", sign, (long)ip,
(long)fp);
}

```

```

static void fmt_dec1(char *out, size_t out_sz, int32_t deci)
{
    char sign = '+';
    int32_t a = deci;

    if (a < 0)
    {
        sign = '-';
        a = -a;
    }

    int32_t ip = a / 10;
    int32_t fp = a % 10;

    // фіксована довжина: s iii.f
    (void)snprintf(out, out_sz, "%c%3ld.%01ld", sign, (long)ip,
(long)fp);
}

```

```

static void UI_DrawStatic(void)
{
    ILI9341_FillScreen(ILI9341_BLACK);

    ILI9341_WriteString(8, 6, "MPU6050 MONITOR", Font_11x18,
ILI9341_GREEN, ILI9341_BLACK);

    ILI9341_WriteString(8, 34, "ACC (g)", Font_7x10,
ILI9341_YELLOW, ILI9341_BLACK);
    ILI9341_WriteString(8, 84, "GYRO (dps)", Font_7x10,
ILI9341_YELLOW, ILI9341_BLACK);
    ILI9341_WriteString(8, 134, "TEMP / RPY", Font_7x10,
ILI9341_YELLOW, ILI9341_BLACK);
}

```



```

static void UI_UpdateValues(const MPU6050_t *m)
{
    char ax[16], ay[16], az[16];
    char gx[16], gy[16], gz[16];
    char t1[16], r1[16], p1[16], y1[16];
    Yaw_UpdateFromGyroZ(m);

    // Масштабування до формату числа з фіксованою комою
    int32_t ax_mg = (int32_t)(m->Ax * 1000.0);
    int32_t ay_mg = (int32_t)(m->Ay * 1000.0);
    int32_t az_mg = (int32_t)(m->Az * 1000.0);

    int32_t gx_mdps = (int32_t)(m->Gx * 1000.0);
    int32_t gy_mdps = (int32_t)(m->Gy * 1000.0);
    int32_t gz_mdps = (int32_t)(m->Gz * 1000.0);

    int32_t temp_deci = (int32_t)(m->Temperature * 10.0); // 0.1 C
    int32_t roll_deci = (int32_t)(m->KalmanAngleX * 10.0); // 0.1 deg
    int32_t pitch_deci = (int32_t)(m->KalmanAngleY * 10.0); // 0.1 deg
    int32_t yaw_deci = (int32_t)(g_yaw_deg * 10.0); // 0.1 deg

    fmt_milli3(ax, sizeof(ax), ax_mg);
    fmt_milli3(ay, sizeof(ay), ay_mg);
    fmt_milli3(az, sizeof(az), az_mg);

    fmt_milli3(gx, sizeof(gx), gx_mdps);
    fmt_milli3(gy, sizeof(gy), gy_mdps);
    fmt_milli3(gz, sizeof(gz), gz_mdps);

    fmt_deci1(t1, sizeof(t1), temp_deci);
    fmt_deci1(r1, sizeof(r1), roll_deci);
    fmt_deci1(p1, sizeof(p1), pitch_deci);
    fmt_deci1(y1, sizeof(y1), yaw_deci);

    // Рядки фіксованої довжини
    char line[64];

    // Показання акселерометра
    (void)snprintf(line, sizeof(line), "X:%s Y:%s Z:%s", ax, ay, az);
    ILI9341_WriteString(8, 52, line, Font_7x10, ILI9341_CYAN,
ILI9341_BLACK);

    // Показання гіроскопа
    (void)snprintf(line, sizeof(line), "X:%s Y:%s Z:%s", gx, gy, gz);
    ILI9341_WriteString(8, 102, line, Font_7x10, ILI9341_WHITE,
ILI9341_BLACK);

    // Температура та ROLL/PITCH/YAW
    (void)snprintf(line, sizeof(line), "T:%s C", t1);

```

```

    ILI9341_WriteString(8, 152, line, Font_7x10, ILI9341_YELLOW,
    ILI9341_BLACK);

    (void)snprintf(line, sizeof(line), "ROLL :%s deg", r1);
    ILI9341_WriteString(8, 166, line, Font_7x10, ILI9341_YELLOW,
    ILI9341_BLACK);

    (void)snprintf(line, sizeof(line), "PITCH:%s deg", p1);
    ILI9341_WriteString(8, 180, line, Font_7x10, ILI9341_YELLOW,
    ILI9341_BLACK);

    (void)snprintf(line, sizeof(line), "YAW :%s deg", y1);
    ILI9341_WriteString(8, 194, line, Font_7x10, ILI9341_YELLOW,
    ILI9341_BLACK);

}

static void Yaw_UpdateFromGyroZ(const MPU6050_t *m)
{
    uint32_t now = HAL_GetTick();

    if (g_yaw_last_ms == 0)
    {
        g_yaw_last_ms = now;
        return;
    }

    double dt = (double)(now - g_yaw_last_ms) / 1000.0;
    g_yaw_last_ms = now;

    // Інтегруємо Gz (deg/s) щоб отримати градуси
    g_yaw_deg += m->Gz * dt;

    // Нормалізація до [-180; +180)
    while (g_yaw_deg >= 180.0) g_yaw_deg -= 360.0;
    while (g_yaw_deg < -180.0) g_yaw_deg += 360.0;
}

/* USER CODE END 4 */

```

2.5. Додати прототипи даних функцій у відповідному блоці:

```

/* USER CODE BEGIN PFP */
static void fmt_milli3(char *out, size_t out_sz, int32_t milli);
static void fmt_deci1(char *out, size_t out_sz, int32_t deci);

static void UI_DrawStatic(void);
static void UI_UpdateValues(const MPU6050_t *m);
static void Yaw_UpdateFromGyroZ(const MPU6050_t *m);
/* USER CODE END PFP */

```

2.6. Написати код для ініціалізації дисплею, модуля MPU6050 та інтерфейсу:

```
/* USER CODE BEGIN 2 */
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_SET);
ILI9341_Init();
ILI9341_FillScreen(ILI9341_BLACK);

// Ініціалізація модуля MPU6050
uint8_t r = MPU6050_Init(&hi2c2);
if (r != 0)
{
    ILI9341_WriteString(10, 90, "MPU6050: FAIL", Font_11x18,
ILI9341_RED, ILI9341_BLACK);
    while (1) { HAL_Delay(1000); }
}

// Створення інтерфейсу
UI_DrawStatic();

/* USER CODE END 2 */
```

2.7. В головному циклі написати алгоритм опитування модуля та оновлення показань на дисплеї:

```
/* USER CODE BEGIN 3 */
    MPU6050_Read_All(&hi2c2, &MPU6050);
    UI_UpdateValues(&MPU6050);
    HAL_Delay(30);
}

/* USER CODE END 3 */
```

2.8. Завантажити програму до мікроконтролера та перевірити правильність її роботи. Відслідкувати зміни показань модуля при поворотах датчика у просторі.

3. Оновити програму таким чином, щоб реалізувати найпростіший авіагоризонт. Лінія горизонту повинна рухатись вертикально залежно від кута тангажу (Pitch). Фіксований умовний символ літака повинен завжди залишатися в геометричному центрі екрану. Внизу екрану повинні виводитись значення Pitch та Roll. Під основною зоною повинна розміщуватись проста шкала крену

(Roll). Зверху по центру повинно виводитись поточне значення курсу (Yaw). Передбачити можливість керування напрямком осей та діапазонами значень, а також обирати формат відображення курсу ($0 \dots 360^\circ$ або $-180^\circ \dots +180^\circ$).

3.1. Підключити бібліотеки для обчислень та роботи з рядковими даними:

```
/* USER CODE BEGIN Includes */
#include <stdio.h>
#include "fonts.h"
#include "ili9341_dma.h"
#include "mpu6050.h"
#include <math.h>
#include <string.h>

/* USER CODE END Includes */
```

3.2. Оголосити константи зі значеннями геометричних параметрів дисплею та інтерфейсу, а також кольорів:

```
/* USER CODE BEGIN PD */
#define SCR_W    ILI9341_WIDTH
#define SCR_H    ILI9341_HEIGHT

#define TOP_H      24
#define BOTTOM_H   48

#define MAIN_Y      (TOP_H)
#define MAIN_H      (SCR_H - TOP_H - BOTTOM_H)
#define MAIN_Y0     (MAIN_Y)
#define MAIN_Y1     (MAIN_Y + MAIN_H - 1)

#define CX          (SCR_W / 2)
#define CY          (MAIN_Y + MAIN_H / 2)

static const uint16_t COL_DARK      = ILI9341_COLOR565(0x0A, 0x0A, 0x0C);
static const uint16_t COL_SKY      = ILI9341_COLOR565(0x80, 0xC0, 0xFF);
static const uint16_t COL_GROUND   = ILI9341_COLOR565(0x3F, 0x2A, 0x0F);
static const uint16_t COL_HORIZON  = ILI9341_COLOR565(0xFF, 0xFF, 0x80);
static const uint16_t COL_PLANE    = ILI9341_COLOR565(0xFF, 0xFF, 0x00);
static const uint16_t COL_TEXT     = ILI9341_WHITE;
static const uint16_t COL_TICK     = ILI9341_COLOR565(0xC0, 0xC0, 0xC0);

#define HORIZON_THICK 5
#define PLANE_THICK   4

#define ROW_SET565(rowp, x, col) do { \
    (rowp)[2*(x) + 0] = (uint8_t)((col) >> 8); \
    (rowp)[2*(x) + 1] = (uint8_t)((col) & 0xFF); \
} while (0)
/* USER CODE END PD */
```

3.3. Оновити блок з оголошенням змінних, створивши деякі нові:

- *g_yaw_offset_deg* – кутова поправка (у градусах) для “повороту нуля” курсу під конкретний монтаж/орієнтацію датчика;
- *g_yaw_dir_sign* – множник напрямку для Yaw, що визначає, який знак матиме поворот;
- *g_yaw_show_signed_delta* – режим відображення Yaw. Якщо 1, то значення буде показувати відхилення від опорного курсу з відповідним знаком (–180...+180), якщо 0 – показувати абсолютний курс (0...359).;
- *g_heading_ref_deg* – опорний (базовий) курс у градусах, відносно якого рахується відхилення при повороті;
- *g_heading_ref_set* – перемикач напрямків осей; 1 – поміняти місцями значення Pitch і Roll, 0 – залишити як є;
- *g_roll_sign* – інверсія знаку Roll для узгодження з конкретним монтажем датчику/системою координат;
- *g_pitch_sign* – інверсія знаку Pitch для узгодження з конкретним монтажем датчику/системою координат.

Реалізація в коді:

```
/* USER CODE BEGIN PV */
// Зсув курсу (наприклад, 90, якщо при іншому монтажі датчика “нуль” має
// бути повернутий)
static int16_t g_yaw_offset_deg = 0;
// +1: правий поворот збільшує курс, -1: лівий поворот збільшує курс
static int8_t g_yaw_dir_sign = -1;
// 1: показувати знакове відхилення від опорного курсу (–180...+180), 0:
// показувати абсолютний курс (0...359)
static uint8_t g_yaw_show_signed_delta = 1;
// Опорний курс (встановлюється один раз або вручну)
static int16_t g_heading_ref_deg = 0;
static uint8_t g_heading_ref_set = 0;

// Напрямок осей - 0: Roll=X, Pitch=Y, 1: поміняти місцями
static uint8_t g_swap_roll_pitch = 1;
// Інверсія знаків Roll та Pitch (під конкретний монтаж датчика)
static int8_t g_roll_sign = -1;
static int8_t g_pitch_sign = -1;
/* USER CODE END PV */
```

3.4. В блоці *USER CODE BEGIN 4* створити функції для малювання інтерфейсу та графіків, опрацювання даних з датчиків та деякі допоміжні функції. Призначення функцій наступне:

- *UI_DrawStatic(void)* – малює статичний фон інтерфейсу, очищає екран, формує верхню й нижню панелі та статичні підписи.
- *UI_UpdateValues(const MPU6050_t *m)* – зчитує Roll/Pitch зі структури датчика, оновлює Yaw, формує рядок Yaw у верхній зоні, перемальовує горизонт і виводить Pitch/Roll як цілі значення.
- *Yaw_UpdateFromGyroZ(const MPU6050_t *m)* – інтегрує кутову швидкість Gz в кут Yaw з урахуванням часу між вимірами та нормалізує результат у діапазон 0...359°.
- *fmt_heading3(char *out, size_t out_sz, int heading_0_359)* – форматує курс у вигляді тризначного числа з нулями зліва.
- *center_x_for_text(const char *s, FontDef f)* – обчислює координату X для центрування рядка тексту на екрані за шириною шрифту.
- *overlay_aircraft_row(uint8_t *rowp, int ys)* – накладає на заданий рядок буфера «хрестик літака» (вертикальна риска та крила) у фіксованому положенні.
- *overlay_roll_scale_row(uint8_t *rowp, int ys, float roll_disp_deg)* – накладає на рядок нижню шкалу крену, що містить базову лінію, мітки та показчик поточного крену.
- *draw_horizon(float pitch_deg, float roll_deg)* – генерує центральну область (небо/земля/лінія горизонту) з урахуванням Pitch/Roll і відправляє пікселі на дисплей блоками.
- *map_pitch_roll(float pitch_in, float roll_in, float *pitch_out, float *roll_out)* – застосовує перестановку осей та інверсії знаків для Pitch/Roll відповідно до конкретного монтажу датчика.
- *norm360_i16(int16_t a)* – нормалізує кут до діапазону 0...359°.
- *wrap180_i16(int16_t a)* – нормалізує кут до діапазону -180...+180°.

- *fmt_yaw_signed(char *out, size_t out_sz, int16_t yaw_deg)* – приводить значення Yaw до фіксованої довжини.

Реалізація в коді:

```
/* USER CODE BEGIN 4 */
static void UI_DrawStatic(void)
{
    ILI9341_FillScreen(COL_DARK);

    // Верхня зона
    ILI9341_FillRectangle(0, 0, SCR_W, TOP_H, COL_DARK);

    // Нижня зона
    ILI9341_FillRectangle(0, SCR_H - BOTTOM_H, SCR_W, BOTTOM_H,
COL_DARK);

    // Центральна зона намалюється в UI_UpdateValues)

    // Статичні підписи внизу
    ILI9341_WriteString(10, SCR_H - BOTTOM_H + 4, "PITCH", Font_7x10,
COL_TEXT, COL_DARK);
    ILI9341_WriteString(210, SCR_H - BOTTOM_H + 4, "ROLL", Font_7x10,
COL_TEXT, COL_DARK);
}

static void UI_UpdateValues(const MPU6050_t *m)
{
    // Roll/Pitch беремо з наявних полів структури (як і раніше)
    float roll_raw = m->KalmanAngleX;
    float pitch_raw = m->KalmanAngleY;
    float roll_deg, pitch_deg;
    map_pitch_roll(pitch_raw, roll_raw, &pitch_deg, &roll_deg);

    // Обчислюємо Yaw
    Yaw_UpdateFromGyroZ(m);

    // 1) Абсолютний курс 0..359 з урахуванням зсуву
    int16_t heading_abs = (int16_t)(g_yaw_deg + (g_yaw_deg >= 0 ? 0.5f :
-0.5f));

    heading_abs = norm360_i16((int16_t)(heading_abs + g_yaw_offset_deg));

    // 2) Ініціалізація опорного курсу (якщо ще не встановлено)
    if (!g_heading_ref_set)
    {
```

```

        g_heading_ref_deg = heading_abs;
        g_heading_ref_set = 1;
    }

    // Верхня зона - виведення курсу
    char hdg[12];

    if (g_yaw_show_signed_delta)
    {
        // знакове відхилення від опорного курсу
        int16_t yaw_delta = wrap180_i16((int16_t)(heading_abs -
g_heading_ref_deg));
        fmt_yaw_signed(hdg, sizeof(hdg), yaw_delta);
    }
    else
    {
        // абсолютний курс
        fmt_heading3(hdg, sizeof(hdg), heading_abs);
    }

    uint16_t xh = center_x_for_text(hdg, Font_11x18);
    ILI9341_WriteString(xh, 3, hdg, Font_11x18, COL_TEXT, COL_DARK);

    // Центральна зона - небо/земля/горизонт і символ літака
    draw_horizon(pitch_deg, roll_deg);

    // Pitch і Roll як цілі числа
    int pitch_int = (int)(pitch_deg + (pitch_deg >= 0 ? 0.5f : -0.5f));
    int roll_int = (int)(roll_deg + (roll_deg >= 0 ? 0.5f : -0.5f));

    char line[32];

    // Лівий блок (Pitch)
    (void)snprintf(line, sizeof(line), "%+3d deg ", pitch_int);
    ILI9341_WriteString(10, SCR_H - BOTTOM_H + 18, line, Font_11x18,
COL_TEXT, COL_DARK);

    // Правий блок (Roll)
    (void)snprintf(line, sizeof(line), "%+3d deg ", roll_int);
    ILI9341_WriteString(210, SCR_H - BOTTOM_H + 18, line, Font_11x18,
COL_TEXT, COL_DARK);

}

static void Yaw_UpdateFromGyroZ(const MPU6050_t *m)
{
    uint32_t now = HAL_GetTick();
    if (g_yaw_last_ms == 0)
    {
        g_yaw_last_ms = now;
    }
}

```



```

        return;
    }

    float dt = (float)(now - g_yaw_last_ms) / 1000.0f;
    g_yaw_last_ms = now;

    // Інтегруємо Gz (deg/s) щоб отримати градуси, з урахуванням напрямку
    g_yaw_deg += (float)g_yaw_dir_sign * m->Gz * dt;

    // Нормалізація yaw у [0; 360)
    while (g_yaw_deg >= 360.0f) g_yaw_deg -= 360.0f;
    while (g_yaw_deg < 0.0f) g_yaw_deg += 360.0f;
}

static void fmt_heading3(char *out, size_t out_sz, int heading_0_359)
{
    if (heading_0_359 < 0) heading_0_359 = 0;
    if (heading_0_359 > 359) heading_0_359 = 359;
    (void)sprintf(out, out_sz, "%03d", heading_0_359);
}

static uint16_t center_x_for_text(const char *s, FontDef f)
{
    uint16_t w = (uint16_t)(strlen(s) * f.width);
    if (w >= SCR_W) return 0;
    return (uint16_t)((SCR_W - w) / 2);
}

static void overlay_aircraft_row(uint8_t *rowp, int ys)
{
    // Геометрія хрестика
    const int wing = 96;
    const int vlen = 46;
    const int t = PLANE_THICK;

    const int t0 = (t - 1) / 2; // для парної/непарної товщини
    const int t1 = t / 2;

    const int yv0 = CY - vlen/2;
    const int yv1 = CY + vlen/2;

    const int yh0 = CY - t0;
    const int yh1 = CY + t1;

    const int xv0 = CX - t0;
    const int xv1 = CX + t1;

    const int xw0 = CX - wing/2;
    const int xw1 = CX + wing/2;

    // вертикальна лінія

```

```

    if (ys >= yv0 && ys <= yv1)
    {
        int xa = xv0, xb = xv1;
        if (xa < 0) xa = 0;
        if (xb > (int)SCR_W - 1) xb = (int)SCR_W - 1;

        for (int x = xa; x <= xb; x++)
            ROW_SET565(rowp, x, COL_PLANE);
    }

    // горизонтальна лінія (крила)
    if (ys >= yh0 && ys <= yh1)
    {
        int xa = xw0, xb = xw1;
        if (xa < 0) xa = 0;
        if (xb > (int)SCR_W - 1) xb = (int)SCR_W - 1;

        for (int x = xa; x <= xb; x++)
            ROW_SET565(rowp, x, COL_PLANE);
    }
}

static void overlay_roll_scale_row(uint8_t *rowp, int ys, float
roll_disp_deg)
{
    // Параметри шкали
    const int W = 220;
    const int H = 24;

    const int scale_y0 = MAIN_Y1 - H + 1;
    const int scale_y1 = MAIN_Y1;

    if (ys < scale_y0 || ys > scale_y1)
        return;

    const int x0 = CX - W/2;
    const int x1 = x0 + W - 1;

    const float max_ang = 60.0f;

    // обмеження
    if (roll_disp_deg > 60.0f) roll_disp_deg = 60.0f;
    if (roll_disp_deg < -60.0f) roll_disp_deg = -60.0f;

    const int base_y = scale_y0 + (H/2);

    // товщина базової лінії 2 px
    if (ys == base_y || ys == base_y + 1)
    {
        int xa = x0, xb = x1;
        if (xa < 0) xa = 0;
        if (xb > (int)SCR_W - 1) xb = (int)SCR_W - 1;
    }
}

```

```

        for (int x = xa; x <= xb; x++)
            ROW_SET565(rowp, x, COL_TICK);
    }

    // мітки
    const int angles[7] = {-60, -45, -30, 0, 30, 45, 60};
    for (int i = 0; i < 7; i++)
    {
        int a = angles[i];
        int xt = CX + (int)((float)a * (float)(W/2) / max_ang);

        int th = (a == 0) ? 10 : 6;
        int y0t = base_y - th;
        int y1t = base_y - 1;

        if (ys >= y0t && ys <= y1t)
        {
            for (int x = xt - 1; x <= xt + 1; x++)
            {
                if (x >= 0 && x < (int)SCR_W)
                    ROW_SET565(rowp, x, COL_TICK);
            }
        }
    }

    // покажчик
    int xp = CX + (int)(roll_disp_deg * (float)(W/2) / max_ang);
    int y0p = base_y - 12;
    int y1p = base_y - 2;

    if (ys >= y0p && ys <= y1p)
    {
        for (int x = xp - 2; x <= xp + 2; x++)
        {
            if (x >= 0 && x < (int)SCR_W)
                ROW_SET565(rowp, x, COL_PLANE);
        }
    }
}

static void draw_horizon(float pitch_deg, float roll_deg)
{
    // Скільки рядків відправляється одним блоком
    enum { BLOCK_ROWS = 16 };

    // 320 * 2 * 8 = 5120 байт
    static uint8_t blkbuf[SCR_W * 2 * BLOCK_ROWS];

    // Кольори у форматі старший/молодший байт
    const uint8_t sky_hi = (uint8_t)(COL_SKY >> 8);

```

```

const uint8_t sky_lo      = (uint8_t)(COL_SKY & 0xFF);
const uint8_t gr_hi       = (uint8_t)(COL_GROUND >> 8);
const uint8_t gr_lo       = (uint8_t)(COL_GROUND & 0xFF);
const uint8_t hz_hi       = (uint8_t)(COL_HORIZON >> 8);
const uint8_t hz_lo       = (uint8_t)(COL_HORIZON & 0xFF);

// pitch_deg, roll_deg – КУТИ ЛІТАКА.
// Для горизонту (земля/небо) треба інверсія, бо фон рухається
навпаки.
float pitch_air = pitch_deg;
float roll_air  = roll_deg;

// обмеження ДО геометричних перетворень
if (roll_air > 90.0f) roll_air = 90.0f;
if (roll_air < -90.0f) roll_air = -90.0f;
if (pitch_air > 90.0f) pitch_air = 90.0f;
if (pitch_air < -90.0f) pitch_air = -90.0f;

float pitch_geom = -pitch_air;
float roll_geom  = -roll_air;

// Pitch
float px_per_deg = (float)(MAIN_H / 2) / 90.0f;
float y0 = (float)CY - pitch_geom * px_per_deg;

float roll_rad = roll_geom * 0.01745329252f;
float k = tanf(roll_rad);

// Починаємо потоковий запис у центральну область
ILI9341_BeginWrite(0, MAIN_Y0, SCR_W, MAIN_H);

int y = MAIN_Y0;

// Гілка roll ~ 0 - горизонт майже рівний
if (fabsf(k) < 0.01f)
{
    int yh = (int)(y0 + (y0 >= 0 ? 0.5f : -0.5f)); // округлення
    int yb0 = yh - (HORIZON_THICK / 2);
    int yb1 = yb0 + (HORIZON_THICK - 1);

    while (y <= MAIN_Y1)
    {
        int rows = MAIN_Y1 - y + 1;
        if (rows > BLOCK_ROWS) rows = BLOCK_ROWS;

        for (int r = 0; r < rows; r++)
        {
            int yy = y + r;
            uint8_t *rowp = &blkbuf[r * (SCR_W * 2)];

            // Вибір фону для цього рядка
            uint8_t hi, lo;

```

```

        if (yy >= yb0 && yy <= yb1)
        {
            hi = hz_hi; lo = hz_lo;        // товста смуга горизонту
        }
        else if (yy >= yh)
        {
            hi = gr_hi; lo = gr_lo;        // земля
        }
        else
        {
            hi = sky_hi; lo = sky_lo;      // небо
        }

        // Заповнити весь рядок одним кольором
        for (int x = 0; x < (int)SCR_W; x++)
        {
            rowp[2 * x + 0] = hi;
            rowp[2 * x + 1] = lo;
        }

        overlay_roll_scale_row(rowp, yy, roll_air);
        overlay_aircraft_row(rowp, yy);
    }

    ILI9341_PushPixels(blkbuf, (uint32_t)(rows * (SCR_W * 2)));
    y += rows;
}

ILI9341_EndWrite();
return;
}

// Загальний випадок - похила лінія
float invk = 1.0f / k;

// xf для першого рядка; далі будемо додавати invk при переході на
наступний y
float xf = (float)CX + ((float)MAIN_Y0 - y0) * invk;

while (y <= MAIN_Y1)
{
    int rows = MAIN_Y1 - y + 1;
    if (rows > BLOCK_ROWS) rows = BLOCK_ROWS;

    for (int r = 0; r < rows; r++)
    {
        int yy = y + r;
        uint8_t *rowp = &blkbuf[r * (SCR_W * 2)];

        // 1) Спочатку весь рядок - це небо
    }
}

```

```

for (int x = 0; x < (int)SCR_W; x++)
{
    rowp[2 * x + 0] = sky_hi;
    rowp[2 * x + 1] = sky_lo;
}

// Обчислюємо положення лінії на цьому рядку
int xb = (int)(xf + (xf >= 0 ? 0.5f : -0.5f));
xf += invk;

// Земля ліворуч або праворуч
int gx0, gx1;
if (k > 0.0f)
{
    gx0 = 0;
    gx1 = xb - 1;
}
else
{
    gx0 = xb;
    gx1 = (int)SCR_W - 1;
}

if (gx0 < 0) gx0 = 0;
if (gx1 > (int)SCR_W - 1) gx1 = (int)SCR_W - 1;

if (gx0 <= gx1)
{
    for (int x = gx0; x <= gx1; x++)
    {
        rowp[2 * x + 0] = gr_hi;
        rowp[2 * x + 1] = gr_lo;
    }
}

// “Горизонт” - це невеликий сегмент навколо xb
int xh0 = xb - (HORIZON_THICK / 2);
int xh1 = xb + (HORIZON_THICK / 2);

if (xh0 < 0) xh0 = 0;
if (xh1 > (int)SCR_W - 1) xh1 = (int)SCR_W - 1;

if (xh0 <= xh1)
{
    for (int x = xh0; x <= xh1; x++)
    {
        rowp[2 * x + 0] = hz_hi;
        rowp[2 * x + 1] = hz_lo;
    }
}

overlay_roll_scale_row(rowp, yy, roll_air);

```

```

        overlay_aircraft_row(rowp, yy);
    }

    ILI9341_PushPixels(blkbuf, (uint32_t)(rows * (SCR_W * 2)));
    y += rows;
}

ILI9341_EndWrite();
}

static void map_pitch_roll(float pitch_in, float roll_in, float
*pitch_out, float *roll_out)
{
    float p = pitch_in;
    float r = roll_in;

    if (g_swap_roll_pitch)
    {
        float tmp = p;
        p = r;
        r = tmp;
    }

    *pitch_out = (float)g_pitch_sign * p;
    *roll_out = (float)g_roll_sign * r;
}

static int16_t norm360_i16(int16_t a)
{
    while (a >= 360) a -= 360;
    while (a < 0) a += 360;
    return a;
}

// Переводить у діапазон [-180; +180]
static int16_t wrap180_i16(int16_t a)
{
    while (a > 180) a -= 360;
    while (a < -180) a += 360;
    return a;
}

// Формат: +015 або -015 (стала довжина 4 символи)
static void fmt_yaw_signed(char *out, size_t out_sz, int16_t yaw_deg)
{
    if (yaw_deg > 180) yaw_deg = 180;
    if (yaw_deg < -180) yaw_deg = -180;
    (void)sprintf(out, out_sz, "%+4d", (int)yaw_deg);
}

/* USER CODE END 4 */

```

3.5. Додати прототипи даних функцій у відповідному блоці:

```
/* USER CODE BEGIN PFP */
static void fmt_deci1(char *out, size_t out_sz, int32_t deci);
static void UI_DrawStatic(void);
static void UI_UpdateValues(const MPU6050_t *m);
static void Yaw_UpdateFromGyroZ(const MPU6050_t *m);

static void fmt_heading3(char *out, size_t out_sz, int heading_0_359);
static uint16_t center_x_for_text(const char *s, FontDef f);
static void draw_horizon(float pitch_deg, float roll_deg);

static void overlay_aircraft_row(uint8_t *rowp, int ys);
static void overlay_roll_scale_row(uint8_t *rowp, int ys, float
roll_disp_deg);

static void map_pitch_roll(float pitch_in, float roll_in, float
*pitch_out, float *roll_out);

static int16_t norm360_i16(int16_t a);
static int16_t wrap180_i16(int16_t a);
static void fmt_yaw_signed(char *out, size_t out_sz, int16_t yaw_deg);
/* USER CODE END PFP */
```

3.6. Модифікувати код в блоці ініціалізації наступним чином:

```
/* USER CODE BEGIN 2 */
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_SET);
ILI9341_Init();
ILI9341_FillScreen(ILI9341_BLACK);

uint8_t r = MPU6050_Init(&hi2c2);
if (r != 0)
{
    ILI9341_WriteString(10, 90, "MPU6050: FAIL", Font_11x18,
ILI9341_RED, ILI9341_BLACK);
    while (1) { HAL_Delay(1000); }
}

UI_DrawStatic();
g_yaw_deg = 0.0;
g_yaw_last_ms = 0;

/* USER CODE END 2 */
```

3.7. В головному циклі код залишається без змін відносно попереднього завдання.

3.8. Завантажити програму до мікроконтролера та перевірити правильність її роботи. У випадку виявлення невідповідності між рухом модуля MPU6050 та показаннями на дисплеї можна використати описані раніше налаштування в блоці *USER CODE BEGIN PV*, щоб скоригувати напрямки осей та діапазони показань.

Індивідуальні завдання

1. В даній роботі всі бригади виконують одне завдання. Необхідно створити систему попередження, яка буде сигналізувати про вихід показників Pitch або Roll за межі допустимих значень (задаються програмно). Попередження повинно максимально привертати до себе увагу оператора. Буде оцінюватись креативність та технічна глибина запропонованого рішення, яке у кожній бригади має бути унікальним.

Контрольні запитання

1. Ви помічаєте, що курс (Yaw) «пливе» навіть коли модуль лежить нерухомо. Який експеримент ви запропонуєте, щоб відокремити дрейф нуля гіроскопа від похибки кроку часу dt , і як на основі цього компенсуєте показання?
2. У коді dt для визначення курсу береться з `HAL_GetTick()`. Які типові джерела похибки для такого вимірювання часу?
3. Уявіть, що модуль встановлено з невеликим перекосом (неідеально паралельно корпусу). Як це проявиться на авіагоризонті під час чистого повороту по курсу, і яку математичну корекцію ви б додали, щоб мінімізувати «паразитний» Pitch/Roll?
4. Чому інтегрування лише G_z є принципово недостатнім для стабільного курсу в довгій перспективі? Запропонуйте мінімальне апаратне/алгоритмічне

доповнення, яке дасть «абсолютну» прив'язку курсу, і опишіть основні переваги цього доповнення.

5. Якщо пристрій працює у середовищі з вібраціями (наприклад, на рухомій платформі), які саме симптоми ви очікуєте побачити в Roll/Pitch на дисплеї, і який підхід ви оберете для підвищення стійкості?
6. Поясніть, як би ви визначили та компенсували температурну залежність дрейфу гіроскопа, маючи лише покази температури з MPU6050 і можливість довгого логування (без додаткових датчиків).
7. Запропонуйте спосіб «автоматичної установки нуля» для Pitch і Roll (не лише для курсу) так, щоб користувач міг поставити модуль у довільне положення, натиснути кнопку (або подати команду), і далі авіагоризонт працював відносно цього положення.
8. Де у вашій реалізації потенційно найбільші витрати часу/пам'яті під час малювання горизонту? Запропонуйте способи оптимізації, які не змінюють вигляд інтерфейсу і не змінюють поточний принцип «малювання».
9. Якщо зчитування MPU6050 інколи повертає «правдоподібні», але помилкові значення (сплески), як би ви відрізняли це від реальних швидких рухів? Запропонуйте критерії відсікання та поясніть ризики помилкового відсікання.

СПИСОК РЕКОМЕНДОВАНОЇ ЛІТЕРАТУРИ